电商销量预测系统 · 开发技术文档

前端 React · 后端 FastAPI · 数据库 SQLite

目 录

0. 文档信息与阅读指南 7

0.1 文档信息 7

0.2 修订记录 7

0.3 本文档回答什么问题 8

0.4 本文档与既有文档的关系（📌 冲突仲裁规则） 8

0.5 阅读路径建议 9

0.6 编号体系 9

0.7 标记说明 10

0.8 配套代码资产 12

1. 技术选型与版本锁定 13

1.1 选型总表 13

1.2 后端依赖清单（requirements.txt） 15

1.3 前端依赖清单（package.json） 16

1.4 版本锁定的强制理由 17

1.5 已实测的环境事实 ⏱ 18

1.6 设计系统移植方案（React 落地） 20

1.7 实现时必须删除的原型痕迹（UX-GOV-01）🚧 21

1.8 技术选型风险与应对 22

2. 工程结构与开发环境 23
2.1 仓库总体结构 23
2.2 后端目录结构与模块职责 24
2.3 前端目录结构 26
2.4 环境变量清单 27
2.5 本地开发环境搭建 29
2.6 常用命令速查 31
2.7 端口与进程约定 32
2.8 文件存储与磁盘策略 32
3. 数据库设计 33
3.1 设计原则 33
3.2 实体关系 34
3.3 表结构详细设计 35
3.4 索引设计 43
3.5 约束与触发器 44
3.6 初始化数据 44
3.7 典型查询与性能考量 45
3.8 数据清理策略 46
3.9 SQLite 配置与并发处理 46
4. 后端设计与实现 47
4.1 应用启动与生命周期 48
4.2 中间件链 49
4.3 依赖注入 50
4.4 认证与授权实现 53
4.5 统一异常处理 57
4.6 日志规范 58
4.7 算法层调用契约（★ 核心章节） 59

4.8 预测任务编排实现（services/predict_service.py） 64
4.9 上传与校验实现（services/dataset_service.py + app/validators/） 66
4.10 导出实现（services/export_service.py） 69
4.11 埋点实现（core/telemetry.py） 70
5. 接口设计 70
5.1 通用约定 71
5.2 错误码全表（FR-SYS-02） 72
5.3 接口清单总览（21 个既有 + 9 个管理端 = 30 个） 74
5.4 认证类接口（API-01 ~ API-05） 77
API-01 用户注册 77
API-02 用户登录 79
API-03 用户登出 81
API-04 获取当前用户信息 82
API-05 修改个人信息 / 密码 83
5.5 数据集接口（API-06 ~ API-11） 84
API-06 上传 CSV 数据集 84
API-07 加载内置示例数据 87
API-08 下载标准 CSV 模板 89
API-09 获取数据集详情 NEW 89
API-10 数据预览 91
API-11 下载错误明细 NEW 91
5.6 预测任务接口（API-12 ~ API-16） 92
API-12 创建预测任务 93
API-13 查询任务状态 95
API-14 获取预测结果 ★ 96
API-15 结果明细分页 NEW 100
API-16 导出预测结果 102

5.7 历史记录接口（API-17 ~ API-20）.....104
API-17 历史任务列表.....104
API-18 跨任务曲线对比 🆕.....105
API-19 历史任务详情.....107
API-20 删除历史任务（软删除）.....108
5.8 系统接口（API-21）.....109
API-21 健康检查.....109
6. 前端设计与实现.....110
6.1 应用结构与 Provider 组装.....110
6.2 状态管理设计（Zustand）.....111
6.3 接口调用与数据流.....114
6.4 主题与令牌落地.....115
6.5 图表实现（ECharts）.....117
6.6 表单与校验.....119
6.7 表格与分页.....121
6.8 错误处理与用户提示.....122
6.9 页面状态规范.....122
6.10 性能优化.....123
7. 前后端协作与联调流程.....123
7.1 契约优先原则（JT-05）.....123
7.2 开发顺序与依赖（📌 关键路径）.....124
7.3 Mock 策略（前后端并行开发）.....125
7.4 联调检查清单（JT-09）.....126
7.5 错误码映射（前后端一致，JT-10）.....127
7.6 跨端一致性自动校验（JT-11）.....128
8. 开发流程与工程规范.....128
8.1 分支策略.....128

8.2 提交规范.....129
8.3 代码规范.....130
8.4 代码评审检查单（REV-1）.....132
8.5 提交前自检（GIT-6）.....133
8.6 变更管理.....134
8.7 迭代节奏建议.....135
9. 测试方案.....135
9.1 测试分层.....135
9.2 算法层测试（已有资产）.....136
9.3 集成测试要点.....136
9.4 边界与异常用例（🔒 重点）.....137
9.5 服务端测试用例清单.....139
9.6 验收检查清单（交付前逐项确认）.....139
9.7 性能测试.....140
10. 部署与运行.....142
10.1 部署形态.....142
10.2 环境准备.....142
10.3 Nginx 配置.....143
10.4 Swagger UI 资源离线化（国内网络必需）.....144
10.5 启动与健康检查.....145
10.6 目录与磁盘.....146
10.7 安全加固清单.....146
11. 需求追溯与验收映射.....147
11.1 功能需求追溯矩阵.....147
11.2 非功能需求追溯.....149
11.3 上游文档交叉引用.....150
附录 A：接口字段数据字典.....151

A.1 DatasetSummary 数据集概要 151
A.2 MetricBundle 指标集 152
A.3 Coverage 覆盖率 152
A.4 Distribution 逐序列 MAPE 分布 153
A.5 TimingMs 分阶段耗时 153
A.6 SeriesCurve 序列曲线 154
A.7 PredictionRow 明细行 154
A.8 HistoryItem 历史列表项 155
A.9 TaskStatusResponse 任务状态 155
附录 B：待确认技术问题清单（TQ-*） 155
附录 C：本文档对上游的增量汇总 157
C.1 口径纠正（3 项，见 §0.7.1） 157
C.2 接口增量（4 个，见 §5.3） 157
C.3 设计细化（把「逻辑模型」落到「DDL」） 158
C.4 技术约束补充（开发方案未覆盖） 158
附录 D：配套资产与校验证据 159
D.1 配套文件清单 160
D.2 校验脚本 160
D.3 本文档引用的实测数据来源 161
D.4 文档与代码的同步责任 161

0. 文档信息与阅读指南

0.1 文档信息

项	内容
文档名称	电商销量预测系统 · 开发技术文档
版本号	V1.0
文档状态	已定稿，可据此进入编码
适用范围	本项目全部前后端编码、联调、测试与部署工作
上游依据	《PRD V1.0》《开发方案文档 V1.1》《UI/UX 设计文档 V1.0》《项目任务书 V1.0.0》
配套代码资产	`code/db/schema.sql`、`code/db/seed.sql`、`code/backend/schemas.py`、`code/frontend/types.ts`、`code/frontend/api-client.ts`
校验脚本	`scripts/verify_schema.py`、`scripts/verify_schemas.py`、`scripts/verify_types.mjs`

0.2 修订记录

版本	日期	修订人	修订内容
V1.0	2026-10-09	技术负责人	首版。基于 PRD / 开发方案 / UI/UX 三份文档落地到可编码粒度：数据库 DDL、20 个接口全量 schema、前后端工程实现规范、协作流程、测试与部署方案

0.3 本文档回答什么问题

本文档只回答一个问题：**怎么做。**

它把上游文档中的「需求」与「设计」翻译成可以直接写代码的规格：表结构、字段类型、接口报文、目录结构、命名规范、联调顺序。**本文档不含需求决策与界面设计决策**——那两项分别以 PRD 与 UI/UX 文档为准。

0.4 本文档与既有文档的关系（📌 冲突仲裁规则）

文档	回答什么	颗粒度示例	谁看
项目任务书 V1.0.0	交付什么	六大功能模块、技术栈、时间要求	全体
PRD V1.0	**做什么 / 做到什么程度算完成**	`FR-UPL-03`：单文件 ≤ 20MB、≤ 500,000 行	产品、研发、测试
开发方案文档 V1.1	**怎么想 / 为什么这么选**	架构分层、模块划分、逻辑数据模型、技术风险	负责人、评审
UI/UX 设计文档 V1.0	**长什么样 / 怎么交互**	56 个设计令牌、11 页逐页规格、无障碍标准	设计、前端
本文档（开发技术文档 V1.0）	**怎么做**	`tasks` 表建表语句、`POST /predictions` 请求响应报文、`algo/` 函数调用契约	前端、后端、测试

📌 冲突仲裁规则（三级优先）

1. **功能范围与验收标准**：以 **PRD V1.0** 为准。本文档若与 PRD 的 `FR-*` / `NFR-*` / `AC-*` 冲突，一律以 PRD 为准。
2. **技术口径与实现选择**：以 **本文档** 为准。本文档是 PRD 与开发方案在实现层的**最终收敛点**——当上游文档存在表述不一致时，以本文档的确定值为准并回写上游（见 §0.7 冲突纠正清单）。

3. 界面与视觉：以 **UI/UX 设计文档 V1.0** 为准。本文档只负责把设计令牌搬进 React 工程，不改变任何视觉取值。

0.5 阅读路径建议

角色	建议路径	重点章节
后端开发	§1 → §2 → §3 → §4 → §5	数据库 DDL、接口 schema、`algo/` 调用契约
前端开发	§1 → §2 → §5 → §6 → §7	接口报文、TS 类型、路由与状态管理、联调顺序
测试	§5 → §9 → §11	接口边界、测试用例、需求追溯矩阵
负责人 / 评审	§0 → §1 → §7 → §9 → §11	选型依据、协作流程、验收映射
部署运维	§2 → §10	环境变量、启动方式、健康检查

0.6 编号体系

本文档延续 PRD 与 UI/UX 文档的编号习惯，采用「域前缀 + 序号」：

前缀	含义	示例
`DB-<表>-<序号>`	数据库设计项	`DB-TASKS-03` 任务状态字段
`API-<序号>`	接口项（按 §5 顺序编号，01–20）	`API-12` 创建预测任务
`BE-<序号>`	后端实现规范项	`BE-07` 异常处理中间件
`FE-<序号>`	前端实现规范项	`FE-05` 状态管理约定
`JT-<序号>`	前后端契约项	`JT-02` 错误码映射
`TQ-<序号>`	待确认技术问题	`TQ-1` 是否启用 Redis 缓存

交叉引用格式：`PRD §3.2.2`（章节）、`FR-UPL-03`（需求编号）、`UX-P05`（页面编号）、`NFR-PERF-01`（非功能需求）。

0.7 标记说明

标记	含义
🔒	**硬约束。**违反即视为未完成，属验收红线
📌	**已确认决策。**源自上游文档的确认结论，不再讨论
⚠	**建议值 / 待确认。**已给出默认值，但需评审拍板
⏱	**本机实测数据。**非估算，附带复现方式
🚧	**技术债。**本次不修，已知且已披露
【待补充：xxx】	信息缺失占位符。**禁止用猜测值填充**

0.7.1 本文档对上游的三处口径纠正（📌 已核实，须回写）

在撰写本技术文档时对照代码与文档进行了交叉核实，发现三处表述不一致。**本文档以下列取值为准**，并建议在后续修订中同步回写上游文档：

编号	冲突点	上游表述	实际事实（已核实）	本文档取值
JT-01	**CSV 列名语言**	开发方案 §6.2.2 错误示例：「缺少必需字段：**销量**」（中文）	`app/algo/config.py` 定义 `COL_SALES = "sales"`，注释「列名不可改」；`prototype/upload.html` 第 23/131/279 行同步使用英文列名	**英文列名** `sku_id / warehouse_id / date / sales`
JT-02	**模型超参数**	开发方案附录 D 性能基准：`hidden=32 / layers=1 / window=14`	生产训练产物 `outputs/models/lstm_v1.report.json`：`hidden_size=48 / num_layers=2 / window=21`	**分列两套**：基准配置（压测对照）与生产配置（实际上线），见 §1.5
JT-03	**模型参数量**	开发方案 §6.3 R6 写「模型仅 5,454 参数」	该数值对应基准配置；生产配置实测参数量 **31,729**	**31,729**（生产配置）

这三处若不在编码前定死，会在联调期以「上传报错字段名不对」「服务加载的模型与文档描述不一致」的形式暴露，排查成本远高于现在纠正。

0.7.2 两个阻塞级问题的决议（📌 已拍板，TQ-4 → JT-10、TQ-7 → JT-11）

本文档附录 B 曾列出两个会卡住开发的问题。**现已决议，编码时直接按此执行，不再讨论**：

原编号	问题	决议	落地位置
TQ-4 → JT-10	游客态如何落库（FR-USR-08 要求未登录可跑到结果页，但 datasets.user_id 是 NOT NULL）	**user_id = 0 表示游客，并在 users 表落一条 id = 0 的哨兵账号。**哨兵的存在不是为了「让游客能登录」，而是为了在 PRAGMA foreign_keys = ON 下让 user_id = 0 能通过外键校验	schema.sql T01a、§3.3、§4.3、§5.5
TQ-7 → JT-11	错误明细（API-11）的临时文件放哪、TTL 多久	**data/exports/tmp/dataset_{dataset_id}_{upload_id}.csv，TTL 1 小时。**与正式导出产物**分目录**，避免被 7 天清理任务误保留	API-11、§2.8、§3.8、§10.6

⏱ 两项决议均已**实测验证**，不是纸面结论：

```
[ OK ] 游客哨兵存在：id=0, email='__guest__@local.invalid', nickname='游客（未登录）'
[ OK ] 哨兵 password_hash 为 '!'，非 bcrypt 串 → 永远无法通过校验
[ OK ] 游客数据集写入成功：datasets.id=1, user_id=0
[ OK ] 游客任务写入成功：tasks.id=1, user_id=0
[ OK ] 非法 user_id=99999 被外键拒绝：FOREIGN KEY constraint failed
[ OK ] 哨兵不干扰自增：正常注册 id 从 1 开始 [1, 2, 3]
[ OK ] 无孤儿数据集（user_id 均有对应用户或哨兵）
```

倒数第二条同样关键：id = 0 的哨兵不会让真实用户的 id 分配错位（AUTOINCREMENT 仍从 1 开始）。最后一条证明 user_id = 0 是**精准开的一个口子**，而不是把外键这道墙拆了。复现命令 python scripts/verify_schema.py。

0.7.3 落地 TQ-4 时新发现的两处缺陷（📌 已修复，JT-12 / BE-40）

在落实游客态时做了一轮依赖行为核实，发现两处**会导致 500 而非 4xx** 的真实缺陷。它们不属于 TQ-4 本身，但同属「让文档与真实行为对上」的范畴，故一并修掉并在此披露：

编号	缺陷	证据	修复
JT-12	**密码上限必须按字节校验，不能按字符**	bcrypt 5.x 对 >72 字节的密码**直接抛 ValueError**（4.x 是静默截断，行为已变）。旧限制只有「64 字符」，而 64 个中文字符 = 192 字节，会在 hashpw 处 500	schemas.py 增加 _enforce_bcrypt_bytes()，RegisterRequest / UpdateUserRequest 均接入；verify_schemas.py 加 6 条回归用例（含精确 72 / 73 字节边界）
BE-40	**hash_password 的注释与实现不一致，埋了 500 隐患**	§4.4.1 原示例的注释写「超出部分会被静默截断，**故先做长度校验**」，但代码里**根本没有长度校验**——注释承诺了没做的事。而「静默截断」这一前提在 bcrypt 5.x 下已不成立（改为抛错），照抄该示例即 500	§4.4.1 重写：补齐前置校验代码，并把「为什么 checkpw 必须捕获 ValueError」写入决策表

📌 关于 JT-12 的易错细节：**「64 字符」与「72 字节」两道限制缺一不可**。ASCII 场景下 72 字节 = 72 字符 > 64，先被字符上限拦下；中文场景下 24 个字符就已达 72 字节，必须靠字节校验拦下。只留一道必漏。

📌 哨兵邮箱用 RFC 2606 保留域 .invalid，实测 EmailStr **会主动拒绝该域**（The part after the @-sign is a special-use or reserved name that cannot be used with email）。即注册接口天然无法占用 __guest__@local.invalid，**无需额外代码防护**。同时实测 EmailStr 不做 DNS 可达性校验（a@b.com 被接受），故离线 / 内网环境可用。

0.8 配套代码资产

本文档不是纯文字说明，配套 5 份可直接落盘的文件，全部经过实际执行验证：

文件	内容	验证方式	验证结果
`code/db/schema.sql`	9 张表 + 14 索引 + 3 触发器 + 1 视图 + **游客哨兵账号**（`JT-10`）	`python scripts/verify_schema.py`	✅ 9 表齐备、幂等、约束生效；游客态 6 项实测通过
`code/db/seed.sql`	模型版本注册数据 + 哨兵幂等补插	随 `schema.sql` 一并执行	✅ 幂等
`app/schemas/`（7 模块，**真源**） `code/backend/schemas.py`（生成件）	**64 个 Pydantic v2 模型** + bcrypt 72 字节校验（`JT-12`）	`verify_schemas.py` + `gen_backend_schemas.py --check`	✅ **61/61 通过**、生成件与真源一致
`code/frontend/types.ts`	30 个接口类型 + 5 个类型别名 + 密码字节校验函数	`node scripts/verify_types.mjs`	✅ 语法通过
`code/frontend/api-client.ts`	Axios 封装 + 全量 API 函数	同上	✅ 语法通过

📌 **配套文件与本文档同步维护**：修改接口必须先改 `schemas.py` 与 `types.ts`，再更新本文档 §5，最后跑三个校验脚本。任一处遗漏视为技术债。

---

1. 技术选型与版本锁定

1.1 选型总表

层	技术	版本	依据	状态
运行环境	Python	**3.12.14**	任务书 3.2 要求 3.12+；本机实测版本	✅ 已就绪

层	技术	版本	依据	状态
后端框架	FastAPI	**0.141.1**	任务书 3.2 明确	✅ 已就绪
ASGI 服务器	Uvicorn	**0.52.4**	FastAPI 官方推荐搭配	✅ 已就绪
数据校验	Pydantic	**2.9.2**	FastAPI 0.141 依赖	✅ 已就绪
ORM	SQLAlchemy	**2.0.x**	参数化查询防注入（NFR-SEC-06）	⚠ 需安装
数据库	SQLite	内置（3.45+）	任务书 3.2 明确；单机小规模场景	✅ 已就绪
密码哈希	bcrypt	**5.0.0**	FR-USR-03 要求 bcrypt 或等价慢哈希。📌 锁 5.x：对 >72 字节的密码**显式报错**而非静默截断，行为更安全（见 JT-12）	✅ 已就绪
Token	PyJWT	**2.9.x**	FR-USR-04 要求 JWT	⚠ 需安装
文件上传	python-multipart	**0.0.12**	FastAPI `UploadFile` 必需	✅ 已就绪
异步文件 IO	aiofiles	**23.2.1**	流式落盘（FR-UPL-04）	✅ 已就绪
算法建模	PyTorch	**2.9.1+cu128**	任务书要求 LSTM；CUDA 12.9 匹配	✅ 已就绪
数据处理	pandas / numpy	**2.3.3 / 2.5.3**	任务书 3.2 明确	✅ 已就绪
Excel 导出	openpyxl	**3.1.x**	FR-VIS-05 要求导出 Excel	⚠ 需安装
前端框架	React	**18.3.x**	任务书 3.2 明确	⚠ 需安装
前端语言	TypeScript	**5.4.5**	接口契约需要类型保障	✅ 已就绪
构建工具	Vite	**5.4.x**	React 18 官方推荐，启动快	⚠ 需安装
UI 组件库	Ant Design	**5.21.x**	任务书明确；含中文语境组件	⚠ 需安装
原子化 CSS	TailwindCSS	**3.4.x**	任务书明确；3.4+ 支持 oklch 任意值	⚠ 需安装
图表	ECharts	**5.5.x**	UI/UX §12.1 已确认（Q16）	⚠ 需安装

层	技术	版本	依据	状态
HTTP 客户端	Axios	**1.7.x**	拦截器与进度回调成熟	⚠ 需安装
路由	React Router	**6.26.x**	主流方案	⚠ 需安装
状态管理	Zustand	**4.5.x**	轻量，无需 Redux 样板代码	⚠ 需安装

📌 **技术栈不变更声明**：以上选型全部来自任务书 3.2 或上游文档的已确认决策。**编码期间不得擅自更换框架或组件库**，如需变更须走 §8.6 变更流程。

1.2 后端依赖清单（requirements.txt）

```
# ============ 运行环境 ============
# Python 3.12.14


# ---------- 核心框架 ----------
fastapi==0.141.1
uvicorn[standard]==0.52.4
pydantic==2.9.2
pydantic[email]          # 邮箱校验，EmailStr 依赖


# ---------- 数据与存储 ----------
SQLAlchemy==2.0.36
python-multipart==0.0.12
aiofiles==23.2.1


# ---------- 认证与安全 ----------
bcrypt==5.0.0
PyJWT==2.9.0


# ---------- 算法层（🔒 版本锁定，见 NFR-COMP-07）----------
torch==2.9.1+cu128
pandas==2.3.3
numpy==2.5.3


# ---------- 导出 ----------
openpyxl==3.1.5


# ---------- 开发期（不进生产镜像）----------
ruff==0.6.9
pytest==8.3.3
httpx==0.27.2
```

依赖分组的理由

分组	说明
核心框架	必须。缺失服务起不来
数据与存储	必须。`python-multipart` 缺失时 FastAPI 上传接口直接报错，是高频踩坑点
认证与安全	必须。**不选 passlib** —— passlib 已停止维护，与 bcrypt ≥ 4.1 有 `__about__` 属性兼容问题，bcrypt 5.x 已彻底移除该属性，用 passlib 必然报错；直接用 `bcrypt` 库。🔒 **锁 5.0.0**：它把「>72 字节静默截断」改为「显式抛 `ValueError`」，配合 `JT-12` 的字节校验才能保证不 500
算法层	🔒 三者版本必须与离线训练环境**完全一致**。若训练与推理的 pandas / numpy 版本不同，浮点行为差异会导致预测值漂移，且极难定位
导出	Excel 导出需要
开发期	不打进生产依赖

⚠ **PyTorch 安装注意**：`torch==2.9.1+cu128` 需从 PyTorch 官方源安装，直接 `pip install torch` 会装成 CPU 版。安装命令：

pip install torch==2.9.1+cu128 --index-url https://download.pytorch.org/whl/cu128

若目标机器无 NVIDIA GPU，改为安装 CPU 版并对齐 `app/algo/model.py` 的设备自动回退逻辑（`FR-MOD-10`），功能不受影响。

1.3 前端依赖清单（package.json）

```
{
  "name": "sales-forecast-web",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview",
```

```
    "lint": "eslint . --ext .ts,.tsx --max-warnings 0",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2",
    "antd": "^5.21.6",
    "echarts": "^5.5.1",
    "echarts-for-react": "^3.0.2",
    "axios": "^1.7.7",
    "zustand": "^4.5.5",
    "dayjs": "^1.11.13"
  },
  "devDependencies": {
    "@types/react": "^18.3.11",
    "@types/react-dom": "^18.3.1",
    "@vitejs/plugin-react": "^4.3.2",
    "typescript": "^5.4.5",
    "vite": "^5.4.9",
    "tailwindcss": "^3.4.14",
    "postcss": "^8.4.47",
    "autoprefixer": "^10.4.20",
    "eslint": "^9.13.0",
    "@typescript-eslint/eslint-plugin": "^8.10.0",
    "@typescript-eslint/parser": "^8.10.0",
    "@testing-library/react": "^16.0.1",
    "vitest": "^2.1.3",
    "jsdom": "^25.0.1"
  }
}
```

📌 **TailwindCSS 版本必须 ≥ 3.4**：设计令牌使用 `oklch()` 色彩空间（UI/UX §3.2），3.4 以下版本不支持写入 `theme.extend`。降级会直接破坏无障碍修复成果（`UX-GOV-02`）。

📌 **不引入 Redux**：本项目状态复杂度低（用户信息、当前任务、主题），Zustand 足够。引入 Redux 会带来大量与业务无关的样板代码。

1.4 版本锁定的强制理由

`NFR-COMP-07` 要求「后端依赖锁定为 torch 2.9.1+cu128、pandas 2.3.3、numpy 2.5.3、Python 3.12+」。这不是形式要求，有三条实质理由：

理由	后果
离线训练与在线推理必须同环境	架构红线 2 要求在线离线共用 `algo/` 同一份代码。若两侧 pandas / numpy 版本不同，`rolling().mean()` 的浮点

理由	后果
	舍入差异会让同一模型的预测值出现漂移，且**不会报错**，只在指标上体现为「怎么又对不上了」
序列化兼容性	`torch.save()` 的权重包在不同 torch 版本间不保证可加载。跨版本加载失败会表现为 `_pickle.UnpicklingError`，排查方向容易跑偏
复现评审	答辩时需复现「加权 MAPE ≤ 20%」这一硬指标。版本漂移会让复现结果与提交材料不一致，直接失去可信度

执行方式：`requirements.txt` 全部使用 `==` 精确锁定，禁止使用 `>=` 或 `~=`。CI（或提交前自查）执行 `pip install -r requirements.txt` 应零冲突。

1.5 已实测的环境事实 ⏱

以下为本机 `deploy` conda 环境的实测结果，**不是估算值**：

项	实测值
Python	3.12.14
torch	2.9.1+cu128（CUDA 12.9 可用）
pandas / numpy	2.3.3 / 2.5.3
fastapi / uvicorn / pydantic	0.141.1 / 0.52.4 / 2.9.2
显卡	NVIDIA MX450，2048 MiB 显存，Compute Capability 7.5
内存 / CPU	23.8 GB / 8 逻辑核

1.5.1 模型生产配置 vs 性能基准配置（📌 必须区分，对应 JT-02）

项	生产配置（实际上线）	基准测试配置（仅用于压测对照）
来源	`app/algo/config.py` +	开发方案附录 D-3

项	生产配置（实际上线）	基准测试配置（仅用于压测对照）
	outputs/models/lstm_v1.report.json	
window_size	**21**	14
hidden_size	**48**	32
num_layers	**2**	1
特征数	11	——
参数量	**31,729**	5,454
训练耗时	45.26 s（cuda）	3.67 s（CPU）/ 0.84 s（GPU）
训练显存峰值	111 MB	111 MB
验证集加权 MAPE	**14.07%**	——

⚠ **为什么必须分列**：两套配置的数字混用会直接导致「文档说 hidden=32，代码里是 48」这类评审质疑。基准配置的作用是**证明性能预算充裕**（10 万样本训练仅秒级，故 60 秒 SLA 有余量），生产配置的作用是**实际交付的模型**。二者目的不同，不可互换。

1.5.2 端到端耗时实测 ⏱（succeeded 任务，horizon=14，113 序列）

来源：outputs/eval_report.json，复现命令 python scripts/eval_report.py --horizon 14

阶段	实测耗时	NFR-PERF-02 预算	余量
预处理	675 ms	≤ 10 s	14.8×
特征工程	763 ms	≤ 15 s	19.7×
模型推理	**21 ms**	≤ 15 s	**714×**
指标计算	473 ms	≤ 5 s	10.6×
端到端	**7,873 ms**	**≤ 60 s**	**7.6×**

📌 **推理仅 21 ms** 这一数字是「不做在线训练」决策的最有力证据：在线成本几乎全部来自数据处理与指标计算，与建模无关。这也说明**性能瓶颈不在 GPU**，优化方向不应放在模型侧。

1.6 设计系统移植方案（React 落地）

UI/UX §12.1 已披露技术栈落差：PRD 定的是 React + TailwindCSS + Ant Design + ECharts，而高保真原型是纯静态 HTML + 手写 CSS + 自研 SVG 图表。本节给出**可执行的移植方案**。

原型资产	React 落点	具体做法	风险
`app.css` 56 个令牌	`src/styles/tokens.css` + Tailwind `theme.extend`	令牌**原样保留 `oklch()`**，不改写为 hex	改为 hex 会丢失感知均匀性，深色模式微调失准
AntD 主题	`<ConfigProvider theme={{ token, algorithm }}>`	把品牌色 / 语义色 / 圆角 / 字号映射到 AntD token；深色用 `theme.darkAlgorithm`	映射不全会导致 AntD 组件与自定义组件色差
`charts.js` 自研 SVG	**ECharts 自定义主题**	注册为独立 theme，从 CSS 变量读取颜色；监听主题切换事件重绘	`UX-GOV-02a`：不重绘会导致深色模式下图表仍是浅色
组件类（`.btn` / `.card` / `.badge`）	AntD 组件 + Token 覆盖	**优先用 `ConfigProvider` 覆盖令牌**，而非重写组件样式	直接改 AntD 内部类名会随版本升级失效
`demo-data.js` 演示数据	真实 API	全量删除，接口替代	见 §1.7 移除清单
演示状态药丸	——	删除，改由真实状态驱动	见 §1.7 移除清单

1.6.1 令牌桥接实现（FE-01）

```
// src/theme/tokens.ts — 单一真源：所有颜色从这里出发
export const BRAND_HUE = 205; // 青蓝，刻意避开 Bootstrap 蓝


// 从 CSS 变量读取，保证 React 与 Tailwind、ECharts 三处同源
function cssVar(name: string): string {
  return getComputedStyle(document.documentElement).getPropertyValue(name).trim();
}


export function buildAntdToken(isDark: boolean) {
  return {
    colorPrimary: cssVar('--primary'),
    colorSuccess: cssVar('--success'),
    colorWarning: cssVar('--warning'),
    colorError: cssVar('--danger'),
    colorInfo: cssVar('--info'),
    colorBgBase: cssVar('--surface'),
    colorTextBase: cssVar('--text-1'),
```

```
    borderRadius: 8,
    fontFamily: "'Inter', 'PingFang SC', 'Microsoft YaHei', sans-serif",
    // 🚧 不在此硬编码颜色，一律引用令牌（UX-GOV-03 的整改要求）
  };
}
```

```
// src/theme/EChartTheme.ts —— 图表主题随 CSS 变量走
export function buildEChartTheme() {
  return {
    color: [
      cssVar('--chart-1'),
      cssVar('--chart-2'),
      cssVar('--chart-3'),
      cssVar('--chart-4'),
      cssVar('--chart-5'),
    ],
    backgroundColor: 'transparent',
    textStyle: { color: cssVar('--text-2') },
  };
}
```

FE-02：**令牌是唯一真源**。禁止在组件内散落硬编码色值——这是 UX-GOV-02 与 UX-GOV-03 的整改要求。代码评审将此项列为必查项。

FE-03：主题切换必须触发 ECharts 重绘。原型已实现该能力（监听 themechange 事件后调用 renderAllCharts()），React 侧须保留，实现方式为在 Zustand 的 theme 变更时调用 chart.setOption 重设颜色。

1.7 实现时必须删除的原型痕迹（UX-GOV-01）🚧

UI/UX §12.3 已列出移除清单，本节把它转为**工程任务**，并在 §9.6 设为验收检查项：

移除对象	原型位置	处理方式
演示状态药丸（.state-bar / .state-pill）	各页页头下方	删除 DOM 与样式，状态改由真实数据驱动
原型说明块（.spec-note / #card-spec）	各页底部	删除
差异标注块（.diff-note）	index.html / README.html	删除

移除对象	原型位置	处理方式
原型说明页	`README.html`	**整页不迁移**
演示账号信息块（`.demo-accounts`）	`login.html`	删除
`REAL_EVAL` 常量与合成数据生成器	`assets/demo-data.js`	删除，改由接口提供
原型专用样式	`app.css` 第 19 / 20 章	不迁移

必须保留：全局页脚免责声明（`FR-SYS-05` 🔒）、所有真实业务组件、无障碍相关样式（聚焦环、`prefers-reduced-motion`）。

> ⚠ **常见疏漏**：删除 `demo-data.js` 时容易连 `renderHeader()` / `renderSidebar()` 的 DOM 结构一起丢掉。这两个函数定义的是**真实布局骨架**，React 侧需将其转写为 `<AppLayout>` 组件后再删除脚本。建议做法：先实现布局组件，视觉走查通过后再删脚本。

1.8 技术选型风险与应对

编号	风险	影响	应对
`BE-01`	SQLAlchemy 2.0 与 Python 3.12 + SQLite 的 `datetimestamp` 适配	时间字段读写异常	📌 本项目**统一用 TEXT 存时间**（`YYYY-MM-DD HH:MM:SS`），不使用 SQLAlchemy 的 `DateTime` 类型，从根上规避；依据 `NFR-COMP-04`「不引入 UTC 转换」
`FE-04`	TailwindCSS 与 AntD 样式优先级冲突	视觉不一致	Tailwind 只做布局与微调类（`flex` / `gap` / `grid`），**不覆盖 AntD 组件内部样式**；组件外观统一走 `ConfigProvider`
`FE-05`	ECharts 体积大（约 1 MB gzip 前）	首屏变慢（`NFR-PERF-06` 要求 ≤ 3 s）	按需引入：使用 `echarts/core` + 显式注册 `LineChart` / `BarChart` / `GridComponent` / `TooltipComponent` / `LegendComponent`，禁用全量 `import * as echarts`
`BE-02`	2 GB 显存被其他程序占用导致 OOM	推理中断	`FR-MOD-10` 已要求 CUDA→CPU 自动回退。实现要点：捕获 `torch.cuda.OutOfMemoryError` 后**不是简单重试**，而是

编号	风险	影响	应对
			`torch.cuda.empty_cache()` → 把模型与输入 `.to('cpu')` → 重跑；结果须与 GPU 一致
`BE-03`	SQLite 并发写锁	多用户同时提交任务时 500	`NFR-REL-04`：开启 WAL 模式 + 批量插入。**连接级注意**：SQLAlchemy 需设 `connect_args={"check_same_thread": False}`，并在启动时执行 `PRAGMA journal_mode=WAL`

---

2. 工程结构与开发环境

2.1 仓库总体结构

```
销量预测/
├── app/                        # 后端服务（FastAPI），可独立启动
│   ├── api/                    # 路由层：只做路由与参数绑定
│   ├── core/                   # 配置、安全、异常、日志、依赖注入
│   ├── models/                 # SQLAlchemy ORM 实体
│   ├── schemas/                # 🔧 模型真源：common/auth/dataset/task/result/history/admin
│   ├── services/               # 业务编排层
│   ├── dao/                    # 数据访问层
│   ├── algo/                   # 🔧 算法层（9 模块，已实现，无 Web 依赖）
│   └── main.py                 # 应用入口
│
├── web/                        # 前端（React + Vite）
│   ├── src/
│   │   ├── api/                # 接口封装（见 code/frontend/api-client.ts）
│   │   ├── components/         # 通用组件
│   │   ├── pages/              # 14 个页面
│   │   ├── layouts/            # 布局骨架
│   │   ├── store/              # Zustand 状态
│   │   ├── theme/              # 设计令牌桥接
│   │   ├── types/              # 类型定义（见 code/frontend/types.ts）
│   │   └── utils/
│   ├── public/
```

```
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   └── package.json
│
├── data/                        # 📌 运行期数据（必须放工作区内，禁写系统盘）
│   ├── app.db                   # SQLite 数据库文件
│   ├── uploads/                 # 上传的原始 CSV，按 user_id 分目录
│   │   └── u{user_id}/{dataset_id}_{stored_name}.csv     # u0 目录 = 游客（JT-10）
│   └── exports/                 # 导出文件
│       ├── {task_id}_{timestamp}.csv                     # 正式产物，TTL 7 天
│       └── tmp/                                          # 临时错误明细，TTL 1 小时（JT-11）
│
├── outputs/                     # 交付物与算法产物
│   └── models/
│       ├── lstm_v1.pt           # 模型权重 bundle
│       ├── lstm_v1.report.json # 训练报告
│       └── demo_sales.csv       # 内置示例数据集
│
├── code/                        # 📌 本文档配套代码资产（可直接落盘使用）
│   ├── db/schema.sql
│   ├── db/seed.sql
│   ├── backend/schemas.py
│   └── frontend/{types.ts, api-client.ts}
│
├── docs/                        # 文档源文件（.md）
├── prototype/                   # 高保真原型（设计基线，不参与构建）
└── scripts/                     # 离线脚本（训练、评估、校验，不参与服务启动）
```

📌 **`data/` 必须位于工作区内**：本机 C 盘可用空间约 29 GB（实测），中间文件写入系统盘会快速耗尽空间（开发方案 R12）。所有路径在配置中以 `BASE_DIR` 为基准拼接，禁止硬编码绝对路径。

2.2 后端目录结构与模块职责

```
app/
├── main.py                      # FastAPI 实例、中间件注册、路由挂载、启动钩子
├── core/
│   ├── config.py                # Settings（读 .env），路径常量
│   ├── security.py              # bcrypt 哈希、JWT 签发/校验
│   ├── deps.py                  # 依赖注入：get_db / get_current_user
│   ├── errors.py                # 业务异常类 + 统一异常处理器
│   └── logging.py               # 日志配置
├── api/
│   ├── v1/
│   │   ├── auth.py              # API-01~05
│   │   ├── datasets.py          # API-06~11
│   │   ├── predictions.py       # API-12~15
```

```
│   │   ├── history.py          # API-16~19
│   │   └── system.py           # API-20
│   └── router.py               # 汇总挂载到 /api/v1
├── models/                     # ORM 实体，与 code/db/schema.sql 一一对应
│   ├── user.py  dataset.py  task.py
│   ├── prediction.py  metric.py
│   └── registry.py  oplog.py  event.py
├── schemas/                    # 🔧 模型真源（R-08 后不再在运行期读 code/）
│   ├── common.py               # 基类 / 常量 / 统一响应包装 / 健康检查
│   ├── auth.py  dataset.py  task.py
│   └── result.py  history.py  admin.py
├── services/
│   ├── auth_service.py         # 注册/登录/限流/Token
│   ├── dataset_service.py      # 上传落盘、断档处置、详情/预览/错误明细（校验委托 validators）
│   ├── predict_service.py      # 🔧 任务编排：调用 algo 流水线
│   ├── history_service.py      # 历史查询、对比、软删除
│   └── export_service.py       # CSV/Excel 生成与清理
├── validators/                 # 🔧 7 步文件校验（R-06 从 dataset_service 拆出）
│   ├── models.py               # Issue / Inspection 两个数据结构
│   ├── constants.py            # 阈值与提示文案（唯一定义处）
│   ├── csv_io.py               # 编码探测 / 读表 / 读表头 / 日期与数值转换
│   ├── stats.py                # 统计摘要 / 断档检测 / 预览
│   ├── steps.py                # 一步一个函数：
check_extension/size/structure/dates/sales/series_limit
│   └── pipeline.py             # inspect_file()：只做编排与短路返回
├── dao/                        # 纯 CRUD，不含业务判断
└── algo/                       # 🔧 已实现的算法层，9 模块
    ├── config.py  preprocess.py  features.py  classify.py
    ├── model.py   train.py       infer.py     fallback.py  metrics.py
```

BE-04 分层依赖方向（单向，禁止反向）

```
api  →  services  →  dao  →  models
                ↘  algo
```

约束	说明	依据
`algo/` 不得 `import fastapi` / `sqlalchemy`	保证可被离线脚本零 Web 依赖复用	开发方案架构红线 1
`services/` 不得 `import algo.train`	服务进程内存中不应存在训练逻辑	FR-MOD-07
`api/` 不得直接 `import dao`	所有业务必须经 services，避免业务规则散落在路由层	开发方案 §4.2
`dao/` 不得 `import algo`	DAO 只做数据存取	开发方案 §4.2

建议用 `ruff` 的 `flake8-tidy-imports`（`TID` 规则）在提交前自动检查，配置见 §8.3。

2.3 前端目录结构

```
web/src/
├── main.tsx                    # 入口，挂载 Provider
├── App.tsx                     # 路由表
├── api/
│   ├── client.ts               # Axios 实例 + 拦截器（见 code/frontend/api-client.ts）
│   └── index.ts                # 按模块导出 authApi / datasetApi / ...
├── types/
│   └── index.ts                # 见 code/frontend/types.ts
├── theme/
│   ├── tokens.css              # 🔧 56 个设计令牌（从 app.css 原样搬入）
│   ├── antd-token.ts           # AntD ConfigProvider 映射
│   └── echarts-theme.ts        # ECharts 主题
├── layouts/
│   ├── AppLayout.tsx           # 模板 A：页头 + 侧栏 + 主内容 + 页脚
│   ├── AsideLayout.tsx         # 模板 B：A + 右栏（with-aside）
│   ├── AuthLayout.tsx          # 模板 C：认证页左右分栏
│   └── MarketingLayout.tsx     # 模板 D：营销页全宽
├── pages/
│   ├── Index.tsx               # UX-P01
│   ├── Login.tsx               # UX-P02
│   ├── Register.tsx            # UX-P03
│   ├── Dashboard.tsx           # UX-P04
│   ├── Upload.tsx              # UX-P05
│   ├── Params.tsx              # UX-P06
│   ├── Running.tsx             # UX-P07
│   ├── Result.tsx              # UX-P08 ★ 核心
│   ├── History.tsx             # UX-P09
│   ├── HistoryDetail.tsx       # UX-P10
│   └── Compare.tsx             # UX-P11
├── components/
│   ├── common/                 # 免责声明、错误边界、空状态、加载骨架
│   ├── charts/                 # LineCompare / Histogram / Sparkline
│   ├── metric/                 # MetricCard / CoverageBar / VerdictBar
│   └── upload/                 # DropZone / ValidationPanel / PreviewTable
├── store/
│   ├── useAuthStore.ts
│   ├── useTaskStore.ts
│   └── useThemeStore.ts
└── utils/
    ├── format.ts               # 数字、日期、百分比格式化
    └── validate.ts             # 前端预检（与后端同源常量）
```

FE-06 页面路由表（与 UI/UX §6.1 的 11 个产品页一一对应）

路由	页面	模板	登录要求	对应 UX 编号
/	产品首页	D	否	UX-P01
/login	登录	C	否	UX-P02
/register	注册	C	否	UX-P03
/dashboard	工作台	A	是	UX-P04
/upload	上传数据	B	否（游客可体验）	UX-P05
/params/:datasetId	参数配置	B	是	UX-P06
/running/:taskId	执行进度	B	是	UX-P07
/result/:taskId	预测结果 ★	B	是	UX-P08
/history	历史任务	A	是	UX-P09
/history/:taskId	任务详情	A	是	UX-P10
/compare	跨任务对比	A	是	UX-P11

📌 **README.html 不迁移**（UX-GOV-01）。原型的 12 个 HTML 中它属非产品页。

2.4 环境变量清单

后端 .env（提交 .env.example，.env 进 .gitignore）：

变量	类型	默认值	必填	说明
APP_ENV	str	development	否	development / production
APP_PORT	int	8000	否	后端监听端口
BASE_DIR	str	项目根目录	否	所有路径的基准，**禁止硬编码绝对路径**
DB_PATH	str	data/app.db	否	SQLite 文件路径（相对 BASE_DIR）
UPLOAD_DIR	str	data/uploads	否	上传文件目录

变量	类型	默认值	必填	说明
EXPORT_DIR	str	data/exports	否	导出文件目录
MODEL_DIR	str	outputs/models	否	模型权重目录
ACTIVE_MODEL	str	lstm_v1	**是**	启用模型版本，需与 model_registry.version 一致
JWT_SECRET	str	——	**是**	JWT 签名密钥。生产环境必须改为随机长串
JWT_EXPIRE_MINUTES	int	720	否	Token 有效期，默认 12 小时
LOGIN_MAX_FAIL	int	3	否	连续失败上限（FR-USR-06）
LOGIN_LOCK_SECONDS	int	60	否	冷却时长（FR-USR-06）
MAX_FILE_SIZE_MB	int	20	否	NFR-PERF-03
MAX_ROWS	int	500000	否	NFR-PERF-04
MAX_SERIES	int	1000	否	NFR-PERF-05
TASK_BUDGET_MS	int	60000	否	端到端预算（NFR-PERF-01）
EXPORT_TTL_DAYS	int	7	否	导出文件保留天数（NFR-REL-05）
CORS_ORIGINS	str	http://localhost:5173	否	开发期前端地址，逗号分隔
TORCH_DEVICE	str	auto	否	auto / cuda / cpu

前端 web/.env：

变量	默认值	说明
VITE_API_BASE	/api/v1	开发期由 Vite proxy 转发到后端
VITE_APP_TITLE	电商销量预测系统	页面标题

BE-05：JWT_SECRET **缺失时必须启动失败**，不允许用默认值兜底。否则开发期生成的 Token 会在生产环境被接受，构成严重安全漏洞。

BE-06：`ACTIVE_MODEL` 与数据库 `model_registry.is_active = 1` 的记录**必须一致**。启动时校验，不一致直接报错退出，避免「服务正常但模型加载错版本」这种静默故障。

2.5 本地开发环境搭建

2.5.1 前置条件

项	要求	检查命令
Python	3.12+	`python --version`
Node.js	18+（本机 22.22.2）	`node --version`
GPU（可选）	NVIDIA 驱动 + CUDA 12.9	`nvidia-smi`
磁盘	工作区可用 ≥ 5 GB	——

2.5.2 后端启动步骤

```
# 1. 激活环境（本机已有 deploy 环境，含 torch/pandas/fastapi）
conda activate deploy

# 2. 安装增量依赖（deploy 环境已含 torch/fastapi/pydantic/python-multipart/aiofiles）
pip install SQLAlchemy==2.0.36 bcrypt==5.0.0 PyJWT==2.9.0 openpyxl==3.1.5 \
    -i https://mirrors.aliyun.com/pypi/simple/

# 3. 初始化数据库（幂等，可重复执行）
sqlite3 data/app.db < code/db/schema.sql
sqlite3 data/app.db < code/db/seed.sql

# 4. 校验数据库对象
python scripts/verify_schema.py

# 5. 配置环境变量
cp .env.example .env
# 编辑 .env，至少填写 JWT_SECRET

# 6. 启动服务（开发模式，代码热重载）
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

启动成功后：

地址	用途
http://127.0.0.1:8000/health	健康检查（API-20）
http://127.0.0.1:8000/docs	Swagger UI（FastAPI 自动生成，FR-SYS-02 的接口文档交付物）
http://127.0.0.1:8000/redoc	ReDoc 形式

⚠ **国内网络注意**：/docs 默认从 CDN 加载 Swagger UI 资源，受限网络下会白屏。解决方式为离线本地化（随仓库自带 swagger-ui-bundle.js 与 swagger-ui.css，通过 app.mount("/static", ...) 提供），配置见 §10.4。

2.5.3 前端启动步骤

```
cd web
npm install --registry=https://registry.npmmirror.com
npm run dev
```

访问 http://127.0.0.1:5173。Vite 配置中已设置代理，/api 前缀请求转发到 http://127.0.0.1:8000，**开发期无跨域问题**：

```
// web/vite.config.ts
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': { target: 'http://127.0.0.1:8000', changeOrigin: true },
    },
  },
});
```

2.5.4 常见问题

现象	原因	解决
`No module named 'multipart'`	缺 `python-multipart`	`pip install python-multipart==0.0.12`
上传接口 422 且无详细信息	同上	同上
`RuntimeError: no running event loop`	在同步函数里用了 `async` IO	上传落盘用 `async def` + `aiofiles`
CUDA 报 `no kernel image is available`	torch 装成 CPU 版或 CC 不匹配	确认 `torch.version.cuda` 不为 None；MX450 CC 7.5 需 cu118+
前端请求 404	未配 Vite proxy 或后端未启动	检查 `vite.config.ts` 与后端进程
SQLite `database is locked`	未开 WAL，或存在长事务	见 §3.9 与 `BE-12`

⚠ **环境冲突提醒**：本机用户在 PyCharm 中并发执行 `pip` 与后台安装曾多次产生依赖锁冲突。**安装依赖时请确保没有其他 pip / conda 进程在运行。**

2.6 常用命令速查

场景	命令
启动后端（开发）	`uvicorn app.main:app --reload --port 8000`
启动前端（开发）	`cd web && npm run dev`
类型检查	`cd web && npm run typecheck`
前端单测	`cd web && npx vitest run`
后端单测	`pytest tests/ -v`
代码规范检查	`ruff check app/ scripts/` ; `cd web && npm run lint`
重新训练模型（离线）	`python scripts/train_and_register.py`
重跑算法评估	`python scripts/eval_report.py --horizon 14 --export-series`
校验数据库脚本	`python scripts/verify_schema.py`
校验后端模型	`python scripts/verify_schemas.py`
校验前端类型	`node scripts/verify_types.mjs`
导出交付文档	`python scripts/build_md_docx_formal.py`

🔒 **两条命令的顺序约束**：重训模型后**必须重跑** `eval_report.py --export-series`，否则结果页展示的指标会落后于模型（此约束源自项目历史踩坑：曾出现「原型显示一套数字、算法产出另一套数字」的问题）。

2.7 端口与进程约定

服务	端口	说明
后端 FastAPI	**8000**	可经 `APP_PORT` 覆盖
前端 Vite dev	**5173**	开发期
前端预览 / 生产	**80** 或 **8080**	生产由 Nginx 托管静态资源并反代 `/api`

生产部署形态：Uvicorn 单进程 + Nginx 托管前端静态文件并反向代理 `/api` 到 8000。📌 不使用多 worker：SQLite 写锁与模型单例加载都更适合单进程（`NFR-PERF-09` 只要求同时 3 个任务不崩溃，单进程可满足）。

2.8 文件存储与磁盘策略

目录	内容	命名规则	清理策略
`data/uploads/u{user_id}/`	原始 CSV	`{dataset_id}_{stored_name}.csv`，`stored_name` 为 UUID，**不使用用户原始文件名**（`NFR-SEC-07`）	保留（`FR-SYS-08` 要求原始数据保留）。📌 游客数据落在 `u0/`，TTL 24 小时（`JT-10`）
`data/exports/`	正式导出结果	`{task_id}_{timestamp}.csv/xlsx`	TTL 7 天自动清理（`NFR-REL-05`）
`data/exports/tmp/`	**临时错误明细**（🆕 `JT-11`）	`dataset_{dataset_id}_{upload_id}.csv`	🔒 TTL **1 小时**，独立于上层目录清理
`outputs/models/`	模型权重	`lstm_v1.pt`	永久保留
`data/app.db`	数据库	——	永久保留

📌 JT-11：**临时错误明细为何要单独目录。**若不分开，data/exports/ 的 7 天清理逻辑会把 1 小时的临时文件一并当正式产物保留 7 天——用户几次上传失败就会堆出大量无用文件。分目录后两套 TTL 互不干扰，清理实现也只需按前缀扫描。upload_id 取上传请求的 UUID，与 dataset_id 组合可保证同一数据集多次上传的明细不互相覆盖。

🔒 **安全要求（NFR-SEC-07）**：上传目录**不可执行**，且必须按 user_id 分目录隔离。用户 A 的文件路径不得出现在用户 B 的任何响应中。实现时所有文件路径由服务端拼接，**禁止接受前端传入的文件路径参数**（防目录穿越）。

---

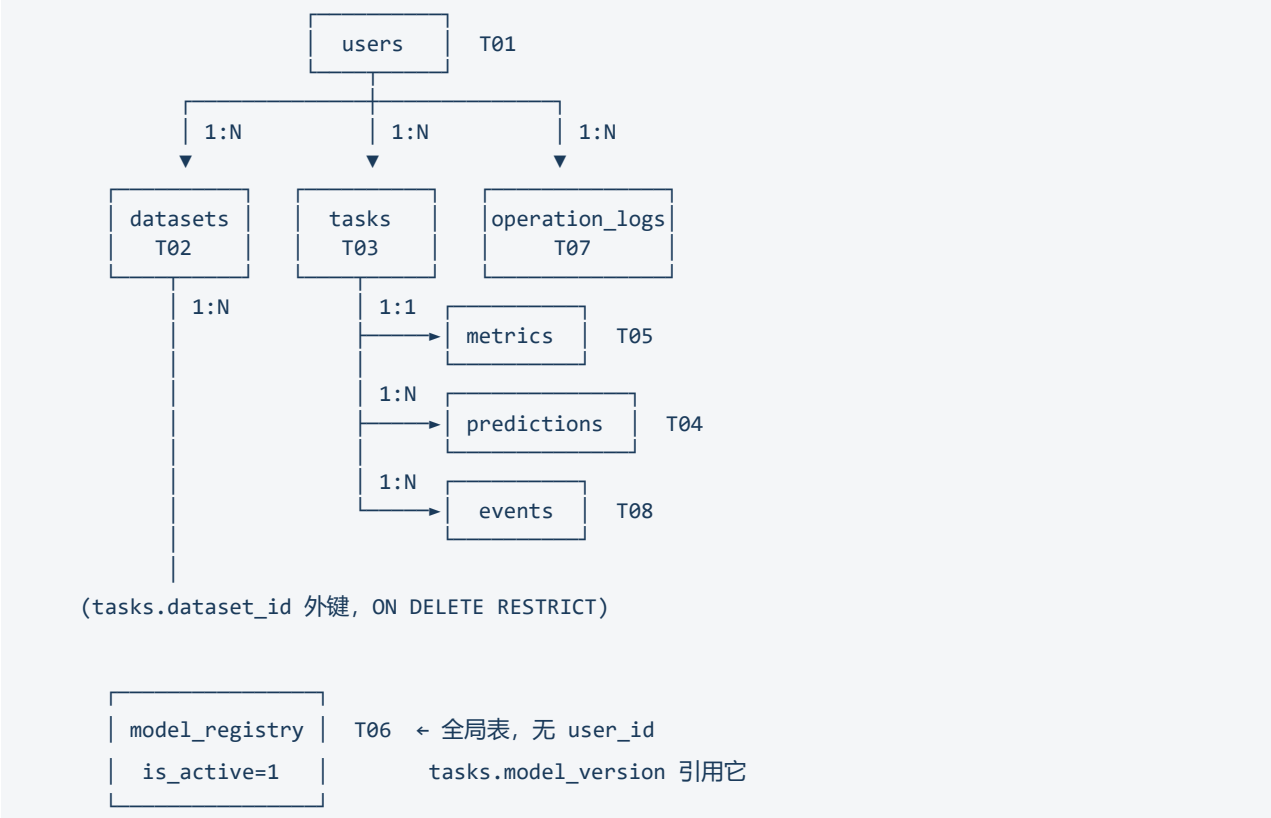
3. 数据库设计

3.1 设计原则

编号	原则	理由
DB-01	**全表带 user_id**（users 除外）	NFR-SEC-02 数据隔离红线。任何查询不带此条件即为缺陷
DB-02	**时间统一 TEXT，格式 YYYY-MM-DD HH:MM:SS**	NFR-COMP-04 要求不引入 UTC 转换。SQLite 无原生时间类型，用 TEXT 可避免驱动层隐式时区处理
DB-03	**命名 snake_case**，表名复数	与 Pydantic / JSON / TS 四处同名，消除转换层
DB-04	**主键统一 INTEGER PRIMARY KEY AUTOINCREMENT**	需严格控制 id 单调递增（供前端展示任务编号）
DB-05	**软删除用 is_deleted 标记**，不物理删除任务	FR-HIS-05 要求「删除后可通过数据库查回，满足可追溯」
DB-06	**枚举值用 TEXT + CHECK 约束**	SQLite 无 ENUM 类型。用 CHECK 保证取值合法，且可读性优于整数编码
DB-07	**JSON 字段用 TEXT 存储 + 应用层序列化**	仅用于 metrics.distribution_json / timing_json / events.properties

编号	原则	理由
		三处，属「写一次读一次」的附属数据，不参与查询
DB-08	**不使用外键级联删除 users 之外的层级**	datasets 被 tasks 引用时用 RESTRICT，避免误删数据集导致任务结果失去溯源

3.2 实体关系



关系说明表

关系	基数	外键	删除行为	说明
users → datasets	1:N	datasets.user_id	CASCADE	删用户即删其数据集
users → tasks	1:N	tasks.user_id	CASCADE	删用户即删其任务

关系	基数	外键	删除行为	说明
datasets → tasks	1:N	tasks.dataset_id	**RESTRICT**	禁止删除仍被任务引用的数据集，保证结果可溯源
tasks → predictions	1:N	predictions.task_id	CASCADE	删任务即删明细（实测规模：113 序列 × 14 天 ≈ 1,582 行）
tasks → metrics	**1:1**	metrics.task_id（UNIQUE）	CASCADE	一任务一指标集
tasks → events	1:N	events.task_id	不设外键	埋点表刻意不设外键：埋点不得阻塞主流程，写失败仅记日志
model_registry → tasks	1:N（逻辑）	tasks.model_version（字符串）	——	用字符串版本号而非 id 关联：模型可被替换而任务需保留「当时用的哪个版本」

📌 **为什么 tasks.model_version 不做外键**：模型版本表中的记录可能因迭代被清理，但历史任务必须永远记住「当时用的是 lstm_v1 还是 lstm_v2」。用字符串快照比外键更符合审计需求。

3.3 表结构详细设计

📌 完整的、可执行的建表语句见 **code/db/schema.sql**（300 行，已验证可执行且幂等）。本节逐表说明**设计意图与关键字段**，不重复粘贴全部 DDL。

DB-T01 users 用户表

字段	类型	约束	说明
id	INTEGER	PK 自增	
email	TEXT	NOT NULL，**NOCASE 唯一索引**	登录名。用 COLLATE NOCASE 使 Alice@x.com 与

字段	类型	约束	说明
			alice@x.com 视为同一账号，避免绕过唯一性校验
password_hash	TEXT	NOT NULL	🔒 **bcrypt 哈希串，禁止明文**（FR-USR-03）。接口响应与日志中一律不得输出（NFR-SEC-09）
nickname	TEXT	可空	展示名
status	INTEGER	NOT NULL DEFAULT 1，CHECK IN (0,1)	1 正常 / 0 停用
fail_count	INTEGER	NOT NULL DEFAULT 0	连续密码错误次数，驱动 FR-USR-06 限流
locked_until	TEXT	可空	冷却截止时间，NULL 表示未锁定
last_login_at	TEXT	可空	
created_at / updated_at	TEXT	NOT NULL	updated_at 由触发器自动维护

DB-T01-01：**识别规格** —— 无 role 字段。PRD §3.1 明确「本期仅一种角色（注册用户）」，不存在管理员视角。若后续引入角色，需同步修改 NFR-SEC-02 的数据隔离实现。

📌 DB-T01-02 **游客哨兵账号（id = 0）** —— 表内固定存在一行哨兵，email = '__guest__@local.invalid'、password_hash = '!'、nickname = '游客（未登录）'。它是 JT-10（游客态）的结构前提：

要素	取值	为什么这么定
id	0	自增从 1 开始，0 永不会被正常注册分配，不存在覆盖风险
email	__guest__@local.invalid	.invalid 是 RFC 2606 保留域，实测 EmailStr **会主动拒绝该域**，故注册接口天然无法占用它——不需要额外校验代码
password_hash	'!'	不是合法 bcrypt 串，checkpw 对它抛 ValueError → **永远无法登录**（Django 的 unusable password 同法）
status	1	保留为正常态；该账号不参与登录流程，status 不承担拦截职责

为何不把 datasets.user_id 直接改成可空：user_id 是数据隔离的第一道闸门（NFR-SEC-02），所有 DAO 查询都把它当整数用。允许 NULL 会让每个查询都要处理 IS NULL 分支，且 WHERE user_id = :uid 在 NULL 语义下行为反直觉。用 0 表达「游客」保持了类型与查询形式的一致性。

DB-T02 datasets 数据集表

字段	类型	说明
id / user_id	INTEGER	归属，user_id 为隔离红线的第一道闸门。📌 **0 表示游客**（JT-10），非 0 即为真实注册用户
file_name	TEXT	**用户原始文件名**，仅用于展示
stored_name	TEXT	**实际落盘文件名**（UUID），NFR-SEC-07 要求重命名存储
file_path	TEXT	相对 BASE_DIR 的路径，拼接时禁止接受前端输入
file_size	INTEGER	字节数
row_count	INTEGER	有效数据行数（示例数据集为 14,305）
sku_count	INTEGER	去重商品数（示例为 40）
warehouse_count	INTEGER	去重仓库数（示例为 5）。📌 仓库列为必需字段，此值不应为 1 或 0
valid_series_count	INTEGER	sku_id × warehouse_id 组合数（示例为 113）
date_start / date_end	TEXT	数据时间范围（示例 2015-01-28 ~ 2015-12-27）
gap_filled	INTEGER	是否执行过插值补齐（FR-UPL-13 用户选择「自动插值」后置 1）
gap_count	INTEGER	补齐的断档天数合计，用于结果页披露
is_sample	INTEGER	1 表示内置示例数据（FR-UPL-02）
status	INTEGER	1 可用 / -1 已失效

DB-T02-01：valid_series_count 与 row_count / 有效天数 的关系是校验的依据之一。若 valid_series_count > MAX_SERIES(1000)，须在写入前拦截（FR-UPL-14）。

DB-T03 tasks 预测任务表

字段	类型	说明
id / user_id / dataset_id	INTEGER	归属与输入。📌 **user_id = 0 表示游客任务**（JT-10）。游客任务天然不出现在任何登录用户的历史列表中（历史查询恒带 user_id = 当前用户），故不构成越权
task_name	TEXT	用户可命名；为空时后端生成，格式建议 {数据文件名截断}_{MMDD HH:mm}
granularity	TEXT	📌 固定 warehouse_sku，CHECK 约束限定取值。商品级视图由前端聚合实现，不落库
horizon	INTEGER	预测步长，CHECK IN (7,14,30)
use_holiday_feature	INTEGER	节假日特征开关
status	TEXT	📌 created / running / succeeded / failed，**无 queued**（取消异步排队后简化）
error_code	TEXT	E_TIMEOUT / E_TOO_MANY_SERIES / E_INFER_FAIL
error_msg	TEXT	失败详情，面向排障
completed_stage	TEXT	🔒 软失败时记录**已完成阶段**（NFR-REL-02 要求返回已完成进度，不得给 502）
model_version	TEXT	**NOT NULL**。因为在线不再有「本次训练的模型」，必须记明用的哪个版本
device	TEXT	实际推理设备（cuda / cpu）
device_fallback	INTEGER	是否发生过 CUDA→CPU 回退（FR-MOD-10），驱动 EV-PRED-05 埋点
series_total / l1_count / l2_count / l3_count	INTEGER	覆盖率披露数据源（FR-VIS-07）。🔒 三项之和必须等于 series_total

字段	类型	说明
duration_ms	INTEGER	🔒 端到端耗时，用于验证 ≤ 60 s 硬指标
started_at / finished_at	TEXT	
is_deleted	INTEGER	软删除标记，所有列表查询须带 is_deleted = 0

DB-T03-01：**状态流转只允许单向**：created → running → succeeded | failed。禁止从 succeeded 回到 running（本项目不做重跑，重跑即新建任务，符合 FR-HIS-01 的列表语义）。

DB-T03-02：**失败任务的重试是「新建任务」而非「复用任务」**（NFR-REL-01 的「原参数一键重试」= 用原参数调 POST /predictions，生成新 task_id）。理由：失败记录需保留以供排障，覆盖会丢失证据。

DB-T04 predictions 预测结果表

字段	类型	说明
id	INTEGER	PK
task_id	INTEGER	所属任务
sku_id / warehouse_id	TEXT	序列标识。📌 warehouse_id **NOT NULL**（仓库列为必需维度）
forecast_date	TEXT	预测目标日期
predicted_value	REAL	预测销量
actual_value	REAL	测试集真值，**仅回测时有值**。生产环境为 NULL
baseline_value	REAL	移动平均基线值，用于对比图与基线口径核算（FR-MOD-06 要求同预测点）
granularity_level	TEXT	🔒 CHECK IN ('L1','L2','L3')，标识建模层级与置信度来源
lower_bound / upper_bound	REAL	置信区间，P2 启用

DB-T04-01：**联合唯一索引**（task_id, warehouse_id, sku_id, forecast_date）。这既是数据完整性保障，也防止批量插入时因重试产生重复行。

DB-T04-02：**写入必须批量**（NFR-REL-04 明确要求）。实测规模：113 序列 × 14 天 = 1,582 行；若扩展至 1,000 序列 × 30 天 = 30,000 行，逐行 commit 会产生 30,000 次磁盘同步，耗时从数百毫秒膨胀到十几秒，直接威胁 60 秒 SLA。

DB-T05 metrics 任务指标表

与 tasks 一对一（task_id UNIQUE）。字段全部按 **「加权口径 + 达标序列占比」双口径**设计：

分组	字段	说明				
主指标	mape / mae / rmse	📌 加权口径。mape 即 WAPE = `Σ\	y-ŷ\	/ Σ\	y\	`，硬指标 ≤ 20
基线	baseline_mape / baseline_mae / baseline_rmse	🔒 **必须与主模型同预测点、同序列集合**（FR-MOD-06）				
提升幅度	mae_improve_pct / rmse_improve_pct	🔒 硬指标 ≥ 15				
达标序列	series_evaluated / series_passed / series_pass_rate	辅助指标 ≥ 60%。注意 series_evaluated 是**分母**，等于 L1 序列数而非总序列数				
分组指标	l1_mape / l2_mape	🔒 **分开统计，不得混算后宣称达标**（PRD §3.4.3）				
判定	is_over_threshold	加权 MAPE 是否超 20%，驱动前端不可折叠警告（FR-VIS-06）				
披露	eval_note	🔒 固定文案：				

分组	字段	说明				
		「本结果使用预训练模型推理，未针对本次输入数据微调」（FR-MOD-13）				
附属	distribution_json / timing_json	直方图数据与分阶段耗时				

DB-T05-01：**实测数据参考值**（outputs/eval_report.json，horizon=14）。编码期可用于验证接口返回是否正确：

{

"metrics": {

"weighted_mape": 14.26, "weighted_mae": 1.32, "weighted_rmse": 3.37,

"baseline_mae": 1.8, "baseline_rmse": 4.65,

"mae_improve_pct": 26.42, "rmse_improve_pct": 27.52,

"series_pass_rate": 83.95, "series_passed": 68, "series_evaluated": 81

}

}

🔒 DB-T05-02：series_evaluated = 81（L1 序列数），而总序列数为 113。**这是刻意的**：L2 序列在测试段缺乏可靠真值（实测 l2_mape 为 nan），纳入分母会使指标失去意义。前端展示「达标序列占比」时必须同时给出分母，避免误导（FR-VIS-07 的披露精神）。

DB-T06 model_registry 模型版本表

字段	说明
version	版本号，如 lstm_v1，UNIQUE
file_path	权重 bundle 路径
超参	window_size / hidden_size / num_layers / dropout / learning_rate / batch_size

字段	说明
训练上下文	train_data_range / train_row_count / series_count
验证指标	val_mae / val_rmse / val_mape（加权口径）
环境	device（cuda/cpu） / peak_memory_mb / param_count
状态	is_active，**部分唯一索引保证全局仅一条为 1**

DB-T06-01：**部分唯一索引**是这张表的关键设计。

CREATE UNIQUE INDEX ux_model_active ON model_registry (is_active) WHERE is_active = 1;

普通唯一索引做不到「只允许一条 is_active=1，但允许多条 is_active=0」。这个索引让「同时存在两个启用模型」在**数据库层**不可能发生，而不是靠应用层自觉。

DB-T06-02：实测注册数据（来自 outputs/models/lstm_v1.report.json）：window_size=21、hidden_size=48、num_layers=2、param_count=31729、val_mape=14.07、device=cuda、peak_memory_mb=111。

DB-T07 operation_logs 操作日志表

面向**排障审计**，记录 login / register / logout / upload / predict / export / delete 七类动作（FR-SYS-03 要求四类，本表按实现需要扩充到七类），含 user_id / target_id / result / detail / ip / created_at。

DB-T07-01：user_id 可空 —— 注册失败的请求尚未产生用户，但也需要留痕。

DB-T08 events 埋点表

面向**指标分析**，承载 PRD §5.1 的 18 个 EV-* 事件。字段 event_name / user_id / task_id / properties(JSON) / created_at。

DB-T08-01：**与 operation_logs 分表的理由** —— 二者生命周期不同：操作日志用于审计，需长期保留；埋点数据是分析素材，可批量归档或清理。合表会导致其中一方无法独立清理。

🔒 DB-T08-02：**埋点写入失败不得抛异常**（PRD §5.1）。实现方式：埋点写入包在 try/except 中，失败仅 logger.warning。绝不允许因为埋点表锁导致用户任务失败。

3.4 索引设计

索引	表	字段	支撑场景
ux_users_email	users	email COLLATE NOCASE	登录查询、注册查重
ix_datasets_user	datasets	(user_id, created_at DESC)	我的数据集列表
ix_tasks_user_list	tasks	(user_id, is_deleted, created_at DESC)	🔧 历史列表页的主要查询路径（FR-HIS-01 要求按创建时间倒序）
ix_tasks_status	tasks	(status)	任务状态统计、埋点
ux_predictions_key	predictions	(task_id, warehouse_id, sku_id, forecast_date) UNIQUE	唯一性保障 + 结果查询
ix_predictions_task_sku	predictions	(task_id, sku_id, warehouse_id)	按序列聚合查询、导出排序（FR-VIS-05 列序固定）
ux_model_active	model_registry	(is_active) WHERE is_active = 1	🔧 部分唯一索引，保证唯一启用版本
ix_oplogs_user_time	operation_logs	(user_id, created_at DESC)	操作审计
ix_events_name_time	events	(event_name, created_at DESC)	埋点分析
ix_events_task	events	(task_id)	按任务回溯埋点

复合索引的字段顺序理由（DB-09）：ix_tasks_user_list 把 user_id 放首位，是因为**所有查询都必须带 user_id**（NFR-SEC-02），它天然是最高选择性的前缀。is_deleted 放第二位，因其区分度低但过滤掉绝大多数行。

⚠ **不要给 predictions.sku_id 单独建索引**：该表查询永远带 task_id 前缀，单独索引不会被命中，纯属浪费写入性能。

3.5 约束与触发器

CHECK 约束汇总（共 13 处）——把「程序里约定」变成「数据库层强制」：

表	约束字段	取值
users	status	0 / 1
datasets	is_sample	0 / 1
datasets	status	1 / -1
tasks	granularity	`warehouse_sku`
tasks	horizon	7 / 14 / 30
tasks	use_holiday_feature	0 / 1
tasks	status	created / running / succeeded / failed
tasks	device_fallback	0 / 1
tasks	is_deleted	0 / 1
predictions	granularity_level	L1 / L2 / L3
metrics	is_over_threshold	0 / 1
model_registry	is_active	0 / 1
operation_logs	result	success / fail

触发器（3 个）：`trg_users_updated` / `trg_datasets_updated` / `trg_tasks_updated`，在 `AFTER UPDATE` 时自动刷新 `updated_at`。

> `DB-10`：`predictions` / `metrics` / `events` / `operation_logs` **不设 `updated_at` 与触发器** —— 这四张表只增不改。为只读表维护更新时间戳是无意义的开销。

3.6 初始化数据

见 **`code/db/seed.sql`**，包含：

内容	说明
🔒 **游客哨兵账号**（`id = 0`）	**随建表脚本一同写入**，不需人工干预。它是外键完整性的结构前提，不属于「业务种子数据」（详见 §3.3 `DB-T01-02` 与 `JT-10`）
`model_registry` 启用记录	真实训练数据（`lstm_v1`，参数量 31,729，`val_mape=14.07`）
演示账号	⚠ 默认**注释掉**，需填入真实 bcrypt 哈希后启用；生产环境必须删除
内置示例数据集	**不预置**。示例数据按**当前调用者**惰性创建：登录用户 → `user_id = 该用户`；游客 → `user_id = 0`。重复调用返回同一条（幂等）

📌 **示例数据集的处理方式**：共享文件放在 `outputs/models/demo_sales.csv`（14,305 行 / 40 商品 / 5 仓 / 113 序列 / 约 479 KB），每个调用者首次加载时创建一条**指向该共享文件**的 `datasets` 记录。这样既满足「数据按用户隔离」的语义，又不为每个用户复制一份 479 KB 文件。游客同理——只是归属改为哨兵账号，文件仍共享同一份。

3.7 典型查询与性能考量

场景	查询要点	预期规模
历史列表分页	`v_task_summary` 视图 + `is_deleted=0` + `ORDER BY created_at DESC LIMIT/OFFSET`	单用户任务数量级：数十至数百条，无需优化
结果页曲线	`predictions` 按 `task_id` 取全部行，应用层按 `(sku_id, warehouse_id)` 分组	最大 30,000 行（1,000 序列 × 30 天）
明细表分页	`predictions` + 可选 `granularity_level` 过滤 + 排序	每页 20 行
覆盖率统计	`SELECT granularity_level, COUNT(*) FROM predictions WHERE task_id=? GROUP BY 1`	3 组
历史对比	对 2~4 个 `task_id` 分别取指标与曲线，应用层合并	最多 4 × 30,000 行，⚠ 需注意响应体大小

DB-11：**对比接口的响应控制**。4 个任务 × 1,000 序列的曲线数据可达数十 MB，直接内联会拖垮前端。实现要求：对比接口默认只返回**指标对比**，曲线数据按 `sku_ids` 筛选后返回（前端先让用户选商品）；未筛选时最多返回前 20 条序列并在响应中标注 `truncated: true`。

3.8 数据清理策略

对象	策略	依据
导出文件	TTL 7 天，定时任务清理	NFR-REL-05、FR-SYS-08
🔒 **临时错误明细**	TTL **1 小时**，独立目录 `data/exports/tmp/`，与正式导出**分开清理**	JT-11。用户修正数据的动作通常发生在几分钟内，1 小时足够；与 7 天产物混放会导致磁盘被临时文件挤占
上传原始 CSV	保留	FR-SYS-08 明确「原始 CSV 保留」
🔒 **游客数据**（`user_id = 0`）	**优先清理**：游客数据集与任务的 TTL 取 24 小时，短于正式数据	JT-10。游客身份无主，数据不可能被认领；若不清理，任何人都能通过反复上传撑爆磁盘（NFR-SEC-04）
软删除的任务	**保留不清理**	FR-HIS-05 要求可查回
埋点数据	⚠ 建议 90 天后归档至独立文件	无明确需求，防止表无限增长
模型历史版本	保留最近 3 个	⚠ 建议值，需评审确认

🔒 DB-12：**清理任务的实现约束**。清理是**唯一的全局跨用户操作**，容易成为隔离漏洞。要求：① 只按时间与路径前缀判断，不按业务字段；② 删除游客数据时必须同时清 `data/uploads/u0/`，否则会留下无主 CSV；③ 清理日志不得输出其他用户的数据内容。

3.9 SQLite 配置与并发处理

BE-12 启动时必须执行：

```
# app/core/db.py
from sqlalchemy import create_engine, event
```

```
engine = create_engine(
    f"sqlite:///{DB_PATH}",
    connect_args={"check_same_thread": False},   # FastAPI 多线程访问必需
    echo=False,
)


@event.listens_for(engine, "connect")
def _set_sqlite_pragma(dbapi_conn, _):
    cur = dbapi_conn.cursor()
    cur.execute("PRAGMA journal_mode=WAL")        # NFR-REL-04：并发读写
    cur.execute("PRAGMA foreign_keys=ON")         # SQLite 默认关闭外键！
    cur.execute("PRAGMA busy_timeout=5000")       # 写锁等待 5 秒而非立即报错
    cur.execute("PRAGMA synchronous=NORMAL")      # WAL 下兼顾安全与性能
    cur.close()
```

参数	作用	不设置的后果
`journal_mode=WAL`	读写并发（读不阻塞写）	多用户提交任务时 `database is locked`
`foreign_keys=ON`	🔒 **启用外键约束**	SQLite 默认**不执行**外键约束，schema.sql 里的外键形同虚设，会静默产生孤儿数据
`busy_timeout=5000`	写锁等待	瞬时并发立即抛异常
`synchronous=NORMAL`	减少 fsync 次数	批量插入变慢（FULL 模式下每次提交都 fsync）

🔒 BE-13：**`PRAGMA foreign_keys=ON` 是最容易被忽略的一行。**SQLite 出于历史兼容默认关闭外键检查，若不显式开启，`schema.sql` 中精心设计的外键与 `RESTRICT` 行为**完全不会生效**——删数据集时不会报错，任务却指向了不存在的记录。

---

4. 后端设计与实现

4.1 应用启动与生命周期

`app/main.py` 承担四件事：注册中间件、挂载路由、启动自检、管理模型单例。

```
# app/main.py (骨架)
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware


from app.core import config
from app.core.errors import register_exception_handlers
from app.core.db import init_db
from app.core.model_holder import load_active_model, holder
from app.api.router import api_router



@asynccontextmanager
async def lifespan(app: FastAPI):
    # ---------- 启动自检 (BE-06 / BE-14) ----------
    init_db()                       # 建库建表 (幂等) + PRAGMA 设置
    load_active_model()             # 预加载模型，避免首个请求承担加载开销
    yield
    # ---------- 关闭 ----------
    holder.clear()



app = FastAPI(
    title="电商销量预测系统 API",
    version="1.0.0",
    lifespan=lifespan,
    docs_url="/docs",               # FR-SYS-02 的接口文档交付物
)


app.add_middleware(
    CORSMiddleware,
    allow_origins=config.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


register_exception_handlers(app)
app.include_router(api_router, prefix="/api/v1")
```

4.1.1 启动自检清单（BE-14）

启动时逐项校验，任一失败**直接退出**（不允许「服务起来了但功能是坏的」）：

检查项	失败原因	处置
`JWT_SECRET` 已配置且长度 ≥ 32	环境变量缺失	🔒 退出。禁止用默认值兜底（`BE-05`）
`data/` 及子目录存在且可写	权限 / 路径错误	自动创建；失败则退出
`PRAGMA foreign_keys` 返回 1	连接配置错误	退出（`BE-13`）
`model_registry` 有且仅有一条 `is_active=1`	未执行 `seed.sql`，或存在多个启用版本	退出并提示执行 `seed.sql`
`ACTIVE_MODEL` 与库中启用版本号一致	配置漂移	退出（`BE-06`）
模型权重文件可加载	文件缺失 / 版本不兼容	退出并提示重跑 `train_and_register.py`
模型参数量与 `model_registry.param_count` 一致	权重与注册记录不匹配	⚠ 打印警告继续（可能是有意换权重）

`BE-15`：**模型预加载的意义**。实测单次推理仅 21 ms，但首次加载模型需读取 128 KB 权重并构建计算图，耗时数秒。若不预加载，第一个用户会遭遇明显卡顿，且该延迟会被误计入 `duration_ms`，污染性能指标。实现上用模块级单例持有模型（见 §4.7.3），并注意 `torch.no_grad()` 包装推理。

4.2 中间件链

执行顺序（自上而下）：

序	中间件	职责	备注
1	`CORSMiddleware`	跨域	生产环境改为 Nginx 同源，可移除此中间件
2	`RequestIDMiddleware`	为每个请求生成 `request_id`，注入日志上下文	排障时用它串联一次请求的全部日志
3	`AccessLogMiddleware`	记录方法、路径、耗时、状态码（**单行 JSON**），并带可选的 `task_id` /	不记录请求体（防密码泄露，`NFR-SEC-09`）；`task_id` / `user_id` 由鉴权

序	中间件	职责	备注
		user_id	后的路由层写入 request.state
—	~~TimeoutGuardMiddleware~~	**不存在**（2026-10-26 复评更正）	端点是同步阻塞执行的，ASGI 中间件拿到控制权时请求已结束，**没有能力**中断正在跑的同步任务；强杀只会留下 status=running 的僵尸任务。预算控制见下方 BE-16

BE-16：**超时不能靠中间件强杀。**预测任务在同步执行中，硬中断会留下 status=running 的僵尸任务。正确做法是**在编排层做软失败**：predict_service 内部维护 deadline = start + TASK_BUDGET_MS，在每个阶段结束时检查；超预算则写入 status=failed、error_code=E_TIMEOUT、completed_stage=<已完成阶段>，正常返回响应（NFR-REL-02 明确要求「不得返回 502/504 原始错误」）。

🟤 **落地位置（2026-10-26 复评补记）**：predict_service._check_deadline() 在**三处**检查预算 —— ① 回测遍与预测遍之间；② CPU 回退重跑之前（回退后耗时约 ×4.3）；③ 构建预测行之后（这一步是逐序列取历史算基线，旧实现每条序列扫一遍全表，1000 序列 × 50 万行实测约 74 秒）。此前唯一的检查点在**两遍流水线跑完之后**（事后才发现），等于 60 秒承诺没有任何约束力。

4.3 依赖注入

```
# app/core/deps.py
from typing import Annotated, Generator
from fastapi import Depends, Header
from sqlalchemy.orm import Session


from app.core.db import SessionLocal
from app.core.errors import BizError
from app.core.security import decode_token
from app.dao import user_dao



def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
```

```
        yield db
        db.commit()            # 成功路径统一提交
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()



def get_current_user(
    authorization: Annotated[str | None, Header()] = None,
    db: Session = Depends(get_db),
):
    """解析 Bearer Token 并返回用户。所有受保护接口都依赖它。"""
    if not authorization or not authorization.startswith("Bearer "):
        raise BizError(40101, "请先登录后再执行此操作")

    payload = decode_token(authorization.removeprefix("Bearer ").strip())
    user = user_dao.get_by_id(db, payload["sub"])
    if user is None or user.status != 1:
        raise BizError(40102, "登录状态已失效，请重新登录")
    return user



CurrentUser = Annotated[object, Depends(get_current_user)]
CurrentAdmin = Annotated[object, Depends(get_current_admin)]   # CH-01，见下
DbSession = Annotated[Session, Depends(get_db)]



# ---------------------------------------------------------- 管理员鉴权（CH-01 / BE-ADM-01）
def get_current_admin(ctx=Depends(get_auth_context)):
    """必须已登录**且** ``role == 'admin'``（`CH-01` 管理端专用）。

    🟤 **依赖注入只能写成注解形式 ``admin: CurrentAdmin``**，
       写成 ``admin=CurrentAdmin``（默认值形式）传进去的是**类本身**而非实例，
       后续 ``admin.id`` 拿到的是 SQLAlchemy 的 ``InstrumentedAttribute`` ——
       语法合法、类型检查也过，一路带到 DAO 层才炸，报错信息完全指不到真正的原因。
       （`EP-7` 已踩过 4 处，`EP-8` 起写入静态守卫 ``scripts/verify_ep8.py``。）

    🔒 **越权判定顺序：先认证后授权，不得调换**
       未登录 → ``40101``/``40102``（由 ``get_current_user`` 先跑）
       已登录但非管理员 → ``40301``
       顺序颠倒会让未登录请求拿到 403，前端就分不清「该去登录」与「登录了但没权限」。
    """
    if ctx.user.role != "admin":
        raise BizError(40301, "当前账号无管理端访问权限，请联系系统管理员")
    return ctx.user



> 🔒 **`BE-ADM-01`：本依赖是数据隔离红线（``NFR-SEC-02``）的唯一例外之门。**
```

```
> 管理端的 9 个接口（``API-22``~``API-30``）需要跨用户聚合，是全项目**唯一**
> 不带 ``user_id`` 过滤的接口组。三条硬约束：
> ① 例外**只允许**存在于 ``app/api/v1/admin.py``；其他 5 个 ``api/v1/*.py``
>    出现不带 ``user_id`` 的查询即为红线破坏；
> ② 每个路由必须声明 ``CurrentAdmin``，**禁止**「先查后判」（漏判即越权）；
> ③ 4 类写操作全部写 ``operation_logs``（``action='admin'``）。



# ---------------------------------------------------------- 可选鉴权（JT-10 游客态）
GUEST_USER_ID = 0   # 游客哨兵账号，见 schema.sql T01a、§3.3 DB-T01-02



def get_optional_user(
    authorization: Annotated[str | None, Header()] = None,
    db: Session = Depends(get_db),
) -> int:
    """返回 user_id；未登录或 Token 无效时返回 GUEST_USER_ID，**不抛错**。

    🔒 使用范围严格受限：仅 API-06（上传）、API-07（示例数据）这两个
    FR-USR-08 明确允许游客体验的接口可以使用。其余接口一律用 get_current_user。
    误用的后果是把越权请求静默降级成游客请求，从而掩盖鉴权缺陷。

    设计取舍：Token 无效时不抛 401 而是降级为游客，符合用户直觉（Token 过期后
    点「上传」应能继续，而不是被拦下）；但涉及数据归属的接口必须严格鉴权，
    故本依赖不提供默认值给其他接口。
    """
    if not authorization or not authorization.startswith("Bearer "):
        return GUEST_USER_ID
    try:
        payload = decode_token(authorization.removeprefix("Bearer ").strip())
        user = user_dao.get_by_id(db, payload["sub"])
    except BizError:
        return GUEST_USER_ID
    if user is None or user.status != 1:
        return GUEST_USER_ID
    return user.id



OptionalUserId = Annotated[int, Depends(get_optional_user)]
```

BE-17：**get_db 里不在 finally 中提交**。把 commit() 放在 yield 之后的正常路径上，异常时走 rollback，避免「业务失败但数据已落库」的不一致。需要精细控制事务边界的场景（如批量插入 predictions），在 service 层显式调用 db.commit()。

🔒 BE-18：**`get_current_user` 是数据隔离的执行点。**它返回的用户对象必须被后续所有 DAO 查询用作过滤条件。任何 `dao.xxx(db, task_id=...)` 而不带 `user_id` 的调用都是越权风险（NFR-SEC-02）。

🔒 BE-41：**数据集归属判定顺序（游客数据的特殊性，JT-10）。**引入 `user_id = 0` 后，「这个数据集是不是我的」不再等价于 `user_id == current_user.id`。正确实现：

```
def assert_dataset_readable(dataset, current_user_id: int) -> None:
if dataset.user_id == GUEST_USER_ID:
return                    # 游客数据匿名共享，任何调用者均可用
if dataset.user_id != current_user_id:
raise BizError(40301, "无权访问该数据集")
```

漏掉第一个分支的后果：游客上传的数据集，用户登录后就无法用来创建任务——FR-USR-08 的体验链路在登录态断裂。
写反顺序或写成「两者任一为 0 即放行」的后果：真实用户之间的隔离被击穿（NFR-SEC-02 红线）。**两个分支的顺序不可交换。**

BE-42：**新数据的归属写入规则**（一句话记牢）：`datasets.user_id` 与 `tasks.user_id` 一律写**当前调用者**的身份——登录用户写其真实 `id`，游客写 `0`。不存在「游客上传的数据集，登录后归到该用户名下」的迁移逻辑；数据归属一经写入不再变更。

4.4 认证与授权实现

4.4.1 密码哈希（FR-USR-03）

```python
# app/core/security.py
import bcrypt

from app.schemas.constants import BCRYPT_MAX_BYTES   # = 72, 与 schemas.py 同源 (JT-12)


def hash_password(raw: str) -> str:
    # 🔒 BE-40 / JT-12：必须做字节校验。
```

```
    # bcrypt 5.x 对 >72 字节的密码**抛 ValueError** (4.x 是静默截断，行为已变)。
    # 正常路径由 schemas.py::_enforce_bcrypt_bytes 拦截并返回 422;
    # 此处再断言一次，是为防止绕过接口层直接调用（脚本批量建号、单元测试）时 500。
    if len(raw.encode("utf-8")) > BCRYPT_MAX_BYTES:
        raise ValueError(
            f"密码编码后不得超过 {BCRYPT_MAX_BYTES} 字节，"
            "应在接口层拦截并返回 4xx（不应到达此处）"
        )
    return bcrypt.hashpw(raw.encode("utf-8"), bcrypt.gensalt(rounds=12)).decode()



def verify_password(raw: str, hashed: str) -> bool:
    # 🔒 BE-40: checkpw 在哈希串非法时抛 ValueError，而**不是**返回 False。
    # 这个分支真实存在——游客哨兵的 password_hash = '!'。
    # 漏掉 try/except 的后果：用哨兵邮箱尝试登录会得到 500，
    # 而不是「邮箱或密码错误」（违反 NFR-USE-03 的可读提示要求）。
    try:
        return bcrypt.checkpw(raw.encode("utf-8"), hashed.encode("utf-8"))
    except ValueError:
        return False
```

决策	说明
用 `bcrypt` 而非 `passlib`	passlib 已停止维护；且 **bcrypt 5.x 已移除 `__about__` 属性**，passlib 引用它必然报错，属于硬性不兼容
🔒 锁定 bcrypt **5.0.0**	5.x 把「>72 字节静默截断」改为「显式抛 `ValueError`」。这一变化本身更安全（不会把不同长密码当作同一个），但要求调用方**必须**前置校验，见下行（`JT-12`）
`rounds=12`	单次哈希约 250 ms。既保证暴力破解成本，又不让登录接口超出 `NFR-PERF-06` 的 3 s 预算
🔒 **72 是「字节」不是「字符」**	用 `len(raw.encode("utf-8"))` 而非 `len(raw)`。中文每字占 3 字节，**24 个中文字符即达 72 字节上限**。故「64 字符」的限制单独存在是不够的
🔒 **`checkpw` 必须捕获 `ValueError`**	见代码注释。哨兵账号与任何被手工改坏的哈希串都会走这个分支

⏱ 上述两条约束均经实测确认（bcrypt **5.0.0**）：

checkpw(b'', h='!') -> ValueError: Invalid salt ← 必须捕获

checkpw(b'Demo@12345',h='!') -> ValueError: Invalid salt

```
hashpw('a'*100)              -> ValueError: password cannot be longer than 72 bytes
hashpw('中文'*32)             -> ValueError: password cannot be longer than 72 bytes
正常 bcrypt 串 checkpw(正确密码) -> True       (对照组，行为正常)
```

4.4.2 Token 签发与校验（FR-USR-04 / FR-USR-05）

```python
import jwt
from datetime import datetime, timedelta, timezone
from app.core import config
from app.core.errors import BizError


ALGO = "HS256"


def create_token(user_id: int) -> tuple[str, int]:
    expire_minutes = config.JWT_EXPIRE_MINUTES
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id),
        "iat": int(now.timestamp()),
        "exp": int((now + timedelta(minutes=expire_minutes)).timestamp()),
    }
    token = jwt.encode(payload, config.JWT_SECRET, algorithm=ALGO)
    return token, expire_minutes * 60


def decode_token(token: str) -> dict:
    try:
        return jwt.decode(token, config.JWT_SECRET, algorithms=[ALGO])
    except jwt.ExpiredSignatureError:
        raise BizError(40102, "登录已过期，请重新登录")
    except jwt.InvalidTokenError:
        raise BizError(40101, "登录凭证无效，请重新登录")
```

`FR-USR-05` 的登出实现（Token 立即失效）——JWT 无状态，登出需额外机制：

方案	机制	评价
A. 短 TTL + 客户端清除	Token 有效期 12 小时，登出仅前端删除	✗ 不满足 `FR-USR-05`「登出后旧 Token 立即失效」
B. Token 黑名单	登出时把 `jti` 写入内存集合或表，校验时查黑名单	✓ **推荐。**单机部署用内存 `set` 即可，进程重启后黑名单清空（可接受：重启后旧 Token 本就应重新登录）
C. Token 版本号	`users` 表加 `token_version`，改密码/	✓ 重量级方案，改密码后强制全端

方案	机制	评价
	登出时自增，Token 内携带版本	下线（FR-USR-07 要求改密码后原 Token 失效）

📌 **建议组合 B + C**：登出用黑名单（轻量、即时），改密码用版本号（FR-USR-07 要求「修改后原 Token 失效并强制重新登录」）。由于 PRD 未要求 `token_version` 字段，若采用方案 C 需在 `users` 表增加该字段——**此为 TQ-1，见附录 D。**

4.4.3 登录限流（FR-USR-06）

```
def check_login_lock(db: Session, user) -> None:
    if user and user.locked_until:
        remain = (parse_dt(user.locked_until) - datetime.now()).total_seconds()
        if remain > 0:
            raise BizError(40104, f"密码连续错误次数过多，请 {int(remain) + 1} 秒后重试")


def on_login_fail(db: Session, user) -> None:
    if user is None:
        return
    user.fail_count += 1
    if user.fail_count >= config.LOGIN_MAX_FAIL:      # 3
        user.locked_until = (datetime.now()
                             + timedelta(seconds=config.LOGIN_LOCK_SECONDS)).strftime(FMT)
        user.fail_count = 0                            # 计数归零，冷却结束后重新计


def on_login_success(db: Session, user) -> None:
    user.fail_count = 0
    user.locked_until = None
    user.last_login_at = datetime.now().strftime(FMT)
```

BE-19：**限流提示必须给出剩余秒数。**FR-USR-06 的验收要点写的是「第 4 次立即尝试被拒绝，提示剩余冷却时间」，只回「请稍后再试」不算达标。

4.4.4 越权返回 403 而非 404（NFR-SEC-05）

```
def get_owned_task(db: Session, task_id: int, user_id: int) -> Task:
    task = task_dao.get(db, task_id)
    if task is None:
```

```
        raise BizError(40401, "任务不存在或已被删除")
    if task.user_id != user_id:
        # NFR-SEC-05：返回 403 而非 404，避免通过状态码探测他人任务是否存在
        raise BizError(40301, "无权访问该任务")
    return task
```

4.5 统一异常处理

FR-SYS-01 要求所有接口响应结构一致。实现上需要把三类异常统一收敛：

```
# app/core/errors.py
class BizError(Exception):
    """业务异常。code 为 PRD 定义的业务错误码（400xx~500xx）。"""
    def __init__(self, code: int, message: str, data: dict | None = None):
        self.code = code
        self.message = message
        self.data = data or {}



def register_exception_handlers(app: FastAPI) -> None:
    @app.exception_handler(BizError)
    async def _biz(_, exc: BizError):
        # 4xxxx 业务错误用 HTTP 200 返回，由 body.code 区分（FR-SYS-01）；
        # 401xxx 返 401；5xxxx（服务端故障）返 500（R-09，见下方 BE-20）
        return JSONResponse(status_code=http_status_for(exc.code),
                            content={"code": exc.code, "message": exc.message,
                                     "data": exc.data})


    @app.exception_handler(RequestValidationError)
    async def _pydantic(_, exc: RequestValidationError):
        first = exc.errors()[0]
        field = ".".join(str(x) for x in first["loc"][1:])
        return JSONResponse(status_code=200, content={
            "code": 40012,
            "message": f"参数 {field} 不合法：{first['msg']}",
            "data": {"field": field, "errors": exc.errors()},
        })


    @app.exception_handler(Exception)
    async def _unknown(_, exc: Exception):
        logger.exception("未捕获异常 request_id=%s", request_id_var.get())
        return JSONResponse(status_code=500, content={
            "code": 50001, "message": "服务内部错误，请稍后重试或联系管理员",
            "data": {},
        })
```

🔒 BE-20：**业务错误一律用 HTTP 200 + body.code**。这是 FR-SYS-01 的字面要求，也是前端拦截器设计的依据（api-client.ts 只在 body.code !== 0 时抛 ApiError）。若混用 HTTP 4xx，前端需要同时处理两套判断逻辑。

例外一：HTTP 401 仍需返回真实状态码，因为浏览器与其他中间件依赖它触发重新认证。实现上 BizError(40101/40102) 由 get_current_user 抛出，处理器中对其返回 status_code=401。

例外二（R-09，2026-10-26）：**code >= 50000 返回真实 HTTP 500**。理由是可观测性——统一 200 时 Nginx / ALB / APM 的 5xx 告警规则对本站完全失效，curl -f 也退化为必须人工解析 body。折中范围为「只加 5xx 映射」：4xxxx（含 40301 / 40401 这类带 HTTP 语义的码）继续返回 200，因此前端「一套 body.code 分支」的收益完整保留，既有校验脚本的状态码断言也不受影响。映射的唯一入口是 errors.py::http_status_for()，客户端需注意 5xx 响应改由拦截器的 **error 分支**取 code/message（下载类接口的 Blob 响应要先解 JSON）。

⚠ BE-21：**Exception 兜底处理器中的 message 不得包含堆栈或原始异常文本**（NFR-SEC-09）。原始信息只进日志，响应给用户的是固定文案。

4.6 日志规范

级别	使用场景	示例
DEBUG	开发期细节	SQL 语句、特征矩阵形状（8990, 21, 11）
INFO	关键业务节点	任务 42 完成 耗时=7873ms 设备=cuda 加权 MAPE=14.26%
WARNING	可恢复异常	CUDA OOM，回退 CPU 重跑 task=42、埋点写入失败 event=EV-PRED-02
ERROR	需人工介入	模型加载失败、数据库写入失败

日志格式：%(asctime)s %(levelname)s [%(request_id)s] %(name)s - %(message)s

🔒 BE-22：**禁止在日志中输出密码、Token 原文、密码哈希**（NFR-SEC-09）。实现建议：在 logging.Filter 中对 password / authorization / access_token 等关键字做遮蔽，而非依赖开发者自觉。

4.7 算法层调用契约（★ 核心章节）

本节定义 `services/` 如何调用已实现的 `app/algo/`。这是**在线与离线共用同一份代码**（架构红线 2）的具体落地。

4.7.1 算法层模块职责与关键函数

模块	关键函数	签名要点	调用时机
`preprocess.py`	`validate_dataframe(df)`	校验失败抛 `ValidationError`	上传时（`API-06`）+ 推理前防御性调用
	`clean_dataframe(df)`	插值 + 尖峰截断，返回新 df	推理前
	`detect_gaps(df)`	返回断档信息	上传校验第 6 步（`FR-UPL-12`）
	`truncate_spikes(..., preserve_calendar=...)`	🔒 需传营销节点，见 §4.7.4	训练与推理前
	`resolve_device(prefer="auto")`	返回 `"cuda"` / `"cpu"`	`run_inference` 内部已调用
`classify.py`	`classify_series(df)`	返回 `dict[(sku, wh) -> SeriesInfo]`	推理流水线第 2 步
	`count_levels(infos)`	返回 `LevelCounts`	覆盖率披露
	`coverage_report(infos)`	返回覆盖率 dict	映射到接口 `coverage` 字段
	`estimate_series_count(df)`	返回序列数	**上传时预检**（`FR-UPL-14`）
	`l1_series(infos)` / `l2_l3_series(infos)`	分流	推理前
`features.py`	`build_inference_windows(...)`	构造推理窗口	推理前
	`baseline_values(sales, window)`	返回基线序列	指标计算
	`min_required_points(window)`	返回最少所需点数	数据量校验（`B5`）
	`residual_to_value(residual_norm, base, sigma)`	🔒 残差还原，见 §4.7.4	推理后
`infer.py`	🔑 **`run_inference(...)`**	**在线主入口**	见 §4.7.2

模块	关键函数	签名要点	调用时机
	`load_bundle(path, device)`	返回 `(model, meta)`	启动时预加载
`fallback.py`	`l2_forecast(...)`	商品级降级 + 仓占比分摊	`run_inference` 内部
	`l3_forecast(df, sku, warehouse)`	返回 `(None, meta)`，**不输出数值**	`run_inference` 内部
`metrics.py`	`compute_task_metrics(series_records, horizon, levels_for_main)`	见 §4.7.3	`run_inference` 内部
	`series_mape_distribution(series_records)`	返回直方图数据	结果页分布图
	`verdict(m)`	返回四条硬指标判定	结果页判定条
`train.py`	🔒 **服务进程禁止导入**	训练入口	仅 `scripts/` 使用

4.7.2 在线主入口 run_inference 的完整签名（📌 契约冻结）

```
def run_inference(
    df: pd.DataFrame,                          # 已校验；建议先 clean_dataframe
    horizon: int = 7,                          # 预测步长，取值 7/14/30
    model: torch.nn.Module | None = None,      # None 时全部走规则兜底（用于自检）
    feat_names: tuple[str, ...] | None = None,
    window: int = C.WINDOW_SIZE,               # 生产配置 21
    holiday: bool = True,                      # 对应 use_holiday_feature
    device: str = "auto",                      # auto 优先 CUDA，OOM 自动回退
    target_mode: str = features.TARGET_MODE, # 📌 恒为 'resid'（学残差）
    baseline_window: int = C.BASELINE_WINDOW,# 7，对应前端"基线"曲线
    backtest: bool = False,                    # 🔒 线上必须为 False
    levels_for_main: tuple[str, ...] = ("L1",),  # 🔒 主指标只算 L1
    debug: bool = False,
) -> InferenceResult:
```

返回结构 `InferenceResult` 与下游的三向映射（`JT-04`）——这张表是本节最重要的产出：

`InferenceResult` 字段	内容	→ 接口响应字段（§5）	→ 数据库落点
`records: list[PredictionRecord]`	每条序列的 `sku` / `warehouse` / `level` / `dates` / `forecast` / `note`	`series[]`（含 `granularity_level`）	`predictions` 表逐行

InferenceResult 字段	内容	→ 接口响应字段（§5）	→ 数据库落点
metrics: C.Metrics	加权 MAE/RMSE/MAPE、基线、提升幅度、达标占比、L1/L2 分组 MAPE	metrics（MetricBundle）	metrics 表
coverage: dict	{L1, L2, L3, total, L1_pct, ...}	coverage（Coverage）	tasks.l1_count/l2_count/l3_count
timing: Timing	preprocess_ms / features_ms / inference_ms / metrics_ms / total_ms	timing_ms（TimingMs）	metrics.timing_json + tasks.duration_ms
distribution: dict	中位 / P75 / P90 / 最大 / 超标数 / 分箱	distribution	metrics.distribution_json
warnings: list[str]	「N 条序列历史不足」等提示	⚠ 需并入 eval_note 或新增 warnings 字段，见 TQ-2	——
device: str	实际使用设备（含回退后结果）	task.device	tasks.device
metric_records: list[dict]	逐序列 y_true / y_pred / baseline	明细表数据源	与 predictions 同源

📌 JT-04 的意义：这张表说明**接口层基本不需要做数据加工**——算法产出已经覆盖了结果页 90% 的字段需求。services/predict_service.py 的职责因此被压缩为「落库 + 组装响应」，而非「重新计算」。这直接消除了「算法一套数字、接口另一套数字」的风险。

4.7.3 指标计算的调用方式（🔒 levels_for_main 口径）

```
from app.algo import metrics as algo_metrics


# ✅ 正确：主指标只统计 L1（模型真正负责的部分）
m = algo_metrics.compute_task_metrics(
    series_records=res.metric_records,
    horizon=horizon,
    levels_for_main=("L1",),
)


# ❌ 错误：混入 L2 —— 实测会把 13.6% 拖到 25.9%，导致"模型不达标"的误判
m = algo_metrics.compute_task_metrics(res.metric_records, horizon, ("L1", "L2"))
```

🔒 BE-23：**`levels_for_main` 必须保持默认值 `("L1",)`**。理由已在 `metrics.compute_task_metrics` 的文档字符串中说明：L2 的预测值由规则（商品级均值 × 仓占比）产生，**不代表 LSTM 的能力**。实测 L2 组 MAPE 约 60%，混入主指标会使达标模型整体变成不达标。

结果页必须**同时披露覆盖率**（`FR-VIS-07`），让评审看到「模型负责了 71.7% 的序列，这部分误差 14.26%；其余 28.3% 走规则降级，质量单独列出」。这比一个混合数字更诚实、也更有说服力。

4.7.4 五条不可违反的算法设计约束（改动前必读）

这五条是算法层实测踩坑后的结论，**后端改动若绕过它们会立刻导致指标崩坏**：

编号	约束	违反后果	涉及函数
`ALGO-1`	**学残差**：目标为 `(y − 移动平均基线)/σ`	预测值塌回序列均值（实测 预测 std/真值 std = 0.26）	`features.make_target` / `residual_to_value`
`ALGO-2`	**按序列切分**（`split_by_series`），禁止按样本切	相邻两天近乎相同，必然数据泄漏	`train.split_by_series`
`ALGO-3`	**退化序列过滤**：`MIN_USABLE_SIGMA = 0.5`	σ≈0.03 的近常量序列除完后残差放大十倍，污染全局 loss	`features` 内部
`ALGO-4`	**营销节点保护**：`truncate_spikes(..., preserve_calendar=...)`	无条件截断会削掉双十一尖峰，而那是唯一可学的业务信号	`preprocess.truncate_spikes`
`ALGO-5`	**主指标只算 L1**：`levels_for_main=("L1",)`	13.6% → 25.9%，误判为不达标	`metrics.compute_task_metrics`

🔒 BE-24：**评测必须 `backtest=True`**。`run_inference` 的 `backtest` 参数会把每条序列末尾 `horizon` 天摘掉再预测，使指标与 `train.evaluate_on_holdout` 同口径。**线上服务必须保持 `backtest=False`**（用全部历史预测真正的未来）。混淆二者会得到「看着答案出题」的虚高指标。

4.7.5 模型单例持有

```
# app/core/model_holder.py
import torch
from app.algo import infer as algo_infer


class ModelHolder:
    model = None
    meta: dict = {}
    version: str | None = None
    device: str = "cpu"
    loaded: bool = False


    def clear(self) -> None:
        self.model = None
        self.meta = {}
        self.loaded = False



holder = ModelHolder()



def load_active_model() -> None:
    """启动时调用。失败会抛异常，由 lifespan 传播导致启动失败（BE-06）。"""
    row = registry_dao.get_active()            # model_registry.is_active = 1
    if row is None:
        raise RuntimeError("model_registry 中没有启用版本，请先执行 code/db/seed.sql")


    device = preprocess.resolve_device("auto")
    model, meta = algo_infer.load_bundle(row.file_path, device=device)


    model.eval()                                # 🔒 必须置 eval，否则 dropout 生效
    holder.model = model
    holder.meta = meta
    holder.version = row.version
    holder.device = device
    holder.loaded = True
    logger.info("模型已加载 version=%s device=%s params=%s",
                row.version, device, meta.get("n_params"))
```

🔒 BE-25：`model.eval()` 与 `torch.no_grad()` 二者都要。缺 `eval()` 会让 dropout 在推理时生效（预测随机漂移）；缺 `no_grad()` 会构建计算图（显存翻倍，在 2 GB 显存上有 OOM 风险）。

4.8 预测任务编排实现（services/predict_service.py）

这是整个后端最核心的流程，对应开发方案数据流 2。

```
def create_and_run(db: Session, user, req: CreateTaskRequest) -> CreateTaskResponse:
    started = time.perf_counter()
    deadline = started + config.TASK_BUDGET_MS / 1000


    # ---------- 1. 前置校验 ----------
    dataset = dataset_service.get_owned(db, req.dataset_id, user.id)
    if dataset.valid_series_count > config.MAX_SERIES:
        raise BizError(40004,
            f"待预测序列数为 {dataset.valid_series_count}，超过上限 {config.MAX_SERIES}，"
            f"请先筛选商品后重试",
            {"series_count": dataset.valid_series_count, "series_limit": config.MAX_SERIES})


    if not holder.loaded:
        raise BizError(40404, "模型尚未就绪，请稍后重试或联系管理员")


    # ---------- 2. 建任务（先落 running，保证异常时有记录可查）----------
    task = task_dao.create(db, user_id=user.id, dataset_id=dataset.id,
                           horizon=req.horizon, granularity="warehouse_sku",
                           use_holiday_feature=req.use_holiday_feature,
                           task_name=req.task_name, status="running",
                           model_version=holder.version,
                           started_at=now_str())
    db.commit()


    try:
        # ---------- 3. 读数据 ----------
        df = dataset_service.read_csv(dataset)          # 跳过重复校验


        # ---------- 4. 算法流水线（唯一计算入口）----------
        result = algo_infer.run_inference(
            df,
            horizon=req.horizon,
            model=holder.model,
            feat_names=holder.meta.get("feat_names"),
            holiday=bool(req.use_holiday_feature),
            device=holder.device,
            backtest=False,                              # 🔒 线上恒为 False
            levels_for_main=("L1",),                     # 🔒 见 BE-23
        )


        # ---------- 5. 预算检查（软失败）----------
        if time.perf_counter() > deadline:
            return _fail_task(db, task, "E_TIMEOUT", "validate", "metrics",
                              f"处理耗时超过 {config.TASK_BUDGET_MS // 1000} 秒预算，"
                              f"建议减少序列数后重试")
```

```
        # ---------- 6. 落库（批量，NFR-REL-04） ----------
        rows = _to_prediction_rows(task.id, result)      # list[dict]
        prediction_dao.bulk_insert(db, rows, chunk=1000)


        task.l1_count = result.coverage["L1"]
        task.l2_count = result.coverage["L2"]
        task.l3_count = result.coverage["L3"]
        task.series_total = result.coverage["total"]
        task.device = result.device
        task.device_fallback = int(result.device != holder.device)   # 发生回退
        task.duration_ms = result.timing.total_ms
        task.status = "succeeded"
        task.finished_at = now_str()


        metric_dao.create(db, task_id=task.id,
                          mape=result.metrics.mape, mae=result.metrics.mae,
                          rmse=result.metrics.rmse,
                          baseline_mae=result.metrics.baseline_mae,
                          baseline_rmse=result.metrics.baseline_rmse,
                          mae_improve_pct=result.metrics.mae_improve_pct,
                          rmse_improve_pct=result.metrics.rmse_improve_pct,
                          series_evaluated=result.metrics.series_evaluated,
                          series_passed=result.metrics.series_passed,
                          series_pass_rate=result.metrics.series_pass_rate,
                          l1_mape=result.metrics.l1_mape, l2_mape=result.metrics.l2_mape,
                          is_over_threshold=int(result.metrics.mape > 20.0),
                          eval_note=EVAL_NOTE,
                          distribution_json=json.dumps(result.distribution, ensure_ascii=False),
                          timing_json=json.dumps(result.timing.as_dict()))
        db.commit()


        emit_event("EV-PRED-02", task_id=task.id, **{...})    # 埋点不得阻塞
        return CreateTaskResponse(task_id=task.id, status="succeeded",
                                  model_version=holder.version,
                                  series_count=task.series_total,
                                  estimated_ms=result.timing.total_ms)


    except torch.cuda.OutOfMemoryError as e:
        logger.warning("CUDA OOM task=%s，回退 CPU 重跑", task.id)
        return _retry_on_cpu(db, task, req, df)
    except Exception as e:
        return _fail_task(db, task, "E_INFER_FAIL", "validate", "features", str(e))
```

关键实现要点

编号	要点	理由
BE-26	**先落 `running` 再算**，且立即 `commit`	若进程在处理中崩溃，数据库里仍有一条 `running` 记录可供排查；不先落库则任务凭空消失
BE-27	批量插入 `chunk=1000`	`NFR-REL-04`。30,000 行逐条 commit

编号	要点	理由
		会因每次 fsync 而耗时十几秒
BE-28	软失败返回正常响应（HTTP 200 + `code:0`，任务状态为 failed）	`NFR-REL-02` 要求不给 502/504。前端据 `task.status` 渲染失败态
BE-29	`device_fallback` 判定用 `result.device != holder.device`	回退发生在 `run_inference` 内部，返回值才是真相。不能用「是否捕获异常」判断
BE-30	OOM 重试时**必须重建 `df` 或重置索引**	首次失败可能已修改 df（如插值），复用会导致重复插值

4.8.1 EVAL_NOTE 固定文案（🔒 FR-MOD-13）

```
EVAL_NOTE = (
    "本结果使用预训练模型推理，未针对本次输入数据微调。"
    "模型基于历史数据训练，对突发外部事件（政策、竞品、疫情）无预测能力。"
)
```

必须同时出现在：结果页（`FR-SYS-07`）、历史详情页、导出文件首行注释（`FR-VIS-05`）。

4.9 上传与校验实现（services/dataset_service.py + app/validators/）

📌 `R-06`（2026-10-26）：7 步校验已从 `dataset_service.py` 拆到独立的 `app/validators/` 包。

拆分前 `inspect_file()` 一个函数 273 行、与上传落盘同处一个 1,366 行的模块，

后果是「加第 8 步校验必须先读懂前 7 段共享的 6 个局部变量在哪个分支被赋值」——

属于加一步就多一处回归风险的写法。

现在职责是：`steps.py` 里**一步一个函数**（只判定、只造 `Issue`），

`pipeline.inspect_file()` **只做编排与短路返回**（约 60 行），

`dataset_service` 负责落盘 / 写库 / 断档处置 / 错误明细导出。

加第 8 步 = 新增一个 `check_xxx()` + 在 pipeline 里插一行。

校验层不依赖数据库，因此可以用临时 CSV 直接单测（tests/test_validators.py，17 例）。

4.9.1 七步校验顺序（🔒 不可调整，FR-UPL-09~FR-UPL-14）

```
def validate_upload(file: UploadFile, df: pd.DataFrame) -> None:
    """按固定顺序校验，任一步失败即抛错终止，不继续后续校验。"""
    # 1. 扩展名
    if not file.filename.lower().endswith(".csv"):
        raise BizError(40009, "仅支持 .csv 格式文件，请另存为 CSV 后重试")


    # 2. 大小 / 行数
    if file.size > config.MAX_FILE_SIZE_MB * 1024 * 1024:
        raise BizError(40007, f"文件超过 {config.MAX_FILE_SIZE_MB}MB 上限")
    if len(df) > config.MAX_ROWS:
        raise BizError(40008, f"数据行数超过 {config.MAX_ROWS:,} 行上限")


    # 3. 表头完整性
    missing = [c for c in config.REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        raise BizError(40001,
            f"缺少必需字段：{'、'.join(missing)}",
            {"fail_stage": "表头完整性", "missing_columns": missing,
             "suggestion": "请下载标准模板，确认表头为 "
                           "sku_id, warehouse_id, date, sales 后重新上传"})


    # 4. 日期可解析性 → 5. 销量非负 → 6. 时间连续性 → 7. 序列数上限
    #    均由 algo.preprocess.validate_dataframe 统一实现（在线离线共用）
    try:
        preprocess.validate_dataframe(df)
    except preprocess.ValidationError as e:
        raise BizError(e.code, e.message, e.data)


    n = classify.estimate_series_count(df)
    if n > config.MAX_SERIES:
        raise BizError(40004, f"待预测序列数为 {n}，超过上限 {config.MAX_SERIES}，请先筛选商品",
                       {"series_count": n, "series_limit": config.MAX_SERIES})
```

📌 BE-31：**校验逻辑委托给 algo.preprocess，不在 service 层重写。**理由与架构红线 2 一致：上传时的校验规则必须与算法内部的一致，否则会出现「上传通过了但训练报错」。校验顺序的常量定义在 algo/config.py 的 VALIDATE_ORDER。

🔒 BE-32：**错误信息必须含「错在哪 + 怎么改」**（NFR-USE-03）。suggestion 字段不得为空。这条要求会被§9.5 的测试用例逐项检查。

4.9.2 断档处置（FR-UPL-13）

```
# 上传时若检测到断档，不自动继续，而是返回待确认状态
gap_info = preprocess.detect_gaps(df)
if gap_info.has_gap and not resolve_action:
    return UploadDatasetResponse(
        dataset=..., preview=...,
        gap_warning=GapWarning(has_gap=True, gap_count=gap_info.count,
                               ranges=gap_info.ranges, interpolated=False),
        ...
    )
# 用户选择"自动插值"后：置 datasets.gap_filled=1 再进入参数页
```

🔒 BE-33：**用户未选择时不得自动继续**（FR-UPL-13 明确要求）。前端据 gap_warning.has_gap 渲染二选一弹窗。

4.9.3 流式落盘（FR-UPL-04）

```
async def save_upload(file: UploadFile, user_id: int) -> tuple[str, str, int]:
    """流式写入，不一次性读入内存。返回 (stored_name, rel_path, size)。"""
    stored = f"{uuid.uuid4().hex}.csv"
    rel = f"uploads/u{user_id}/{stored}"          # NFR-SEC-07：按用户分目录
    abs_path = config.BASE_DIR / "data" / rel
    abs_path.parent.mkdir(parents=True, exist_ok=True)

    size = 0
    async with aiofiles.open(abs_path, "wb") as out:
        while chunk := await file.read(1024 * 1024):     # 1 MB 分块
            size += len(chunk)
            if size > config.MAX_FILE_SIZE_MB * 1024 * 1024:
                await out.close()
                abs_path.unlink(missing_ok=True)         # 超限即时清理
                raise BizError(40007, f"文件超过 {config.MAX_FILE_SIZE_MB}MB 上限")
            await out.write(chunk)
    return stored, rel, size
```

BE-34：**边写边检查大小**。若先写完再检查，20 MB 的限制形同虚设——攻击者上传 10 GB 文件时会先写满磁盘。同时超限必须删除已写入的临时文件。

4.10 导出实现（services/export_service.py）

🔒 列顺序固定（FR-VIS-05）：

```
sku_id, warehouse_id, date, forecast_sales, granularity_level, actual_sales, error
```

```
EXPORT_COLUMNS = ["sku_id", "warehouse_id", "date", "forecast_sales",
                  "granularity_level", "actual_sales", "error"]


def build_export(db: Session, task: Task) -> pd.DataFrame:
    rows = prediction_dao.list_by_task(db, task.id)
    df = pd.DataFrame(rows)
    df["error"] = (df["actual_value"] - df["predicted_value"]).abs()
    out = df.rename(columns={
        "forecast_date": "date",
        "predicted_value": "forecast_sales",
        "actual_value": "actual_sales",
    })[EXPORT_COLUMNS]
    return out


def to_csv_bytes(df: pd.DataFrame) -> bytes:
    # 🔒 NFR-COMP-05：必须带 BOM，否则 Excel 打开中文乱码
    text = df.to_csv(index=False)
    return ("\ufeff" + text).encode("utf-8")


def to_xlsx_bytes(df: pd.DataFrame) -> bytes:
    buf = io.BytesIO()
    with pd.ExcelWriter(buf, engine="openpyxl") as w:
        df.to_excel(w, index=False, sheet_name="预测结果")
    return buf.getvalue()
```

编号	要点	理由
BE-35	CSV 必须带 UTF-8 BOM	NFR-COMP-05 明确要求「CSV 导出带 BOM，保证 Excel 打开中文不乱码」。这是极高频的实际问题
BE-36	导出前校验任务归属	只能导出自己的任务（NFR-SEC-02）
BE-37	导出文件写入 data/exports/ 并记录，供 TTL 清理	NFR-REL-05。小文件（< 5 MB）可直接走 StreamingResponse 不落盘，避免磁盘堆积

编号	要点	理由
BE-38	`actual_value` 为 NULL 时导出为空单元格，**不填 0**	填 0 会让用户误以为预测误差极大

4.11 埋点实现（core/telemetry.py）

```
def emit_event(name: str, *, user_id: int | None = None,
               task_id: int | None = None, **props) -> None:
    """埋点写入。🔒 失败仅记日志，绝不抛异常（PRD §5.1 / DB-T08-02）。"""
    try:
        with SessionLocal() as s:
            s.add(Event(event_name=name, user_id=user_id, task_id=task_id,
                        properties=json.dumps(props, ensure_ascii=False, default=str)))
            s.commit()
    except Exception:
        logger.warning("埋点写入失败 event=%s task=%s", name, task_id, exc_info=False)
```

18 个事件与埋点位置对照（PRD §5.1）：`EV-PRED-02` 是最核心的埋点，携带 `duration_ms` / `l1_count` / `l2_count` / `l3_count` / `weighted_mape`，直接支撑「北极星指标：有效预测达成率 ≥ 80%」的计算。

> ⚠ `TQ-3`：埋点采用**同步写库**（简单、必达）还是**异步队列**（不阻塞）？当前实现是同步写 + 异常吞掉。鉴于单机小规模场景，建议保持同步；若后续 QPS 上升，再改为 `BackgroundTasks`。见附录 D。

---

5. 接口设计

> 📌 本章是前端与后端的**唯一契约来源**。配套代码：后端 `app/schemas/`（Pydantic 模型**真源**，`code/backend/schemas.py` 是它生成的独立副本）、前端 `code/frontend/types.ts`（TS 类型）、`code/frontend/api-client.ts`（调用封装）。三者字段名必须完全一致。

5.1 通用约定

编号	项	约定
API-00-1	**Base Path**	`/api/v1`（`/health` 除外，挂根路径）
API-00-2	**认证**	`Authorization: Bearer <token>`。除注册、登录、`/health` 外全部需要
API-00-3	**数据格式**	请求与响应均为 `application/json; charset=utf-8`；上传为 `multipart/form-data`
API-00-4	**日期格式**	日期 `YYYY-MM-DD`；时间戳 `YYYY-MM-DD HH:MM:SS`；**全部本地时区**（`NFR-COMP-04`）
API-00-5	**字段命名**	全链路 `snake_case`（数据库 / Pydantic / JSON / TS 四处同名）
API-00-6	**分页参数**	`page`（默认 1）、`size`（默认 20，上限 100）
API-00-7	**字符编码**	全链路 UTF-8；CSV 导出带 BOM
API-00-8	**接口文档**	由 FastAPI 自动生成 OpenAPI，`/docs` 与 `/redoc` 可访问（`FR-SYS-02`）

5.1.1 统一响应包（🔒 FR-SYS-01）

成功：

```
{ "code": 0, "message": "ok", "data": { } }
```

失败：

```
{
  "code": 40001,
  "message": "缺少必需字段：warehouse_id（仓库编码）",
  "data": {
```

```
    "fail_stage": "表头完整性",
    "missing_columns": ["warehouse_id"],
    "suggestion": "请下载标准模板，确认表头为 sku_id, warehouse_id, date, sales 后重新上传"
  }
}
```

API-00-9：**业务错误返回 HTTP 200**，由 code 区分成功与否。例外有两处：① 401（需真实状态码以触发前端重新认证）；② code >= 50000（服务端故障，需真实 500 才能被 APM / 负载均衡的 5xx 规则捕获，R-09）。理由与边界见 BE-20。

5.1.2 分页响应结构

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "items": [ ],
    "page": 1,
    "size": 20,
    "total": 45,
    "pages": 3
  }
}
```

5.2 错误码全表（FR-SYS-02）

区间	含义	码	常量名	场景
0	成功	0	OK	——
400xx	**请求 / 数据校验**	40001	MISSING_COLUMN	缺少必需列
		40002	DATE_UNPARSABLE	日期无法解析
		40003	NEGATIVE_VALUE	销量存在负值
		40004	SERIES_OVER_LIMIT	序列数超上限
		40005	DATE_GAP	时间序列断档
		40006	INSUFFICIENT_DATA	数据量不足以预测
		40007	FILE_TOO_LARGE	文件超 20 MB

区间	含义	码	常量名	场景
		40008	ROW_OVER_LIMIT	行数超 500,000
		40009	BAD_EXTENSION	扩展名非 .csv
		40010	EMPTY_FILE	空文件
		40011	WEAK_PASSWORD	密码强度不足
		40012	BAD_PARAM	参数非法（如 horizon=15）
401xx	**认证**	40101	UNAUTHORIZED	未登录 / 凭证无效
		40102	TOKEN_EXPIRED	Token 失效
		40103	BAD_CREDENTIAL	邮箱或密码错误
		40104	LOGIN_LOCKED	限流冷却中
		40105	EMAIL_TAKEN	邮箱已注册
403xx	**权限**	40301	FORBIDDEN	无权访问他人资源（NFR-SEC-05：返回 403 而非 404）
404xx	**不存在**	40401	TASK_NOT_FOUND	任务不存在
		40402	DATASET_NOT_FOUND	数据集不存在
		40403	USER_NOT_FOUND	用户不存在
		40404	MODEL_NOT_READY	无启用模型版本
500xx	**服务内部**	50001	INFER_FAIL	模型推理失败
		50002	IO_ERROR	文件读写异常
		50003	DB_ERROR	数据库异常

任务级失败码（写入 tasks.error_code，与业务错误码是两套体系）：

码	含义	前端表现
E_TIMEOUT	端到端超 60 秒	失败态 + 已完成阶段 + 「减少序列数」建议
E_TOO_MANY_SERIES	序列数超上限	引导回上传页筛选商品
E_INFER_FAIL	推理异常	失败态 + 错误摘要 + 「原参数重试」按钮

5.3 接口清单总览（21 个既有 + 9 个管理端 = 30 个）

📌 **与开发方案 §6.2.3 的差异**：开发方案列出 17 个接口。本技术文档扩充为 **21 个**，新增 4 个；CH-01 管理端落地后（EP-8）再增 **9 个**（API-22~API-30），合计 **30 个**。理由如下。

#	方法	路径	名称	鉴权	优先级	需求
API-01	POST	/auth/register	注册	否	P0	FR-USR-01/02/03
API-02	POST	/auth/login	登录	否	P0	FR-USR-04/06
API-03	POST	/auth/logout	登出	是	P0	FR-USR-05
API-04	GET	/users/me	当前用户	是	P1	FR-USR-07
API-05	PATCH	/users/me	修改资料/密码	是	P1	FR-USR-07
API-06	POST	/datasets	上传 CSV	否 *	P0	FR-UPL-01~15
API-07	GET	/datasets/sample	加载示例数据	否 *	P0	FR-UPL-02
API-08	GET	/datasets/template	下载模板	否	P1	FR-UPL-08
API-09	GET	/datasets/{dataset_id}	数据集详情	是	P0	🆕 参数页数据概要
API-10	GET	/datasets/{dataset_id}/preview	数据预览	是	P1	FR-UPL-07
API-11	GET	/datasets/{dataset_id}/errors	错误明细下载	是	P1	🆕 FR-UPL-06
API-12	POST	/predictions	创建预测任务	是	P0	FR-MOD-07~13
API-13	GET	/predictions/{task_id}/status	任务状态	是	P0	进度页
API-14	GET	/predictions/{task_id}	预测结果	是	P0	FR-VIS-01~09
API-15	GET	/predictions/{task_id}/detail	结果明细分页	是	P0	🆕 FR-VIS-03
API-16	GET	/predictions/{task_id}/export	导出结果	是	P0	FR-VIS-05
API-17	GET	/history	历史列表	是	P0	FR-HIS-01

#	方法	路径	名称	鉴权	优先级	需求
API-18	POST	/history/compare	跨任务对比	是	P1	🆕 FR-HIS-04
API-19	GET	/history/{task_id}	历史详情	是	P0	FR-HIS-02
API-20	DELETE	/history/{task_id}	删除历史（软删除）	是	P1	FR-HIS-05
API-21	GET	/health	健康检查	否	P0	FR-SYS-04

管理端（CH-01，EP-8 落地）—— 🔒 隔离红线的唯一显式例外

编号	方法	路径	名称	鉴权	优先级	对应需求
API-22	GET	/admin/overview	运营看板概览	**管理员**	P1	FR-ADM-01
API-23	GET	/admin/users	用户列表	**管理员**	P1	FR-ADM-03
API-24	PATCH	/admin/users/{user_id}	启停用 + 角色变更	**管理员**	P1	FR-ADM-04 / FR-ADM-05
API-25	POST	/admin/users/{user_id}/reset-password	重置密码	**管理员**	P1	FR-ADM-06
API-26	GET	/admin/tasks	全量任务列表	**管理员**	P1	FR-ADM-07
API-27	DELETE	/admin/tasks/{task_id}	强制删除任务（硬删除）	**管理员**	P1	FR-ADM-08
API-28	GET	/admin/models	模型版本列表	**管理员**	P1	FR-ADM-09
API-29	PATCH	/admin/models/{version}	切换启用版本	**管理员**	P1	FR-ADM-10
API-30	GET	/admin/logs	操作日志查询	**管理员**	P1	FR-ADM-11

🔒 **鉴权列的「管理员」不是普通登录**：指 get_current_admin（§4.3）—— 在 get_current_user 之上强制 role == 'admin'，非管理员返 40301、未登录返 40101（**先认证后授权**）。

🔒 **BE-ADM-02（API-29 事务顺序硬约束）**：`model_registry` 上有部分唯一索引 `ux_model_active ON model_registry (is_active) WHERE is_active = 1`，保证「全局有且仅有一个启用版本」。切换必须**在同一事务内先降旧、再升新** —— ⏱ 实测把两个 `is_active` 都置 1 会被 `UNIQUE constraint failed` 拒绝；**先升新再降旧同样会撞索引**，所以这不是风格偏好而是数据库强制的。

🔒 **BE-ADM-03（切换后必须校验 .env）**：`.env` 的 `ACTIVE_MODEL` 与库不一致会导致「数据库说 v2、进程实际加载 v1」的静默故障。接口在提交后触发 `reload_active_model()` 热切换；若 `.env` 未同步则**仍返回成功但带 warnings** —— 把 `.env` 写进数据库不是接口该做的事，而悄悄失败更糟。

📌 **API-28 与 API-29 必须分离**（QD-07）：单路径承担读写两种语义会让权限与缓存策略无法各自独立演进。

* 🔒 **游客态（FR-USR-08 / JT-10，已决议）**：API-06 与 API-07 允许未登录调用，跑到结果页；但导出（API-16）与历史（API-17~API-19）**仍然要求登录。**实现方式——这两个接口使用 `OptionalUserId` 依赖（§4.3 `get_optional_user`），未登录或 Token 失效时归属写为 `user_id = 0`（游客哨兵）。数据归属判定必须走 BE-41 的两个分支顺序，不得简化。

⚠ **路由注册顺序约束（BE-39）**：`API-18 /history/compare` **必须注册在** `API-19 /history/{task_id}` **之前。**否则 FastAPI 会把字符串 `compare` 当作 `task_id` 解析，导致参数校验失败（报 40012 而非进入对比逻辑）。这是 FastAPI 路径匹配的实现细节，属于高频踩坑点。

```
# app/api/v1/history.py —— 顺序不可调换
router.add_api_route("/history", list_history, methods=["GET"])          # API-17
router.add_api_route("/history/compare", compare, methods=["POST"])      # API-18 ← 必须在下一行之前
router.add_api_route("/history/{task_id}", get_detail, methods=["GET"])   # API-19
router.add_api_route("/history/{task_id}", remove, methods=["DELETE"])    # API-20
```

新增接口的理由

新增	对应需求	理由
API-09 数据集详情	参数页 UX-P06	参数配置页的「数据概要」右栏需要

新增	对应需求	理由
		商品数 / 仓库数 / 序列数 / 分级预判。`preview` 接口只返回行数据，不含统计概要。原型 `params.html` 右栏展示了这些字段，说明需求真实存在
`API-11` 错误明细下载	`FR-UPL-06`	PRD 明确要求「提供错误明细下载，把不合格行导出为 CSV」。开发方案 §6.2.3 遗漏
`API-15` 结果明细分页	`FR-VIS-03`	PRD 要求明细表「支持按任意列排序、分页（默认每页 20 行）」。若并入 `API-14`，单响应会包含全部明细（最大 30,000 行），响应体可达数十 MB
`API-18` 跨任务对比	`FR-HIS-04`	PRD 要求「选择两个历史任务，在同一坐标系叠加对比预测曲线」。开发方案 §6.2.3 遗漏，但原型已实现页面 `UX-P11`

5.4 认证类接口（API-01 ~ API-05）

API-01 用户注册

项	值
方法 / 路径	`POST /api/v1/auth/register`
鉴权	否
用途	邮箱 + 密码注册（`FR-USR-01`/`02`/`03`）
耗时预算	≤ 500 ms（含 bcrypt 哈希约 250 ms）

请求体

字段	类型	必填	约束	说明
`email`	string	是	合法邮箱，≤ 128	全局唯一，大小写不敏感
`password`	string	是	8–64 位，含字母与数字	强度不足返回 `40011`
`nickname`	string	否	≤ 32	为空时默认取邮箱 @ 前缀

```
{ "email": "demo@example.com", "password": "Demo12345", "nickname": "演示账号" }
```

响应体

字段	类型	说明
`user_id`	integer	新用户 ID
`email`	string	归一化后的邮箱（小写）

```
{ "code": 0, "message": "ok", "data": { "user_id": 1, "email": "demo@example.com" } }
```

错误码

码	触发条件	message 示例
`40011`	密码强度不足	`密码需不少于 8 位，且同时包含字母与数字`
`40105`	邮箱已注册	`该邮箱已注册，请直接登录或使用其他邮箱`
`40012`	邮箱格式非法	`参数 email 不合法：value is not a valid email address`

API-01-1：**邮箱冲突判定必须在一次查询内完成**，不能「先查再插」——并发注册会产生竞态。实现上用 INSERT 捕获唯一索引冲突（IntegrityError）后转换为 40105。

🔒 API-01-2（R-07，2026-10-26）：**注册必须限流**，且**判定要前置于 bcrypt**。理由：公开部署后本接口是唯一不需要任何凭证的写入口，而 register() 里在入库之前就要算一次 bcrypt（cost 12，约 250 ms）——不限流时，「反复提交、每次都让服务端算一次哈希」本身就是一次 CPU DoS，且刷失败请求连审计都不会留下（失败不写 operation_logs）。

实现为**两层**，缺一不可：

层	管什么	计数位置	默认额度	为什么必须在这一层
库层	真实建出来的账号数	operation_logs 中 action='register' 的成功条数	10 / 小时 / IP	必须抗重启，否则等一次发布就能重置配额
内存层	**每一次尝试**（含注定失败的）	auth_service._REG_ATTEMPTS 固定窗口	30 / 小时 / IP	挡 bcrypt 的 CPU 消耗；若也落库，被刷时审计表先爆，等于把「耗算力」升级成「耗磁盘 + 污染审计」

可用 REGISTER_MAX_SUCCESS / REGISTER_MAX_ATTEMPTS / REGISTER_WINDOW_SECONDS 调整，置 0 关闭对应层。超限复用错误码 40104（前端镜像里它的语义就是「限流冷却中」），data.retry_after_seconds 给出剩余秒数。

API-02 用户登录

项	值
方法 / 路径	POST /api/v1/auth/login
鉴权	否
用途	登录并签发 JWT（FR-USR-04），含限流（FR-USR-06）
耗时预算	≤ 800 ms

请求体

字段	类型	必填	说明
`email`	string	是	登录邮箱
`password`	string	是	明文密码，服务端比对哈希

响应体

字段	类型	说明
`access_token`	string	JWT
`token_type`	string	固定 `bearer`
`expires_in`	integer	有效期（秒），默认 43200（12 小时）
`user`	object	`UserProfile`，见下

`UserProfile` 字段：`id`、`email`、`nickname`、`status`、`last_login_at`、`created_at`。

🔒 **响应中不得包含 `password_hash`（`NFR-SEC-09`）。**

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "token_type": "bearer",
    "expires_in": 43200,
    "user": {
      "id": 1, "email": "demo@example.com", "nickname": "演示账号",
      "status": 1, "last_login_at": "2026-10-08 17:20:11", "created_at": "2026-10-01 09:12:00"
    }
  }
}
```

错误码

码	触发条件	message 示例
40103	邮箱不存在或密码错误	邮箱或密码错误，请重试（⚠ 不区分二者，防账号枚举）
40104	冷却期内	密码连续错误次数过多，请 42 秒后重试

API-02-1：错误提示**不区分「邮箱不存在」与「密码错误」**，统一为「邮箱或密码错误」。这是标准的安全实践，避免通过响应差异枚举已注册邮箱。

API-02-2：冷却提示必须给出**剩余秒数**（FR-USR-06 验收要点）。

API-03 用户登出

项	值
方法 / 路径	POST /api/v1/auth/logout
鉴权	是
用途	使当前 Token 立即失效（FR-USR-05）
请求体	无

响应体

```
{ "code": 0, "message": "ok", "data": null }
```

API-03-1：实现采用 **Token 黑名单**（BE-19 的方案 B）：登出时把 Token 的 jti 写入黑名单，鉴权链路（deps.get_auth_context / get_optional_user_id）在解出载荷后立即查询。

🔒 **R-03（2026-10-26）起黑名单落库**（revoked_tokens，DB-T09），不再是进程内存。原实现有两个静默失效的场景：① 进程重启 → 已登出的 Token 又恢复可用（直到 exp）；② 多 worker → A 进程登出、旧 Token 打 B 进程照样放行。两者都不报错，只是让「登出」不可靠。落库后多进程部署无需改动，也无需引入 Redis。

⚠ 分层代价：app/core/security.py 按设计不持有 Session，因此 decode_access_token() **只做无状态校验**（签名 / 过期 / 签发者），不查黑名单。任何绕过 deps 直接调用它的新代码都不会自动获得登出保护 —— 这是一处刻意的边界，不是遗漏。

API-04 获取当前用户信息

项	值
方法 / 路径	GET /api/v1/users/me
鉴权	是
用途	前端刷新页面后恢复登录态（FR-USR-07）

响应体：UserProfile，同 API-02 的 user 字段。

API-04-1：工作台（UX-P04）与页头用户菜单的数据源。前端在 App.tsx 挂载时调用一次，失败（401）则跳登录。

API-05 修改个人信息 / 密码

项	值
方法 / 路径	`PATCH /api/v1/users/me`
鉴权	是
用途	昵称与密码修改（`FR-USR-07`）

请求体（部分更新语义，未传字段不变）

字段	类型	必填	约束	说明
`nickname`	string	否	≤ 32	修改昵称
`old_password`	string	条件	——	修改密码时**必填**
`new_password`	string	条件	8–64 位，含字母与数字	修改密码时必填

响应体：更新后的 `UserProfile`。

错误码

码	触发条件
`40103`	未提供 `old_password` 或原密码错误
`40011`	新密码强度不足

🔒 `API-05-1`：**修改密码后原 Token 必须失效，强制重新登录**（`FR-USR-07` 验收要点）。实现方案见 `BE-19` 方案 C——需在 `users` 表增加 `token_version` 字段（`TQ-1`）。若不采纳该方案，则退而求其次：改密后把该用户所有已签发 Token 的 `sub` 加入黑名单（不现实，无法枚举），**故推荐采纳方案 C**。

5.5 数据集接口（API-06 ~ API-11）

API-06 上传 CSV 数据集

项	值
方法 / 路径	`POST /api/v1/datasets`
鉴权	🔓 否（游客可体验，`FR-USR-08`）。依赖 `OptionalUserId`；未登录时 `datasets.user_id = 0`
Content-Type	`multipart/form-data`
用途	上传 + 七步校验 + 落盘 + 预览（`FR-UPL-01~15`）
耗时预算	≤ 10 s（`NFR-PERF-02` 读取校验预算）

请求

参数	类型	必填	约束	说明
`file`	file	是	`.csv`，≤ 20 MB，≤ 500,000 行	单文件
`gap_action`	string	否	`interpolate` / `abort`	断档处置（`FR-UPL-13`）。首次上传不传

请求示例

```
curl -X POST http://127.0.0.1:8000/api/v1/datasets \
  -H "Authorization: Bearer <token>" \
  -F "file=@sales.csv"
```

响应体

字段	类型	说明
`dataset`	object	`DatasetSummary`（见附录 B）
`preview`	object	`{columns, rows, total, shown}`，默认前 20 行
`series_count`	integer	待预测序列数（`sku_id` × `warehouse_id` 去重）
`series_limit`	integer	上限，固定 1000
`gap_warning`	object	`{has_gap, gap_count, ranges[], interpolated}`


```
{
  "code": 0,
  "message": "ok",
  "data": {
    "dataset": {
      "id": 7, "file_name": "demo_sales.csv", "file_size": 490291,
      "row_count": 14305, "sku_count": 40, "warehouse_count": 5,
      "valid_series_count": 113, "date_start": "2015-01-28", "date_end": "2015-12-27",
      "gap_filled": 0, "gap_count": 0, "is_sample": 0, "status": 1,
      "created_at": "2026-10-09 10:15:32"
    },
    "preview": {
      "columns": ["sku_id", "warehouse_id", "date", "sales"],
      "rows": [
        { "sku_id": "item_0000", "warehouse_id": "上海仓", "date": "2015-01-28", "sales": 11 },
        { "sku_id": "item_0000", "warehouse_id": "上海仓", "date": "2015-01-29", "sales": 11 }
      ],
      "total": 14305, "shown": 2
    },
    "series_count": 113,
    "series_limit": 1000,
    "gap_warning": { "has_gap": false, "gap_count": 0, "ranges": [], "interpolated": false }
  }
}
```


五类校验错误响应示例

① 缺列（`40001`）：


```
{
  "code": 40001,
```

```
  "message": "缺少必需字段：warehouse_id（仓库编码）",
  "data": {
    "fail_stage": "表头完整性",
    "missing_columns": ["warehouse_id"],
    "suggestion": "本系统要求按「仓库 × 商品」粒度预测，仓库列必填。请补齐后重传，或下载标准模板"
  }
}
```

② 日期异常（40002）：

```
{
  "code": 40002,
  "message": "日期列存在 3 个无法解析的值",
  "data": {
    "fail_stage": "日期可解析性",
    "unparsable_rows": [
      { "row_no": 128, "raw_value": "2015/13/45", "column": "date" },
      { "row_no": 892, "raw_value": "未知", "column": "date" },
      { "row_no": 1204, "raw_value": "", "column": "date" }
    ],
    "suggestion": "支持格式为 YYYY-MM-DD（推荐），请修正上述行后重新上传。仅展示前 20 条"
  }
}
```

③ 负值（40003）：

```
{
  "code": 40003,
  "message": "销量列存在 2 个负值",
  "data": {
    "fail_stage": "销量非负",
    "negative_rows": [
      { "row_no": 507, "value": -12.0 },
      { "row_no": 1611, "value": -3.5 }
    ],
    "suggestion": "系统不会自动取绝对值，请修正数据后重新上传（负销量通常源于退货未冲销）"
  }
}
```

④ 断档（40005）：

```
{
```

```
  "code": 40005,
  "message": "检测到 2 处日期断档，需确认处理方式",
  "data": {
    "fail_stage": "时间连续性",
    "gap_ranges": [
      { "sku_id": "item_0000", "warehouse_id": "上海仓",
        "gap_start": "2015-03-05", "gap_end": "2015-03-12", "gap_days": 8 },
      { "sku_id": "item_0033", "warehouse_id": "成都仓",
        "gap_start": "2015-07-01", "gap_end": "2015-07-03", "gap_days": 3 }
    ],
    "suggestion": "请选择「自动插值补齐」或「放弃上传」。选择插值将用线性插值填补缺失日期"
  }
}
```

⑤ 序列超限（`40004`）：

```
{
  "code": 40004,
  "message": "待预测序列数为 1,482，超过上限 1000",
  "data": {
    "fail_stage": "序列数上限",
    "series_count": 1482,
    "series_limit": 1000,
    "suggestion": "请在上传前筛选商品，将序列数控制在 1000 以内。当前超出 482 条"
  }
}
```

🔒 API-06-1：**五类错误必须都能定位到具体行/列**（FR-UPL-05/10/11/12）。前端据此渲染可读提示，否则用户无法修正数据。`unparsable_rows` 最多返回前 20 条（FR-UPL-10）。

API-06-2：校验失败**不落盘、不写库**。只有全部通过才创建 `datasets` 记录。

API-07 加载内置示例数据

项	值
方法 / 路径	`GET /api/v1/datasets/sample`

项	值
鉴权	否 * （FR-USR-08）
用途	一键加载示例数据跑通全流程（FR-UPL-02）
耗时预算	≤ 2 s

请求参数：无。

响应体：结构同 API-06 的响应，其中 dataset.is_sample = 1。

幂等语义

场景	行为
该用户首次调用	创建一条 datasets 记录，is_sample=1，file_path 指向共享文件 outputs/models/demo_sales.csv
该用户再次调用	**返回同一条记录**，不重复创建、不复制文件

📌 API-07-1：示例数据**不预置**到数据库，而是按**当前调用者**惰性创建（FR-UPL-02 / JT-10）：

· 登录用户首次调用 → 创建 user_id = 该用户 的记录

· 游客首次调用 → 创建 user_id = 0 的记录

两种身份各自持有一条指向**同一共享文件** outputs/models/demo_sales.csv 的记录，重复调用返回已存在的那条（幂等）。这样既保证隔离语义，又避免为每个用户复制一份 479 KB 文件。

⚠ 注意幂等的判定键是 (user_id, is_sample = 1)，**不是** is_sample = 1——否则游客先创建后，登录用户会复用游客的记录，导致历史列表归属错乱。

API-07-2：示例数据事实（编码期可用于核对）：**14,305 行 / 40 商品 / 5 仓库 / 113 序列 / 2015-01-28 ~ 2015-12-27**。

API-08 下载标准 CSV 模板

项	值
方法 / 路径	`GET /api/v1/datasets/template`
鉴权	否
用途	降低首次使用的试错成本（`FR-UPL-08`）
响应类型	`text/csv`（`Content-Disposition: attachment`）

模板内容（表头 + 2 行示例，`FR-UPL-08` 明确要求）

```
sku_id,warehouse_id,date,sales
item_0001,上海仓,2015-01-01,120
item_0001,上海仓,2015-01-02,135
```

`API-08-1`：文件带 UTF-8 BOM，保证 Excel 双击打开不乱码（`NFR-COMP-05`）。模板中的示例行必须是**合法数据**，用户直接提交也应通过校验——这是「模板可用性」的最低标准。

API-09 获取数据集详情 NEW

项	值
方法 / 路径	`GET /api/v1/datasets/{dataset_id}`
鉴权	是
用途	参数配置页（`UX-P06`）的数据概要 + 分级预判
耗时预算	≤ 1 s

路径参数：`dataset_id`（integer）

响应体

字段	类型	说明
`dataset`	object	`DatasetSummary`
`level_preview`	object	分级预判 `{L1, L2, L3, total}`
`estimated_ms`	integer	耗时预估（前端用于提示「预计约 8 秒」）
`over_budget_risk`	boolean	是否逼近 60 秒预算（序列数 > 800 时置 true）

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "dataset": { "id": 7, "row_count": 14305, "sku_count": 40,
                 "warehouse_count": 5, "valid_series_count": 113,
                 "date_start": "2015-01-28", "date_end": "2015-12-27" },
    "level_preview": { "L1": 81, "L2": 32, "L3": 0, "total": 113 },
    "estimated_ms": 7900,
    "over_budget_risk": false
  }
}
```

📌 `API-09-1`：`level_preview` 由 `classify.classify_series` + `count_levels` 计算，是**参数页「分级预判」右栏的数据源**（原型的 `params.html` 展示了该字段，且真实值为 L1 81 / L2 32 / L3 0）。

⚠ `API-09-2`：该接口会执行一次完整的数据读取与分级，对于 50 万行数据约需 2 秒。参数页在数据集加载后调用一次即可，**不应在每次参数变更时重复调用**（`horizon` 不影响分级结果）。

API-10 数据预览

项	值
方法 / 路径	`GET /api/v1/datasets/{dataset_id}/preview`
鉴权	是
用途	让用户确认「系统读到的和我想的一致」（`FR-UPL-07`）

查询参数

参数	类型	默认	约束	说明
`limit`	integer	20	1–100	返回行数
`keyword`	string	——	——	按商品 / 仓库模糊搜索（对应原型 `upload.html` 的搜索框）

响应体：`{columns: string[], rows: object[], total: integer, shown: integer}`

> `API-10-1`：表头与数据类型须与源文件一致（`FR-UPL-07` 验收要点）。`total` 是该数据集的总行数而非本次返回数，前端用于展示「共 14,305 行，显示前 20 行」。
>
> `API-10-2`：`keyword` 参数支撑原型的搜索框功能（`upload.html` 第 271 行 `placeholder="搜索商品 / 仓库"`）。若不实现该参数，原型的交互能力会退化。

API-11 下载错误明细 🆕

项	值
方法 / 路径	`GET /api/v1/datasets/{dataset_id}/errors`
鉴权	是
用途	把不合格行导出为 CSV，便于用户在原表修正后重传（`FR-UPL-06`）
响应类型	`text/csv`（附 `Content-Disposition`）

响应内容：仅含不合格行，**保留原始列**（`FR-UPL-06` 验收要点），并追加一列 `error_reason` 说明原因。

```
sku_id,warehouse_id,date,sales,error_reason
item_0033,成都仓,2015/13/45,88,日期无法解析
item_0041,武汉仓,2015-06-02,-5,销量为负值
```

`API-11-1`：该接口对应 `FR-UPL-06`，开发方案 §6.2.3 遗漏。实现前需保存校验失败的明细（可存于 `operation_logs.detail` 或独立临时文件），否则上传失败后无法再次导出。

🔒 `API-11-2`：**临时错误明细的存储位置与 TTL（`JT-11`，已决议）**。位置 `data/exports/tmp/dataset_{dataset_id}_{upload_id}.csv`，**TTL 1 小时**。该目录与正式导出产物 `data/exports/` **分离**，由独立清理任务处理（见 §2.8 / §3.8）。`upload_id` 为本次上传请求的 UUID，保证同一数据集多次上传的明细不互相覆盖。

`API-11-3`：错误明细**必须落盘**而不是仅存内存/`operation_logs.detail`。原因：CSV 大小无上界（最坏情况 500,000 行全部非法，约数十 MB），塞进数据库会污染审计表，且 `FR-UPL-06` 要求「保留原始列 + 追加 `error_reason` 列」，本质是一个文件导出操作。

5.6 预测任务接口（API-12 ~ API-16）

API-12 创建预测任务

项	值
方法 / 路径	`POST /api/v1/predictions`
鉴权	是
用途	创建并**同步执行**预测任务（`FR-MOD-07~13`）
耗时预算	🔒 **≤ 60 s**（`NFR-PERF-01`）；实测约 7.9 s

📌 **同步语义**：本接口在 60 秒内必然返回。任务状态从 `created` 直接进入 `running` 再进入终态，**不存在 `queued`**（取消在线训练后不再需要排队）。前端可直接等待响应，无需轮询；`API-13` 保留轮询能力用于将来任务变重时平滑演进。

请求体

字段	类型	必填	约束	说明
`dataset_id`	integer	是	> 0，且属于当前用户	输入数据集
`granularity`	string	否	固定 `warehouse_sku`	📌 主粒度，仅作显式声明。传其他值返回 `40012`
`horizon`	integer	是	`7` / `14` / `30`	预测步长
`use_holiday_feature`	integer	否	0 / 1，默认 1	节假日特征开关
`task_name`	string	否	≤ 64	为空时后端生成

```
{
  "dataset_id": 7,
  "granularity": "warehouse_sku",
  "horizon": 14,
  "use_holiday_feature": 1,
  "task_name": "10 月首周预测"
}
```

响应体

字段	类型	说明
`task_id`	integer	任务 ID
`status`	string	`created` / `running`。同步执行模式下成功时直接返回任务结果可查状态
`model_version`	string	实际使用的模型版本，如 `lstm_v1`（`FR-MOD-07` 要求记明）
`series_count`	integer	待预测序列数
`estimated_ms`	integer	实测耗时，前端用于进度条刻度

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "task_id": 42,
    "status": "running",
    "model_version": "lstm_v1",
    "series_count": 113,
    "estimated_ms": 7873
  }
}
```

错误码

码	触发条件	处理建议
`40004`	序列数 > 1000	`E_TOO_MANY_SERIES`，提示筛选商品
`40012`	`horizon` 非 7/14/30，或 `granularity` 异常	前端应已限制，出现即前端 bug
`40402`	数据集不存在	——
`40301`	数据集不属于当前用户	`NFR-SEC-05` 要求返回 403 而非 404
`40404`	无启用模型版本	提示执行 `seed.sql` 或联系管理员

📌 API-12-1：**序列数预检在前，落库在后。**若序列数超限，任务**不创建**（不产生 tasks 记录），直接返回 40004。这避免了数据库里堆积一批注定失败的任务记录。

🔒 API-12-2：**超预算走软失败。**若处理超过 60 秒，返回 code: 0 但任务状态为 failed、error_code = E_TIMEOUT，并在 data 中给出已完成阶段与建议（NFR-REL-02）。**不得返回 HTTP 502/504。**

API-13 查询任务状态

项	值
方法 / 路径	GET /api/v1/predictions/{task_id}/status
鉴权	是
用途	执行进度页（UX-P07）的五阶段进度、设备、耗时
建议轮询间隔	同步模式下令前端**不轮询**，直接跳结果页；备用轮询 2 s

响应体

字段	类型	说明
task_id	integer	
status	string	created / running / succeeded / failed
stage	string	当前阶段：validate / preprocess / features / inference / metrics
stage_index	integer	1–5
stage_total	integer	固定 5
progress_pct	number	0–100
device	string	cuda / cpu
device_fallback	integer	是否发生 CUDA→CPU 回退（FR-MOD-10）

字段	类型	说明
`elapsed_ms`	integer	已耗时
`budget_ms`	integer	预算，60000
`series_count`	integer	序列数
`l1_count` / `l2_count` / `l3_count`	integer	分级计数（前四阶段可能为 null）
`error_code`	string	失败码
`error_msg`	string	失败详情
`completed_stage`	string	软失败时已完成阶段


```
{
  "code": 0,
  "message": "ok",
  "data": {
    "task_id": 42, "status": "running", "stage": "inference", "stage_index": 4,
    "stage_total": 5, "progress_pct": 72.0,
    "device": "cuda", "device_fallback": 0,
    "elapsed_ms": 1460, "budget_ms": 60000, "series_count": 113,
    "l1_count": 81, "l2_count": 32, "l3_count": 0,
    "error_code": null, "error_msg": null, "completed_stage": null
  }
}
```


📌 `API-13-1`：**五阶段与实测耗时对齐**（`outputs/eval_report.json`）：`preprocess` 675 ms → `features` 763 ms → `inference` **21 ms** → `metrics` 473 ms。`inference` 仅 21 ms 意味着进度条在该阶段会「瞬间跳过」——这不是 bug。前端若想让动效平滑，应基于 `estimated_ms` 做时间轴插值（原型 `running.html` 采用此方式），而非等真实阶段回调。

⚠ `UX-GOV-07`：原型执行页动效总时长 34.3 s，而真实耗时 7.9 s，二者口径不一。**实现时统一为真实耗时**：动效按 `estimated_ms` 驱动，或明示为「示意图」。

API-14 获取预测结果 ★

项	值
方法 / 路径	`GET /api/v1/predictions/{task_id}`
鉴权	是
用途	**结果页（UX-P08）的全部数据**：结论、指标、覆盖率、曲线、分布
耗时预算	≤ 1 s（曲线数据来自库，不含计算）

📌 这是核心交付页的数据源。响应字段与算法产出 `InferenceResult` 几乎一一对应（见 §4.7.2 三向映射表），**接口层不做重新计算。**

查询参数

参数	类型	默认	说明
`with_series`	boolean	true	是否内联曲线数据。`false` 时仅返回指标与覆盖率（用于历史列表预览）
`series_limit`	integer	50	内联曲线最大条数。超限时按 `mape` 降序取前 N 条并置 `truncated: true`
`level`	string	——	按分级过滤曲线（`L1` / `L2` / `L3`）

响应体结构

字段	类型	说明
`task`	object	任务概要（`TaskBrief`）
`dataset`	object	数据集概要（`DatasetBrief`）
`metrics`	object	`MetricBundle`，加权指标 + 基线 + 达标占比 + 分组 MAPE
`coverage`	object	`Coverage`，L1/L2/L3 计数与占比
`distribution`	object	逐序列 MAPE 分布（中位/P75/P90/

字段	类型	说明
		最大/超标数/分箱）
`timing_ms`	object	分阶段耗时
`is_over_threshold`	boolean	加权 MAPE 是否超 20%，驱动不可折叠警告（`FR-VIS-06`）
`eval_note`	string	评估前提文案（`FR-MOD-13`），必须原样展示
`series`	array	`SeriesCurve[]` 曲线数据
`detail_total`	integer	明细总行数（`API-15` 分页用）

响应示例（数值取自真实评测 `outputs/eval_report.json`，horizon=14）


```
{
  "code": 0,
  "message": "ok",
  "data": {
    "task": {
      "id": 42, "task_name": "10 月首周预测", "status": "succeeded",
      "horizon": 14, "granularity": "warehouse_sku", "model_version": "lstm_v1",
      "device": "cuda", "device_fallback": 0,
      "series_total": 113, "duration_ms": 7873,
      "created_at": "2026-10-09 10:20:00", "finished_at": "2026-10-09 10:20:08"
    },
    "dataset": {
      "id": 7, "file_name": "demo_sales.csv", "row_count": 14305,
      "sku_count": 40, "warehouse_count": 5,
      "date_start": "2015-01-28", "date_end": "2015-12-27"
    },
    "metrics": {
      "weighted_mape": 14.26, "weighted_mae": 1.32, "weighted_rmse": 3.37,
      "baseline_mape": null, "baseline_mae": 1.8, "baseline_rmse": 4.65,
      "mae_improve_pct": 26.42, "rmse_improve_pct": 27.52,
      "series_evaluated": 81, "series_passed": 68, "series_pass_rate": 83.95,
      "l1_mape": 14.26, "l2_mape": null
    },
    "coverage": {
      "L1": 81, "L2": 32, "L3": 0, "total": 113,
      "L1_pct": 71.68, "L2_pct": 28.32, "L3_pct": 0.0,
      "predictable": 113, "predictable_pct": 100.0
    },
    "distribution": {
      "median": 14.21, "p75": 17.7, "p90": 25.61, "max": 67.88,
      "over_count": 13, "threshold": 20.0,
      "buckets": [3, 7, 35, 23, 4, 6, 1, 0, 0, 0, 0, 2]
    },
    "timing_ms": {
      "preprocess_ms": 675, "features_ms": 763,
      "inference_ms": 21, "metrics_ms": 473, "total_ms": 7873
    },
    "is_over_threshold": false,
```

```
    "eval_note": "本结果使用预训练模型推理，未针对本次输入数据微调。模型基于历史数据训练，对突发外部事件（政策、竞品、疫情）无预测能力。",
    "series": [
      {
        "key": "item_0000-上海仓",
        "sku_id": "item_0000",
        "warehouse_id": "上海仓",
        "granularity_level": "L1",
        "mape": 8.6371,
        "passed": true,
        "forecast_dates": ["2015-05-04", "2015-05-05", "... (共 14 天) "],
        "predicted": [10.93, 11.44, 11.39, 11.62, 11.56, 14.75, 14.38, 11.48, 11.75, 11.76, 11.68,
11.35, 13.39, 13.43],
        "actual": [10.0, 13.0, 9.0, 12.0, 12.0, 16.0, 14.0, 11.0, 10.0, 11.0, 11.0, 13.0, 15.0, 13.0],
        "baseline": [11.29, 11.29, 11.29, 11.29, 11.29, 11.29, 11.29, 11.29, 11.29, 11.29, 11.29,
11.29, 11.29, 11.29],
        "history_dates": ["2015-04-20", "... (共 14 天，用于折线图左侧实线段) "],
        "history_sales": [9.0, 7.0, 7.0, 8.0, 8.0, 15.0, 12.0, 9.0, 11.0, 9.0, 11.0, 9.0, 16.0, 14.0]
      }
    ],
    "detail_total": 1582
  }
}
```


前端渲染映射（UX-P08 → 响应字段）

结果页区块	数据来源	说明
结论判定条	metrics + is_over_threshold	四条硬指标判定，达标/超标
指标卡×4	metrics.weighted_mape / weighted_mae / weighted_rmse / mae_improve_pct	与基线对比展示
覆盖率面板	coverage	🔒 三项之和 = total（UX-AC-06）
折线对比图	series[].history_* + forecast_dates + predicted + actual	历史实线 + 预测虚线 + 真值点
分布直方图	distribution.buckets	FR-VIS-08，分箱 ≤ 20
明细表	API-15 分页接口	排序与分页
超标警告	is_over_threshold + metrics.weighted_mape	🔒 不可折叠（FR-VIS-06）
适用性说明	eval_note	🔒 固定位置展示（FR-SYS-07）

🔒 API-14-1：**coverage.L1 + L2 + L3 必须恒等于 coverage.total**。这是 UX-AC-06 的验收项，也是「覆盖率披露」可信度的基础。若不等，说明分级或统计环节存在缺陷。

🔒 API-14-2：**l2_mape 允许为 null**。实测 L2 在测试段缺乏可靠真值，MAPE 计算结果为 nan。序列化时必须转为 null 而非 NaN —— 后者不是合法 JSON，会导致前端解析失败（JSON.parse 抛错）。这是极易踩的坑：Python 的 float('nan') 经 FastAPI 序列化会产出 NaN 字面量。

📌 API-14-3：series 默认限制 50 条。113 序列全量内联约 200 KB，尚可接受；但 1,000 序列 × 30 天会达到数 MB。series_limit 与 truncated 标记的组合，让前端可以「先画前 50 条趋势，用户点选后再取具体序列」。

API-15 结果明细分页 NEW

项	值
方法 / 路径	GET /api/v1/predictions/{task_id}/detail
鉴权	是
用途	结果页明细表：排序 + 分页 + 过滤（FR-VIS-03）
耗时预算	≤ 1 s

查询参数

参数	类型	默认	说明
page	integer	1	页码
size	integer	20	每页条数，上限 100
sort_by	string	sku_id	排序列：sku_id / warehouse_id / forecast_date / predicted_value / error_pct

参数	类型	默认	说明
`order`	string	`asc`	`asc` / `desc`
`level`	string	——	按分级过滤：`L1` / `L2` / `L3`
`keyword`	string	——	商品 / 仓库模糊搜索

响应体（分页结构，`items` 为 `PredictionRow[]`）

```
{
  "code": 0, "message": "ok",
  "data": {
    "items": [
      {
        "sku_id": "item_0000", "warehouse_id": "上海仓",
        "forecast_date": "2015-05-04",
        "predicted_value": 10.93, "actual_value": 10.0, "baseline_value": 11.29,
        "granularity_level": "L1", "error": 0.93, "error_pct": 9.3
      }
    ],
    "page": 1, "size": 20, "total": 1582, "pages": 80
  }
}
```

🔒 `API-15-1`：**L3 序列的 `predicted_value` 应如何处理？**（`EP-9` 已裁决，V1.0 内勘误）

原方案写的是「L3 记录**不写入** `predictions` 表，仅在 `coverage` 中体现计数；前端明细表在 L3 序列上无数据行」。**该方案与 PRD 两条 P0 需求相冲突，已废弃**：

冲突点	PRD 原文	若 L3 不落行
`FR-MOD-08`	「L3 输出**历史均值**并标记不可预测」	无处输出历史均值
`FR-VIS-04`（P0）	「预测表格每条记录标注 `granularity_level`」，验收「**三类标签均可出现**且颜色区分」	明细表的 L3 标签、`level=L3` 筛选永远为空

按本项目的仲裁规则（**功能范围以 PRD 为准，技术口径以开发方案为准**），"L3 在明细表里是否可见"属功能范围，应以 PRD 为准。故最终口径为：

L3 序列写入 predictions，但只写 1 行占位：granularity_level='L3'，forecast_date 取窗口首日，predicted_value = 该序列的**历史均值**（本仓 → 商品跨仓 → 全局，取值优先级见 algo/fallback.py::l3_forecast），actual_value / baseline_value / 上下界均为 NULL。

🔒 判据（取代原「不落行」）：

① L3 **不是逐日预测** —— 行数恒为 1，**不得等于** horizon；

② granularity_level='L3' 必须写出，使 FR-VIS-04 的三类标签可达；

③ 前端明细表对该行渲染 「–」（UX-AC-10：不展示数值，只表示"该序列不可预测，此行仅承载降级标记"）；

④ 不参与任何指标（series_evaluated 只数 L1），不计入主指标的分子分母；

⑤ coverage 的 L3 计数来自 tasks.l3_count（序列数），**不是** prediction_dao.count_by_level（明细行数）—— 两者相差一个 horizon 倍，不可混用。

⚠ 「L3 落行」的**唯一**理由是把「不可预测」这件事变得**可查询、可展示、可导出**；它承载的是降级标记，不是预测值。任何消费方（含 API-16 导出的 CSV）都必须先看 granularity_level 再使用 predicted_value。

⚠ API-15-2：当前实测数据集 L3 = 0，该分支在真实数据上不可见。测试时必须**构造稀疏序列**触发 L3，否则该逻辑从未被验证（见 §9.4 的边界用例）。

API-16 导出预测结果

项	值
方法 / 路径	GET /api/v1/predictions/{task_id}/export
鉴权	是
用途	导出 CSV / Excel（FR-VIS-05）
响应类型	text/csv 或 application/vnd.openxmlformats-officedocument.spreadsheetml.sheet

查询参数

参数	类型	默认	取值
format	string	csv	csv / xlsx

导出列顺序（🔒 固定，FR-VIS-05 验收要点）

```
sku_id,warehouse_id,date,forecast_sales,granularity_level,actual_sales,error
item_0000,上海仓,2015-05-04,10.93,L1,10.0,0.93
item_0000,上海仓,2015-05-05,11.44,L1,13.0,1.56
```

列名	来源	说明
sku_id	predictions.sku_id	
warehouse_id	predictions.warehouse_id	
date	predictions.forecast_date	注意导出列名为 date 而非 forecast_date
forecast_sales	predictions.predicted_value	
granularity_level	predictions.granularity_level	L1 / L2
actual_sales	predictions.actual_value	无真值时为空，**不填 0**
error	abs(actual - predicted)	无真值时为空

🔒 API-16-1：**CSV 必须带 UTF-8 BOM**（NFR-COMP-05）。不带 BOM 时 Excel 双击打开中文会乱码，这是国内用户环境的必现问题。

API-16-2：导出文件同时写入 data/exports/ 并登记，供 7 天 TTL 清理（NFR-REL-05 / FR-SYS-08）。小于 5 MB 时直接流式返回不落盘。

API-16-3：Content-Disposition 文件名需 URL 编码，否则中文文件名在部分浏览器下乱码：filename*=UTF-8''%E9%A2%84%E6%B5%8B%E7%BB%93%E6%9E%9C_42_20261009.csv。

5.7 历史记录接口（API-17 ~ API-20）

API-17 历史任务列表

项	值
方法 / 路径	`GET /api/v1/history`
鉴权	是
用途	历史列表页（`UX-P09`），含筛选与分页（`FR-HIS-01`）
数据源	数据库视图 `v_task_summary`（已含数据集名与指标，避免多次拼接）

查询参数

参数	类型	默认	说明
`page`	integer	1	
`size`	integer	20	上限 100
`keyword`	string	——	按任务名 / 数据集名模糊搜索
`date_from`	string	——	起始日期 `YYYY-MM-DD`
`date_to`	string	——	结束日期
`status`	string	——	`succeeded` / `failed`

响应体：分页结构，`items` 为 `HistoryItem[]`。

```
{
  "code": 0, "message": "ok",
  "data": {
```

```
    "items": [
      {
        "task_id": 42, "task_name": "10 月首周预测", "status": "succeeded",
        "granularity": "warehouse_sku", "horizon": 14,
        "dataset_name": "demo_sales.csv", "series_total": 113,
        "duration_ms": 7873, "device": "cuda", "device_fallback": 0,
        "model_version": "lstm_v1",
        "weighted_mape": 14.26, "weighted_mae": 1.32, "weighted_rmse": 3.37,
        "mae_improve_pct": 26.42, "rmse_improve_pct": 27.52,
        "series_pass_rate": 83.95, "is_over_threshold": 0,
        "error_code": null,
        "created_at": "2026-10-09 10:20:00", "finished_at": "2026-10-09 10:20:08"
      }
    ],
    "page": 1, "size": 20, "total": 5, "pages": 1
  }
}
```

🔒 API-17-1：**必须过滤 `is_deleted = 0`**（视图已内置该条件）。若不过滤，软删除的任务会重新出现在列表中。

API-17-2：**按 `created_at DESC` 排序**（FR-HIS-01 验收要点）。索引 `ix_tasks_user_list` 已按此顺序建立。

API-17-3：失败任务（`status = failed`）也应在列表中展示，携带 `error_code`，前端渲染失败态并提供「原参数重试」入口（NFR-REL-01）。

API-18 跨任务曲线对比 🆕

项	值
方法 / 路径	`POST /api/v1/history/compare`
鉴权	是
用途	选择 2–4 个历史任务叠加对比（FR-HIS-04），承载 UX-P11
耗时预算	≤ 3 s

⚠ **路由注册顺序**：本接口必须注册在 `API-19 /history/{task_id}` 之前，否则 `compare` 会被当作 `task_id` 解析（`BE-39`）。

请求体

字段	类型	必填	约束	说明
`task_ids`	integer[]	是	长度 2–4，不重复	对比任务集合
`sku_ids`	string[]	否	≤ 50	限定商品，为空则取前 20 条序列
`warehouse_ids`	string[]	否	≤ 10	限定仓库

```
{ "task_ids": [42, 41, 39], "sku_ids": ["item_0000", "item_0033"] }
```

响应体

字段	类型	说明
`tasks`	array	每个任务的指标、覆盖率、曲线
`attribution`	array	差异归因说明（以第一个任务为基准）
`dataset_consistent`	boolean	各任务是否使用同一数据集。不一致时前端须显著提示（原型已实现该警告）

```
{
  "code": 0, "message": "ok",
  "data": {
    "tasks": [
      {
        "task_id": 42, "task_name": "10 月首周预测", "horizon": 14,
        "dataset_name": "demo_sales.csv", "created_at": "2026-10-09 10:20:00",
        "metrics": { "weighted_mape": 14.26, "mae_improve_pct": 26.42, "...": "..." },
        "coverage": { "L1": 81, "L2": 32, "L3": 0, "total": 113 },
        "series": [ { "key": "item_0000-上海仓", "predicted": ["..."] } ],
        "warnings": []
```

```
      }
    ],
    "attribution": [
      { "task_id": 41, "note": "步长由 14 天改为 7 天，加权 MAPE 下降 0.8 个百分点" }
    ],
    "dataset_consistent": true
  }
}
```

🔒 API-18-1：**必须校验所有 task_ids 都属于当前用户**。任一不属于则返回 40301，不得部分返回（NFR-SEC-02）。

⚠ API-18-2：**不同 horizon 的任务不可直接比较指标**。7 天步长天然比 14 天准确（预测越远误差越大）。实现上应：

① 在 warnings 中标注「步长不一致，指标不可直接比较」；

② 曲线只对比**时间重叠区间**；

③ attribution 中说明差异来源。

API-18-3：dataset_consistent = false 时前端必须显示警告（原型 compare.html 已实现「数据文件不一致警告」）。跨数据集对比虽然允许，但结论参考性下降。

API-19 历史任务详情

项	值
方法 / 路径	GET /api/v1/history/{task_id}
鉴权	是
用途	任务详情页（UX-P10）：参数快照 + 指标 + 曲线 + 阶段耗时 + 失败详情
响应体	**结构与 API-14 完全一致**（复用 PredictionResultResponse）

📌 复用同一响应结构的理由：FR-HIS-02 要求「回看当时的曲线、指标卡与覆盖率披露，详情内容与当时结果一致」。同一结构天然保证一致性，前端也可复用同一套渲染组件。

失败任务的响应差异

当 task.status = failed 时：

字段	值
task.status	failed
task.error_code	E_TIMEOUT / E_TOO_MANY_SERIES / E_INFER_FAIL
metrics	各字段为 null（未计算）
coverage	可能部分有效（若已在分级阶段失败则为 0）
series	空数组
eval_note	保留固定文案

> API-19-1：失败详情页（原型 history-detail.html 已实现）需同时展示 error_code、error_msg、completed_stage 与「原参数重试」按钮。重试即用原参数调用 API-12，**生成新任务**而非复用旧任务（DB-T03-02）。

API-20 删除历史任务（软删除）

项	值
方法 / 路径	DELETE /api/v1/history/{task_id}
鉴权	是
用途	软删除（FR-HIS-05）
请求体	无

响应体

```
{ "code": 0, "message": "ok", "data": null }
```

🔒 API-20-1：**软删除语义**——只置 `tasks.is_deleted = 1`，数据保留在库中（FR-HIS-05 验收要点：「删除后可通过数据库查回，满足可追溯」）。`predictions` / `metrics` 记录**不删除**。

⚠ API-20-2：**是否级联软删除关联数据？** 不需要。所有查询都以 `tasks.is_deleted = 0` 为入口过滤，关联表通过 `task_id` 关联，天然被隔离。若同时软删 `predictions`，反而会在「查回」时增加复杂度。

API-20-3：前端须有**二次确认弹窗**（FR-HIS-05）。原型采用「输入任务 ID 确认」的强化确认方式（UX-P09 的删除确认状态），实现时保留该交互。

5.8 系统接口（API-21）

API-21 健康检查

项	值
方法 / 路径	`GET /health`
鉴权	否
用途	部署验证与存活探测（FR-SYS-04）
注意	**挂根路径，不在 `/api/v1` 下**

响应体

字段	类型	说明
`status`	string	`ok` / `degraded`
`version`	string	应用版本
`database`	string	`ok` / `error`
`model_loaded`	boolean	模型是否已加载
`model_version`	string	启用模型版本
`device`	string	`cuda` / `cpu`
`uptime_seconds`	integer	运行时长

```
{
  "code": 0, "message": "ok",
  "data": {
    "status": "ok", "version": "1.0.0", "database": "ok",
    "model_loaded": true, "model_version": "lstm_v1",
    "device": "cuda", "uptime_seconds": 3612
  }
}
```

`API-21-1`：`status = degraded` 的判定条件：数据库异常 **或** 模型未加载。此时服务仍可响应，但预测接口会返回 `40404`。部署脚本据此判断是否就绪。

`API-21-2`：健康检查**不做鉴权**（否则监控系统无法调用），因此**不得在响应中泄露敏感信息**——只暴露版本号与状态，不暴露路径、配置、连接串（`NFR-SEC-09`）。

---

6. 前端设计与实现

6.1 应用结构与 Provider 组装

```tsx
// src/main.tsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { ConfigProvider, theme as antdTheme, App as AntdApp } from 'antd';
import zhCN from 'antd/locale/zh_CN';
import dayjs from 'dayjs';
import 'dayjs/locale/zh-cn';


import App from './App';
import { useThemeStore } from './store/useThemeStore';
import { buildAntdToken } from './theme/antd-token';
import './styles/tokens.css';   // 🔧 设计令牌必须在最前面加载
import './styles/global.css';


dayjs.locale('zh-cn');          // NFR-USE-05：界面语言为简体中文


function Root() {
  const isDark = useThemeStore((s) => s.isDark);
  return (
    <ConfigProvider
      locale={zhCN}
      theme={{
        algorithm: isDark ? antdTheme.darkAlgorithm : antdTheme.defaultAlgorithm,
        token: buildAntdToken(isDark),
      }}
    >
      <AntdApp>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </AntdApp>
    </ConfigProvider>
  );
}


ReactDOM.createRoot(document.getElementById('root')!).render(<Root />);
```

FE-07：`tokens.css` 必须在任何组件样式之前加载。若 AntD 样式先注入，`ConfigProvider` 的令牌覆盖会失效，出现「色板对了但组件还是默认蓝」的问题。

FE-08：使用 `<AntdApp>` 包裹以启用 `message` / `notification` / `Modal` 的静态方法上下文，否则在 React 18 StrictMode 下会报「static function can not consume context」告警。

6.2 状态管理设计（Zustand）

本项目的状态分为三类，**严格区分归属**，避免把所有数据都塞进全局 store：

类型	归属	示例
全局共享状态	Zustand store	登录用户、Token、主题、当前任务 ID
服务端数据	页面内 `useState` + 请求	任务列表、结果数据、数据集详情
表单临时状态	组件内 `useState`	参数配置页的 `horizon`、上传进度

📌 **不引入 React Query / SWR**：本项目共 14 个页面、31 条路径（33 个操作），且数据变更均由用户主动操作触发（无实时推送、无多标签页同步需求）。引入数据请求库会带来学习成本与额外依赖，收益有限。若后续增加「任务列表自动刷新」等需求，再评估引入。

```
// src/store/useAuthStore.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
import type { UserProfile } from '../types';
import { tokenStore } from '../api/client';


interface AuthState {
  user: UserProfile | null;
  setUser: (u: UserProfile | null) => void;
  logout: () => void;
}


export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      user: null,
      setUser: (user) => set({ user }),
      logout: () => {
        tokenStore.clear();
        set({ user: null });
      },
    }),
    {
      name: 'sp_auth',
      // 🔒 只持久化非敏感字段：Token 由 tokenStore 单独管理
      partialize: (s) => ({ user: s.user }),
    },
  ),
);
```

```
// src/store/useThemeStore.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
```

```
interface ThemeState {
  isDark: boolean;
  toggle: () => void;
}


export const useThemeStore = create<ThemeState>()(
  persist(
    (set, get) => ({
      isDark: true,                    // 📌 深色为默认 (UI/UX §3.2)
      toggle: () => set({ isDark: !get().isDark }),
    }),
    {
      name: 'sp_theme',                // 与原型的 localStorage 键名保持一致
      onRehydrateStorage: () => (state) => {
        // 同步到 <html class="dark">，供 CSS 令牌选择器使用
        document.documentElement.classList.toggle('dark', state?.isDark ?? true);
      },
    },
  ),
);
```

🔒 FE-09：**主题必须在首屏渲染前应用，否则会闪白。**实现方式：在 `index.html` 的 `<head>` 内联一段脚本，从 `localStorage` 读取主题并立刻给 `<html>` 加 `dark` 类（原型 `demo-data.js` 的 `initTheme()` 已采用此方式）。Zustand 的 `persist` 水合发生在 React 挂载后，仅靠它会导致首帧闪烁。

```
<script>
(function () {
var saved = localStorage.getItem('sp_theme');
var isDark = saved ? JSON.parse(saved).state.isDark : true;
document.documentElement.classList.toggle('dark', isDark);
})();
</script>
```

```
// src/store/useTaskStore.ts —— 承载主流程跨页数据
interface TaskState {
  datasetId: number | null;
  datasetSummary: DatasetSummary | null;
  currentTaskId: number | null;
  params: Partial<CreateTaskRequest>;
  setDataset: (id: number, summary: DatasetSummary) => void;
  setParams: (p: Partial<CreateTaskRequest>) => void;
  setTaskId: (id: number) => void;
  reset: () => void;
}
```

FE-10：useTaskStore 承担「主流程草稿」职责。上传页 → 参数页 → 执行页 → 结果页 之间传递 dataset_id 与参数。**路由跳转也携带参数**（/params/:datasetId），store 用于在页面刷新后恢复草稿（对应 NFR-REL-01 的「失败重试不用重走参数页」）。

6.3 接口调用与数据流

```
// 典型页面数据加载模式：Result.tsx
export default function Result() {
  const { taskId } = useParams<{ taskId: string }>();
  const [data, setData] = useState<PredictionResultResponse | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);


  useEffect(() => {
    if (!taskId) return;
    let alive = true;                       // FE-11：防竞态
    setLoading(true);
    predictApi
      .result(Number(taskId))
      .then((d) => alive && setData(d))
      .catch((e: ApiError) => alive && setError(e.message))
      .finally(() => alive && setLoading(false));
    return () => {
      alive = false;
    };
  }, [taskId]);


  if (loading) return <ResultSkeleton />;   // FE-12：骨架屏
  if (error) return <ErrorState message={error} />;
  if (!data) return <EmptyState />;
  return <ResultContent data={data} />;
}
```

FE-11：**必须处理竞态**。用户快速切换任务时，先发出的请求可能后返回并覆盖正确数据。用 alive 标志（或 AbortController）丢弃过期响应。

FE-12：加载态使用**骨架屏**而非全屏 spinner（UI/UX §5 的状态规范）。实测结果接口响应在 1 秒内，骨架屏能显著改善感知性能。

6.4 主题与令牌落地

6.4.1 令牌文件结构

```
/* src/styles/tokens.css —— 从 prototype/assets/app.css 原样搬入 */
:root {
  /* 品牌（浅色）：L 0.475，经 WCAG 反解确定，可承载白字于实色 4.5:1 */
  --primary: oklch(0.475 0.145 205);
  --primary-soft: oklch(0.475 0.145 205 / 0.12);

  /* on-color 令牌组：解决「实色填充承载白字」的对比度问题 */
  --on-primary: oklch(0.99 0 0);
  --on-success: oklch(0.99 0 0);
  --on-danger: oklch(0.99 0 0);

  /* 语义色（浅色取深值，使同一令牌同时满足「白字于实色」与「该色作正文」） */
  --success: oklch(0.51 0.13 150);
  --warning: oklch(0.54 0.13 75);
  --danger:  oklch(0.555 0.19 25);
  --info:    oklch(0.53 0.13 240);

  /* 文字层级 */
  --text-1: oklch(0.25 0.01 250);
  --text-2: oklch(0.42 0.01 250);
  --text-3: oklch(0.535 0.01 250);   /* 浅色 0.60 → 0.535，满足 4.5:1 */

  /* 描边职责分离 */
  --border: oklch(0.90 0.005 250);        /* 装饰性分隔，SC 1.4.11 豁免 */
  --border-field: oklch(0.645 0.01 250);  /* ✎ 功能性描边，须 ≥ 3:1 */

  /* 图表色板 */
  --chart-1: oklch(0.61 0.14 205);
  --chart-2: oklch(0.645 0.15 150);
  --chart-3: oklch(0.665 0.15 60);
  --chart-4: oklch(0.62 0.16 300);
  --chart-5: oklch(0.63 0.15 20);
}

html.dark {
  /* 深色整体反转：亮填充 + 深油墨 */
  --primary: oklch(0.72 0.145 205);
  --primary-soft: oklch(0.72 0.145 205 / 0.10);
  --on-primary: oklch(0.18 0.01 250);

  /* 🔒 必须显式写回，否则继承浅色深值，对比度失守 */
  --success: oklch(0.68 0.13 150);
  --warning: oklch(0.76 0.13 75);
  --danger:  oklch(0.64 0.19 25);
  --info:    oklch(0.68 0.13 240);
```

```
  --text-1: oklch(0.96 0.005 250);
  --text-2: oklch(0.80 0.008 250);
  --text-3: oklch(0.63 0.01 250);
  --border: oklch(0.32 0.008 250);
  --border-field: oklch(0.54 0.01 250);
}
```

🔒 FE-13：**深色块内必须显式写回所有语义色。**CSS 自定义属性会继承，若深色主题只覆盖部分令牌，未覆盖的会沿用浅色的深值，在深色背景上对比度不足——这是 UI/UX 文档实测发现的缺陷模式。

🔒 FE-14：**`--border` 与 `--border-field` 职责不可混用。**前者是装饰性分隔线（表格行线、卡片边框），后者是组件边界（输入框、按钮、选择卡）。组件边界若误用 `--border`，会因对比度不足导致 `UX-AC-01` 失败。

6.4.2 无障碍硬性要求（承接 UI/UX P1 项）

UI/UX §12.5 列出了若干已知未修项，实现时**必须一并解决**：

编号	要求	实现方式
UX-GOV-04	表单控件必须有关联 label	每个 `<Input>` 配 `aria-label` 或用 `<Form.Item label>`（AntD 会生成 `<label for>`）
UX-GOV-04	装饰性 SVG 标 `aria-hidden`	AntD 图标默认已处理；自定义 SVG 需手动加
UX-GOV-05	支持 `prefers-reduced-motion`	见下方 CSS
UX-GOV-06	弹窗支持 Esc 关闭 + 焦点捕获	AntD `Modal` 默认支持，但自定义确认层需手动实现
UX-GOV-08	补 320px 断点	Tailwind `sm:` 断点设为 640px，需额外加 `xs` 断点或使用 `min-w-0` + `truncate`
UX-GOV-10	图表提供数据表替代	ECharts 图表旁提供「查看数据表」折叠区

```
/* src/styles/a11y.css */
@media (prefers-reduced-motion: reduce) {
```

```
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}


/* 触控靶点 (UX-AC-12)  */
@media (pointer: coarse) {
  .btn      { min-height: 44px; padding: 0.625rem 1rem; }
  .btn-sm   { min-height: 40px; }
  .ant-input, .ant-select { min-height: 44px; }
  .side-item { min-height: 44px; }
}
```

FE-15：**结果页的执行动效必须尊重 `prefers-reduced-motion`**。原型 `running.html` 有五阶段进度动效，前庭敏感用户可能不适。降级方式是直接展示静态阶段列表。

6.5 图表实现（ECharts）

📌 按需引入以控制体积（FE-05）：

```
// src/components/charts/echarts-setup.ts
import * as echarts from 'echarts/core';
import { LineChart, BarChart } from 'echarts/charts';
import {
  GridComponent, TooltipComponent, LegendComponent,
  DataZoomComponent, MarkLineComponent, MarkAreaComponent,
} from 'echarts/components';
import { CanvasRenderer } from 'echarts/renderers';


echarts.use([
  LineChart, BarChart,
  GridComponent, TooltipComponent, LegendComponent,
  DataZoomComponent, MarkLineComponent, MarkAreaComponent,
  CanvasRenderer,
]);


export default echarts;
```

6.5.1 核心曲线图（FR-VIS-01）

结果页最关键的可视化：历史实线 + 预测虚线 + 真值点 + 基线与预测起点的标注。

```
export function buildCompareOption(curve: SeriesCurve, theme: EChartTheme) {
  const allDates = [...curve.history_dates, ...curve.forecast_dates];
  // 历史段：最后一点与预测段首点重合，保证视觉连续
  const histSeries = [
    ...curve.history_sales,
    null,                       // 占位，使折线断开
  ];
  const predSeries = [
    ...new Array(curve.history_dates.length - 1).fill(null),
    curve.history_sales.at(-1), // 从历史最后一点起笔
    ...curve.predicted,
  ];


  return {
    color: [theme.chart1, theme.chart3, theme.chart2],
    xAxis: { type: 'category', data: allDates, boundaryGap: false },
    yAxis: { type: 'value', name: '销量' },
    tooltip: { trigger: 'axis' },
    legend: { data: ['历史销量', '预测值', '移动平均基线', '实际值'] },
    series: [
      {
        name: '历史销量', type: 'line', data: histSeries,
        showSymbol: false, lineStyle: { width: 2, type: 'solid' },
      },
      {
        name: '预测值', type: 'line', data: predSeries,
        showSymbol: false, lineStyle: { width: 2, type: 'dashed' },
        // 🔒 FR-VIS-01 要求视觉可区分：实线 vs 虚线
      },
      {
        name: '移动平均基线', type: 'line',
        data: [...new Array(allDates.length - curve.predicted.length).fill(null),
               ...curve.baseline],
        showSymbol: false, lineStyle: { width: 1.5, type: 'dotted', opacity: 0.6 },
      },
      {
        name: '实际值', type: 'scatter',
        data: curve.actual
          ? [...new Array(allDates.length - curve.actual.length).fill(null), ...curve.actual]
          : [],
        symbolSize: 5,
      },
    ],
    // 预测起点标注（MarkLine）
    markLineTarget: curve.history_dates.length - 1,
  };
}
```

🔒 FE-16：**历史与预测折线必须在时间轴上连续**（FR-VIS-01 验收要点）。实现要点：两条 series 共用同一 xAxis.data（完整日期数组），历史 series 在预测段补 null，预测 series 在历史段补 null（除最后一个重合点）。若各自使用独立的 x 轴数据，两条线会在图上「错位」。

🔒 FE-17：**图表的颜色必须来自 --chart-* 令牌并在主题切换时重绘**（UX-GOV-02a）。实现方式：在 useThemeStore 的 isDark 变化时调用 chartInstance.setOption(buildCompareOption(curve, buildEChartTheme()))。

FE-18：UX-GOV-10 要求图表提供无障碍替代。实现：图表下方提供「查看数据表」Collapse，展开后渲染 API-15 的明细行。

6.5.2 其他图表

图表	位置	类型	数据源
覆盖度判定条	结果页	横向堆叠条	coverage
逐序列 MAPE 直方图	结果页	柱状图	distribution.buckets（分箱 ≤ 20，FR-VIS-08）
多序列叠加	结果页	多折线 + 图例筛选	series[]（FR-VIS-09 要求商品数 > 100 时可交互）
跨任务对比	UX-P11	多折线（每任务一色）	API-18
五阶段耗时	执行页 / 详情页	横向条形	timing_ms

FE-19：多序列叠加时**必须限流**。FR-VIS-09 要求「商品数 > 100 时可正常交互」，若一次性渲染 1,000 条折线会卡死浏览器。实现：默认渲染前 20 条，提供搜索多选组件让用户主动添加，最多同时渲染 50 条。

6.6 表单与校验

6.6.1 前端预检（减少无效请求）

```
// src/utils/validate.ts — 与后端同源常量
```

```
import { LIMITS, REQUIRED_COLUMNS } from '../types';


export interface PrecheckResult {
  ok: boolean;
  code?: number;
  message?: string;
  suggestion?: string;
}


export function precheckFile(file: File): PrecheckResult {
  if (!file.name.toLowerCase().endsWith('.csv')) {
    return {
      ok: false, code: 40009,
      message: `仅支持 .csv 格式，当前文件为 ${file.name.split('.').pop()}`,
      suggestion: '请在 Excel 中「另存为 → CSV UTF-8（逗号分隔）」后重新上传',
    };
  }
  if (file.size > LIMITS.max_file_size_mb * 1024 * 1024) {
    return {
      ok: false, code: 40007,
      message: `文件大小 ${(file.size / 1024 / 1024).toFixed(1)}MB，超过 ${LIMITS.max_file_size_mb}MB
上限`,
      suggestion: '请按商品或时间范围拆分后分次上传，或先筛选出需要预测的商品',
    };
  }
  return { ok: true };
}
```

FE-20：**前端预检是为了即时反馈，不是为了替代后端校验。**开发方案 §4.2 明确「前端不做数据校验的最终判定」。前端只做**不消耗带宽**的检查（扩展名、大小），表头与内容校验一律由后端完成。

🔒 FE-21：**前端错误提示同样必须含「错在哪 + 怎么改」**（NFR-USE-03）。上例中 `suggestion` 字段不可省略。

6.6.2 参数配置页表单（UX-P06）

```
<Form layout="vertical" initialValues={{ horizon: 14, use_holiday_feature: 1 }}>
  {/* 📌 粒度固定为仓库×商品，只读展示（FR-MOD 要求；不提供在线重训开关） */}
  <Form.Item label="预测粒度">
    <Input value="仓库 × 商品" disabled />
    <Typography.Text type="secondary">
      本系统按「仓库 × 商品」粒度预测，这是数据集的固有维度
    </Typography.Text>
  </Form.Item>


  <Form.Item label="预测步长" name="horizon" rules={[{ required: true }]}>
    <Radio.Group>
      {LIMITS.allowed_horizons.map((h) => (
```

```
        <Radio.Button key={h} value={h}>{h} 天</Radio.Button>
      ))}
    </Radio.Group>
  </Form.Item>

  <Form.Item label="节假日特征" name="use_holiday_feature">
    <Switch checkedChildren="启用" unCheckedChildren="关闭" />
  </Form.Item>
</Form>
```

🔒 FE-22：**参数默认值必须可用**（NFR-USE-02：用户不做任何修改即可提交并得到有效结果）。默认 horizon=14、use_holiday_feature=1。

📌 FE-23：**不提供「在线重新训练」开关**。开发方案 §1.4 与 FR-MOD-15 明确在线训练不在本期范围。界面上不得出现相关控件或暗示。

6.7 表格与分页

要求	实现
默认每页 20 行	AntD Table 的 pagination.pageSize = 20（FR-VIS-03）
支持任意列排序	服务端排序：onChange 中取 sorter.field / sorter.order 映射到 sort_by / order
分级标签	granularity_level 渲染为带色 Tag：L1 主色、L2 警示色、L3 灰（FR-VIS-04）
置信度标注	依 GRANULARITY_META 渲染文本（高 / 中 / 低）
L3 处理	L3 在明细表中**有 1 行占位**（granularity_level='L3'，predicted_value 为历史均值），该行**渲染为「—」**，数值不展示；coverage 面板同时说明「N 条序列不可预测」（UX-AC-10 / FR-VIS-04 / API-15-1）

FE-24：**服务端排序而非前端排序**。FR-VIS-03 要求「支持按任意列排序、分页」，若只对当前页排序，用户切换页码后顺序会跳变，属功能性缺陷。

6.8 错误处理与用户提示

```
// src/utils/error-handler.ts —— 按错误码给出可执行建议
export const ERROR_SUGGESTIONS: Record<number, string> = {
  40001: '请对照标准模板检查表头，确认包含 sku_id、warehouse_id、date、sales 四列',
  40002: '请将日期统一为 YYYY-MM-DD 格式后重新上传',
  40003: '请修正销量负值（通常源于退货未冲销），系统不会自动取绝对值',
  40004: '请筛选商品，将序列数控制在 1000 以内',
  40007: '请拆分文件或先筛选商品后分次上传',
  40102: '登录已过期，请重新登录（当前参数已保留）',
  40301: '该资源不属于当前账号，无法访问',
  40404: '模型尚未就绪，请稍后重试或联系管理员',
  50001: '服务处理异常，请重试；若持续失败请将任务 ID 提供给管理员',
};

export function renderError(e: ApiError) {
  const tip = ERROR_SUGGESTIONS[e.code];
  return { title: e.message, suggestion: tip ?? '请稍后重试' };
}
```

🔒 FE-25：**禁止只展示错误码**（NFR-USE-03）。所有错误提示都走 renderError，保证「错在哪 + 怎么改」两段式结构。

FE-26：401 的**草稿保留**：api-client.ts 在收到 401 时清除 Token 并触发跳登录。跳转前需把当前表单参数写入 sessionStorage，登录后恢复（对应开发方案 §2.2 B12「跳登录并保留草稿参数」）。

6.9 页面状态规范

UI/UX §5 定义了 5 类状态，前端实现对照：

状态	表现	适用页面
加载态	骨架屏（优先）/ 局部 spinner	所有数据页
空态	插画 + 说明 + 主行动按钮	历史列表首次使用、结果页无数据
错误态	图标 + 原因 + 建议 + 重试按钮	所有数据页
超标警告	不可折叠的警示条	结果页（FR-VIS-06）

状态	表现	适用页面
不可预测态	灰色标注「不可预测」，无数值	覆盖率面板、明细表（UX-AC-10）

FE-27：**MAPE 超标警告不可折叠、不可关闭**（FR-VIS-06 / UX-AC-09）。实现上不得使用可关闭的 Alert（closable 属性必须为 false），也不得放在 Collapse 内。

6.10 性能优化

手段	目标	实现
ECharts 按需引入	首屏 ≤ 3 s（NFR-PERF-06）	见 §6.5 的 echarts/core 用法
路由懒加载	首屏体积	React.lazy + Suspense 拆分 14 个页面
曲线数据限流	交互流畅	默认 20 条，最多 50 条（FE-19）
明细表服务端分页	避免大响应	API-15
长列表虚拟滚动	历史列表	AntD Table 的 virtual 属性（记录数 > 200 时启用）
请求去重	避免重复请求	数据集详情在页面挂载时只请求一次（API-09-2）

---

7. 前后端协作与联调流程

7.1 契约优先原则（JT-05）

本项目的协作原则是 **「契约先行，双向对齐」**：

```
① 本文档 §5 定义接口契约
        ↓
② 后端落地 app/schemas/*.py（Pydantic 真源）→ 生成 code/backend/schemas.py
        ↓
③ 前端落地 code/frontend/types.ts（TS）+ api-client.ts
        ↓
④ 跑三个校验脚本，确认三处字段完全一致
        ↓
⑤ 并行开发（前端用 Mock，后端用真实数据）
        ↓
⑥ 联调
```

契约变更流程（`JT-06`）：任何接口字段调整必须按固定顺序执行，任一步遗漏即视为技术债：

顺序	动作	文件
1	修改后端模型	`app/schemas/*.py`（改完必须跑 `python scripts/gen_backend_schemas.py` 重新生成 `code/backend/schemas.py`）
2	修改前端类型	`code/frontend/types.ts`
3	修改调用封装	`code/frontend/api-client.ts`（若涉及路径/参数）
4	更新本文档	`docs/...开发技术文档_V1.0.md` §5
5	跑校验	`python scripts/verify_schemas.py` + `node scripts/verify_types.mjs`

7.2 开发顺序与依赖（📌 关键路径）

开发方案 §7.1 的原则是「算法链路优先于界面美化」。结合本项目已完成的算法层，实际开发顺序为：

阶段	前后端分工	完成标志	依赖
S0 基建	后端：`main.py` + 数据库初始化 + 健康检查；前端：Vite 工程 + 令牌 + 布局组	`GET /health` 返回 `ok`；前端能渲染空布局	——

阶段	前后端分工	完成标志	依赖
	件		
S1 认证	后端：`API-01~05`；前端：登录/注册页 + Token 管理	能注册、登录、刷新后保持登录态	S0
S2 上传	后端：`API-06~11`；前端：上传页 + 五类错误展示	能上传示例数据并通过校验；五类错误都能正确提示	S1
S3 预测 ★	后端：`API-12~16`（核心）；前端：参数页 + 执行页 + **结果页**	端到端跑通，结果页指标与 `eval_report.json` 一致	S2
S4 记录	后端：`API-17~20`；前端：历史列表 + 详情 + 对比	能回看历史、导出、对比	S3
S5 打磨	无障碍整改（`UX-GOV-04~08`）、空态/错误态补全、性能优化	三个校验脚本全绿 + 视觉走查通过	S4

🔒 `JT-07`：**S3 是整个项目的关键路径**，其余阶段都可与它并行或延后。S3 完成的定义是精确的：**结果页展示的 `weighted_mape` 与 `series_pass_rate` 必须与 `outputs/eval_report.json` 完全一致。**这是「算法与产品同源」这一约束的可验证落地。

7.3 Mock 策略（前后端并行开发）

后端未就绪时，前端使用 Mock 数据推进。**Mock 数据必须来自真实算法产出**，而非手编：

```
// src/mocks/result.ts
// 🔪 来源：outputs/eval_report.json + outputs/eval_series.json（scripts/eval_report.py --export-series 产出）
// 这些是真实评测数据，不是示例值。用它做 Mock 可保证前端渲染逻辑与真实数据形态一致。
export const MOCK_RESULT: PredictionResultResponse = {
  task: { id: 42, status: 'succeeded', horizon: 14, /* ... */ },
  metrics: {
    weighted_mape: 14.26, weighted_mae: 1.32, weighted_rmse: 3.37,
    baseline_mae: 1.8, baseline_rmse: 4.65,
    mae_improve_pct: 26.42, rmse_improve_pct: 27.52,
    series_evaluated: 81, series_passed: 68, series_pass_rate: 83.95,
    l1_mape: 14.26, l2_mape: null,
  },
  coverage: { L1: 81, L2: 32, L3: 0, total: 113, /* ... */ },
  // ...
};
```

Mock 开关：通过环境变量 `VITE_USE_MOCK=1` 启用，`api-client.ts` 在拦截器内短路返回 Mock 数据。**交付前必须关闭该开关**（列入 §9.6 验收清单）。

> 🔒 `JT-08`：**Mock 数据禁止手编。**项目历史上出现过「原型显示一套数字、算法产出另一套数字」的问题。Mock 数据必须来自 `eval_report.json`，并在联调后由真实接口替代。手工编造 14.26% 之外的任何数值都会引入新的不一致风险。

7.4 联调检查清单（JT-09）

按顺序逐项验证，每项都有可观察的通过标准：

#	检查项	通过标准
1	健康检查	`GET /health` 返回 `model_loaded: true`
2	注册 → 登录 → 刷新页面	登录态保持；`localStorage` 中 `sp_token` 存在
3	重复邮箱注册	返回 `40105` 且提示可读
4	密码强度	前端与后端**同时**拦截弱密码（`FR-USR-02` 双端校验）
5	上传非 CSV	前端即时拦截，不产生网络请求
6	上传缺列文件	返回 `40001`，指明缺哪列 + 建议
7	上传含负值文件	返回 `40003`，列出负值行号
8	加载示例数据两次	第二次返回同一 `dataset_id`（幂等）
9	参数页数据概要	L1/L2/L3 预判与 `API-09` 返回一致
10	提交预测	端到端 ≤ 60 s，返回 `task_id`
11	结果页指标	🔒 与 `outputs/eval_report.json` **完全一致**
12	覆盖率三项之和	🔒 `L1 + L2 + L3 === total`（`UX-AC-06`）
13	曲线视觉	历史实线 + 预测虚线，时间轴连续

#	检查项	通过标准
14	明细表排序分页	切换页码后排序保持（服务端排序）
15	导出 CSV	用 Excel 打开中文不乱码；列顺序与 `FR-VIS-05` 一致
16	导出越权	导出他人任务返回 `40301`（非 404）
17	历史列表	按创建时间倒序；软删除的任务不出现
18	删除任务	二次确认后列表消失，数据库中 `is_deleted = 1`
19	跨任务对比	步长不一致时显示警告
20	主题切换	图表颜色同步变化，无残留硬编码色
21	Token 过期	返回 401 → 跳登录 → 登录后参数恢复
22	无障碍	`a11y_audit.py` 退出码 0

7.5 错误码映射（前后端一致，JT-10）

前端按错误码分支处理的场景：

错误码	前端行为
`0`	正常流程
`40001`~`40010`	上传页：渲染结构化错误面板（含行/列定位 + 建议）
`40004`	上传页：提示筛选商品，禁用「开始预测」按钮
`40012`	表单域内联报错（`Form.Item` 的 `help`）
`40101` / `40102`	清 Token → 跳登录 → 保留草稿
`40104`	登录页：展示剩余冷却秒数，禁用提交按钮
`40105`	注册页：邮箱域内联报错
`40301`	全局提示「无权访问」，回退上一页
`40401` / `40402`	展示「资源不存在或已删除」空态
`40404`	全局提示「模型未就绪」，禁用提交
`50001`~`50003`	通用错误提示 + 重试按钮 + 建议提供任务 ID

7.6 跨端一致性自动校验（JT-11）

三个脚本构成**提交前的自动门**，任一失败不得提交：

```
# 1. 数据库脚本可执行且对象齐备
python scripts/verify_schema.py
#    → 9 表 / 14 索引 / 3 触发器 / 1 视图；幂等性通过

# 2. 后端模型可导入、格式约束生效、零告警
python scripts/verify_schemas.py
#    → 40 项检查通过（含 FR-USR-02 密码强度、FR-MOD-03 步长、FR-HIS-04 对比数量）

# 3. 前端类型语法正确
node scripts/verify_types.mjs
#    → types.ts 30 接口 + api-client.ts 语法通过
```

📌 JT-12：**这三个脚本的价值在于「可证伪」**。它们不是形式化的检查，而是实际执行并断言具体行为——例如 `verify_schemas.py` 会真的构造 `horizon=15` 并断言被拒绝。修改接口后若忘记同步某一端，脚本会立即失败。

---

8. 开发流程与工程规范

8.1 分支策略

本项目为课程/答辩性质的小团队项目，采用**简化 Git Flow**，不引入完整的多环境分支模型：

分支	用途	生命周期
`main`	可运行、可演示的稳定版本。**任何时**	永久

分支	用途	生命周期
	刻 `main` 都应当是可启动的	
`dev`	集成分支。功能合并到这里，通过自检后再进 `main`	永久
`feat/<模块>-<简述>`	功能开发。如 `feat/api-upload`、`feat/web-result`	功能完成即合并删除
`fix/<简述>`	缺陷修复	修复完成即合并删除

规则

编号	规则
`GIT-1`	禁止直接向 `main` 推送。`main` 只接受来自 `dev` 的合并
`GIT-2`	每个功能分支对应一个可演示的完整能力，不做「半成品合并」
`GIT-3`	合并前必须跑通 §8.5 的提交前自检（三个校验脚本 + 前后端启动）
`GIT-4`	算法层（`app/algo/`）的改动必须单独提交，并在提交信息中说明是否影响指标

⚠ `GIT-4` 的理由：`app/algo/` 是本项目最难回退的部分。若算法改动与业务改动混在一次提交里，一旦指标退化，无法快速定位是「算法改坏了」还是「数据处理改坏了」。

8.2 提交规范

采用 Conventional Commits（中文描述）：

```
<type>(<scope>): <描述>

[可选的详细说明]
```

type	含义	示例
feat	新功能	feat(api): 实现上传校验五类错误返回
fix	缺陷修复	fix(web): 修复曲线图历史段与预测段错位
docs	文档	docs: 补充 API-14 的 series_limit 说明
refactor	重构	refactor(service): 抽出导出列序常量
test	测试	test(algo): 补 L3 序列不落库的用例
chore	构建/依赖	chore: 锁定 SQLAlchemy 2.0.36
perf	性能	perf(dao): predictions 改批量插入 chunk=1000

scope 取值：api / web / algo / dao / db / docs / scripts

> GIT-5：**涉及接口的提交必须在描述中标注接口编号**，如 feat(api): API-15 结果明细分页。这让接口变更在 git log 中可检索。

8.3 代码规范

8.3.1 Python（ruff）

```
# pyproject.toml
[tool.ruff]
line-length = 100
target-version = "py312"
exclude = ["prototype", "outputs", "node_modules"]


[tool.ruff.lint]
select = [
    "E", "W",    # pycodestyle
    "F",         # pyflakes
    "I",         # isort
    "B",         # flake8-bugbear
```

```
    "UP",        # pyupgrade
    "TID",       # 🔧 flake8-tidy-imports —— 用于约束分层依赖（BE-04）
    "N",         # pep8-naming
    "SIM",       # flake8-simplify
]
ignore = ["E501"]   # 行长由 formatter 处理


[tool.ruff.lint.flake8-tidy-imports.banned-api]
# 🔒 强制分层依赖方向（BE-04）
"app.algo.train".msg = "服务进程禁止导入训练模块（FR-MOD-07）"
```

关键规范

编号	规范
PY-1	全部函数必须有类型注解（`from __future__ import annotations`）
PY-2	禁止裸 `except:`，必须指定异常类型
PY-3	禁止字符串拼接 SQL，一律走 SQLAlchemy 参数化（NFR-SEC-06）
PY-4	对外函数必须有 docstring，说明「为什么」而非「是什么」
PY-5	时间统一用 `datetime.now().strftime("%Y-%m-%d %H:%M:%S")`，禁止 `utcnow()`（NFR-COMP-04）

📌 PY-2 的实际意义：`except:` 会吞掉 `KeyboardInterrupt` 与 `SystemExit`，导致服务无法正常停止。

8.3.2 TypeScript / React（ESLint）

编号	规范
TS-1	禁止 `any`。接口数据一律使用 `types.ts` 定义的类型
TS-2	组件用函数式 + Hooks，禁止 class 组件
TS-3	禁止在组件内硬编码颜色值，一律引用 CSS 令牌（FE-02）
TS-4	所有 `useEffect` 中的异步请求必须处理竞态与卸载（FE-

编号	规范
	11）
TS-5	禁止 dangerouslySetInnerHTML（XSS 风险）
TS-6	用户可见文案一律中文，且必须含「错在哪 + 怎么改」（NFR-USE-03）

```
// .eslintrc.json（关键规则）
{
  "rules": {
    "@typescript-eslint/no-explicit-any": "error",
    "react-hooks/exhaustive-deps": "error",
    "no-restricted-syntax": [
      "error",
      {
        "selector": "Literal[value=/^#[0-9a-fA-F]{3,8}$/]",
        "message": "禁止硬编码颜色，请使用 CSS 令牌（FE-02）"
      }
    ]
  }
}
```

📌 上面这条 no-restricted-syntax 规则直接对应 UX-GOV-02「令牌是唯一真源」。它把一条设计约束变成了**编译期检查**——硬编码 #1f6feb 会导致 lint 失败。

8.4 代码评审检查单（REV-1）

评审者逐项确认，**任一项不通过不得合并**：

#	检查项	依据
1	所有查询是否带 user_id 过滤条件	🔒 NFR-SEC-02
2	是否暴露了 password_hash / Token	🔒 NFR-SEC-09
3	用户可见错误是否含「错在哪 + 怎么改」	🔒 NFR-USE-03
4	是否导入 app.algo.train	🔒 FR-MOD-07
5	levels_for_main 是否保持 ("L1",)	🔒 BE-23
6	评测路径是否用 backtest=True，线	🔒 BE-24

#	检查项	依据
	上是否 `False`	
7	`predictions` 写入是否批量	🔒 NFR-REL-04
8	是否硬编码颜色 / 路径	FE-02 / DB-01
9	接口改动是否同步了 `schemas.py` + `types.ts` + 本文档	JT-06
10	新增错误码是否已登记到 §5.2 与前端映射表	JT-10
11	CSV 导出是否带 BOM	NFR-COMP-05
12	时间是否用了本地时区（无 `utcnow`）	NFR-COMP-04

8.5 提交前自检（GIT-6）

```
#!/usr/bin/env bash
# scripts/pre_commit_check.sh
set -e


echo "== 1/6 后端代码规范 =="
ruff check app/ scripts/


echo "== 2/6 数据库脚本 =="
python scripts/verify_schema.py


echo "== 3/6 后端模型 + 格式约束 =="
python scripts/verify_schemas.py


echo "== 4/6 前端类型 =="
node scripts/verify_types.mjs


echo "== 5/6 前端代码规范 =="
(cd web && npm run lint --silent)


echo "== 6/6 后端可启动 =="
python -c "from app.main import app; print('FastAPI app loaded OK')"


echo ""
echo "✅ 全部通过，可以提交"
```

📌 GIT-6：第 6 步「可启动性检查」看似多余，但它能抓住**导入期错误**（循环导入、缺失依赖、语法错误）。这类问题在服务启动时才会暴露，而开发者往往在改完代码后直接提交就去做别的事了。

📌 R-13（2026-10-26）：**上面这段脚本已被 scripts/tasks.py 取代。**

不必再记 6 条命令，提交前只跑三条：

```
python scripts/tasks.py check    # 8 套秒级静态守卫（含模型生成件一致性、时区纪律）
python scripts/tasks.py lint     # ruff + tsc
python scripts/tasks.py test     # pytest + vitest
```

为什么是 tasks.py 而不是 Makefile：本机（Windows + Git Bash）**没有 make**，

交付一个跑不起来的 Makefile 等于交付一份没验证过的说明书。若日后要加 Makefile，

正确写法是**只做转发**（make test → python scripts/tasks.py test），逻辑仍只有一份。

端到端校验（40 套 verify_*，需模型权重与种子库）用 python scripts/tasks.py verify <名字> 单独跑。

8.6 变更管理

变更类型	处理流程	审批
新增接口	更新 §5 + app/schemas/*.py（再跑 gen_backend_schemas.py 生成 code/backend/schemas.py）+ types.ts + api-client.ts，跑校验	前后端双方确认
修改表结构	更新 schema.sql + §3 + 迁移脚本；**已有数据需评估迁移**	技术负责人
修改算法参数	更新 app/algo/config.py，**必须重跑训练与评估**，回写 §1.5 实测值	技术负责人
修改设计令牌	按 UI/UX §12.6 流程，重跑 a11y_audit.py	设计 + 产品
调整需求范围	以 PRD 第三章优先级表为唯一基准，P2 项一律延后	产品负责人

🔒 GIT-7：**修改算法参数后必须重跑这两条命令**，否则结果页展示的指标会落后于模型：

python scripts/train_and_register.py

python scripts/eval_report.py --horizon 14 --export-series

本项目历史上出现过「原型一套数字、算法另一套数字」的问题，根因就是模型更新后未同步下游展示数据。

8.7 迭代节奏建议

阶段	建议周期	交付标准
S0 基建	1–2 天	`/health` 可用，前端空布局可渲染
S1 认证	2–3 天	注册登录闭环 + Token 管理
S2 上传	3–4 天	五类校验错误全部可读可定位
S3 预测 ★	4–6 天	端到端跑通，指标与算法产出一致
S4 记录	2–3 天	历史、详情、对比、导出
S5 打磨	2–3 天	三个校验脚本全绿 + 无障碍整改完成

⚠ 上述为**按阶段顺序的建议**，具体日历天数需按实际人力确认（开发方案 §7.2 已说明任务书未给工期要求）。

---

9. 测试方案

9.1 测试分层

层	范围	工具	目标
单元测试	算法函数、工具函数、DAO	`pytest` / `vitest`	逻辑正确性
集成测试	接口端到端（含数据库）	`pytest` + `httpx`	契约与数据流
边界测试	异常输入、极限值	`pytest`	健壮性

层	范围	工具	目标
性能测试	端到端耗时、并发	自研脚本	硬指标验证
验收测试	需求追溯	人工走查	`AC-*` 逐条

📌 算法层的单元测试已由 `scripts/self_check.py` 承担（**69/69 通过**）。本节的测试主要覆盖**新增的 Web 层**。

9.2 算法层测试（已有资产）

脚本	覆盖	当前结果
`scripts/self_check.py`	预处理、特征、分级、兜底、指标、残差还原	✅ 69/69 通过
`scripts/eval_report.py`	端到端评估 + 四条硬指标判定	✅ 全部达标
`scripts/verify_schema.py`	数据库对象齐备性 + **游客态与外键完整性（6 项）**	✅ 通过
`scripts/verify_schemas.py`	后端模型、格式约束与 **bcrypt 72 字节边界**	✅ **48/48** 通过
`scripts/verify_types.mjs`	前端类型语法	✅ 通过

🔒 `TEST-1`：**算法层的任何改动都必须重跑 `self_check.py` 且保持 69/69。**它是算法正确性的基线。

9.3 集成测试要点

用例	步骤	预期
完整主流程	注册 → 登录 → 加载示例 → 参数 → 预测 → 结果 → 导出	全程无错，指标与 `eval_report.json` 一致
Token 生命周期	登录 → 用 Token 请求 → 登出 → 复用旧 Token	登出后返回 401
数据隔离	用户 A 创建任务 → 用户 B 请求该任务	返回 `40301`

用例	步骤	预期
软删除	删除任务 → 列表查询 → 数据库直查	列表不含该任务，库中 `is_deleted=1`
示例数据幂等	连续两次调用 `API-07`	返回同一 `dataset_id`
导出一致性	导出 CSV → 对照 `FR-VIS-05` 列序	7 列名称与顺序完全一致

9.4 边界与异常用例（🔒 重点）

📌 **这一节的必要性**：真实数据集（`demo_sales.csv`）的实例是 **L3 = 0**，意味着「不可预测」这条分支在常规测试中**永远不会被执行**。必须构造数据触发它，否则该逻辑等于未验证。

#	用例	构造方式	预期结果
B-01	缺 `warehouse_id` 列	删除该列后上传	`40001`，`missing_columns` 含 `warehouse_id`
B-02	缺 `sales` 列	同上	`40001`，提示不缺其他列
B-03	日期格式混乱	混入 `2015/13/45`、`20150103T99`、`未知`、空串	`40002`，给出行号与原始值（≤ 20 条）
B-04	销量负值	置一行为 -5	`40003`，指出行号
B-05	时间断档	删除中间 8 天	`40005`，`gap_days=8`；**未选择处置方式前不继续**（`FR-UPL-13`）
B-06	序列数超限	构造 1,001 个 `sku_id`	`40004`，提示当前值 1001
B-07	文件超 20 MB	用 25 MB CSV	`40007`；且**磁盘不残留临时文件**
B-08	行数超 50 万	构造 500,001 行	`40008`
B-09	空文件	0 字节 `.csv`	`40010`
B-10	非 CSV 扩展名	`.xlsx` 改名 `.csv` 内容不变	通过扩展名检查，内容校验阶段报错（**不应在扩展名阶段放行**）
B-11	**L3 序列**	构造某 `sku+warehouse` 仅有 5 个历史点	`coverage.L3 ≥ 1`；该序列在 `predictions` 中**只有 1 行占位**（`granularity_level='L3'`，`predicted_value`=历史均值），**行数 ≠ horizon**（即不

#	用例	构造方式	预期结果
			输出逐日数值预测）；覆盖率面板展示「不可预测」；主指标不计入 L3
B-12	**L2 序列**	某仓点数 < 90 但商品跨仓 ≥ 90	`coverage.L2 ≥ 1`；`l2_mape` 为 `null` 而非 `NaN`
B-13	全零销量序列	某序列销量全为 0	不崩溃（除零保护）；`mape` 不出现 `Inf`
B-14	单序列数据	只有 1 个 `sku × 1 仓`	正常完成，`series_total = 1`
B-15	中文/特殊字符	商品名含逗号、引号、空格	上传、展示、导出均正常（`NFR-COMP-06`）
B-16	超 60 秒任务	构造大量序列（接近 1000）× horizon=30	软失败：`E_TIMEOUT` + 已完成阶段；**非 502**（`NFR-REL-02`）
B-17	CUDA OOM	模拟 `torch.cuda.OutOfMemoryError`	自动回退 CPU，任务仍成功；`device_fallback=1`
B-18	无启用模型	清空 `model_registry`	启动失败（`BE-06`）；若运行中则返回 `40404`
B-19	并发 3 任务	同时提交 3 个预测	全部完成，无 `database is locked`（`NFR-PERF-09`）
B-20	越权访问	用户 B 请求用户 A 的 `task_id`	`40301`（**不是 404**，`NFR-SEC-05`）
B-21	导出越权	同上，导出接口	`40301`
B-22	路径穿越	请求 `../app.db` 作为 dataset_id	参数校验失败；**不得读取到工作区外文件**
B-23	SQL 注入	搜索框输入 `'; DROP TABLE tasks;--`	被参数化查询转义，表仍在
B-24	对比数量非法	`task_ids` 传 1 个或 5 个	`40012`（`FR-HIS-04`）
B-25	对比步长不一致	选 7 天与 30 天的任务	返回结果但 `warnings` 提示不可直接比较（`API-18-2`）

🔒 `TEST-2`：**B-11 与 B-12 是必测项**。它们覆盖的是「稀疏长尾数据」这一本项目最核心的现实情况（天池数据集约 4,815 条理论序列中大量稀疏）。若只测示例数据的 L1 路径，等于没测分级降级逻辑。

🔒 `TEST-3`：**B-16 与 B-17 必须实际模拟**，不能只靠代码审查。`E_TIMEOUT` 与 OOM 回退是 `NFR-REL-02` / `NFR-REL-03` 的验收项，且都属于「异常路径」，实际运行中难以自然出现。

📌 B-03 的举例值已于 EP-9 修正（V1.0 内勘误）：原文写「混入 2015/01/01、20150101」，但这两个值**是可解析的等价写法**，PRD FR-UPL-10 的判据是「日期列存在**不可解析**值」，PRD §3.2 字段表亦写「可解析，**推荐** YYYY-MM-DD」——推荐不等于强制。按「功能范围以 PRD 为准」的仲裁规则，放行斜杠/紧凑写法是正确的，故改为上述三个真正不可解析的值。scripts/verify_ep9_api.py 同时保留了「标准格式必须通过」的对照组，防止把 40002 做成一刀切。

9.5 服务端测试用例清单

模块	用例数	覆盖
auth	12	注册/登录/登出/限流/密码强度/邮箱唯一性/Token 过期
datasets	20	七步校验 + 五类错误 + 预览 + 模板 + 示例幂等 + 断档处置
predictions	15	创建/状态/结果/明细/导出 + 序列超限 + 超时软失败
history	10	列表分页/筛选/详情/软删除/对比/越权
system	3	健康检查三种状态
合计	**60**	——

9.6 验收检查清单（交付前逐项确认）

#	检查项	判定方式
1	Mock 开关已关闭	搜索 VITE_USE_MOCK，确认为 0 或不存在
2	原型痕迹已删除	搜索 .state-bar / .spec-note / REAL_EVAL / README.html 零命中（UX-GOV-01）
3	三个校验脚本全绿	见 §8.5
4	无障碍审计通过	python scripts/a11y_audit.py 退

#	检查项	判定方式
		出码 0（UX-AC-01）
5	结果页指标与算法一致	对照 outputs/eval_report.json（UX-AC-07）
6	覆盖率三项之和 = total	数据核对（UX-AC-06）
7	主干路径 ≤ 5 步	走查计数（UX-AC-04）
8	所有错误提示含「错在哪 + 怎么改」	人工走查 ≥ 5 类（UX-AC-05）
9	MAPE 超标警告不可折叠	交互走查（UX-AC-09）
10	L3 序列无数值预测	用 B-11 数据走查（UX-AC-10）
11	导出 CSV 中文不乱码	用 Excel 实际打开
12	四种浏览器可正常使用	Edge / Chrome / Firefox / Safari 最新版（NFR-COMP-01）。EP-9 实测前**三款各 109/109**（Edge/Chrome 走原生 CDP，Firefox 走 WebDriver BiDi）；Safari 在 Windows 无发行版，属平台限制
13	1366×768 分辨率布局正常	缩放到该分辨率走查（NFR-COMP-02）。EP-9 以 1366×768 + 1600×900 **双视口**走查 14 页 × 3 浏览器，横向溢出 **0 例**
14	免责声明全局可达	任意页面能看到页脚声明（FR-SYS-05）
15	接口文档可访问	/docs 正常渲染（FR-SYS-02）

9.7 性能测试

测试项	方法	目标
端到端耗时	示例数据集（113 序列，horizon=14）	🔒 ≤ 60 s（EP-9 实测 **11.9 s**，基线 7.87 s 的 **1.51×**）
极限规模	1,000 序列 × horizon=30	≤ 60 s（EP-9 实测 **250 s** → **E_TIMEOUT 正确软失败**：completed_stage=metrics、无半成品、HTTP 200）
并发	3 个任务同时提交	全部成功，无锁冲突（NFR-PERF-

测试项	方法	目标
		09）。EP-9 实测各自 **32.2 / 37.6 / 37.5 s**，database is locked 命中 **0** 次
首屏	Chrome DevTools Performance	≤ 3 s（NFR-PERF-06）。EP-9 实测 **FCP 中位数 Edge 140 / Chrome 140 / Firefox 367 ms**（3 次采样全有效）
预览接口	20 行预览	≤ 1 s（NFR-PERF-07）。EP-9 实测 **56 ms**
历史列表	单页 20 条	≤ 1 s（NFR-PERF-08）。EP-9 实测 **61 ms**
GPU 回退	强制 CPU 模式	≤ 60 s（NFR-PERF-10）。EP-9 实测 **23.5 s**

⏱ **EP-9 实测**（2026-10-12，本机 MX450 / 8 逻辑核）：上表括号内为 scripts/verify_ep9_perf.py 全量跑出的实测值，原始数据见 outputs/ep9_perf_report.json。

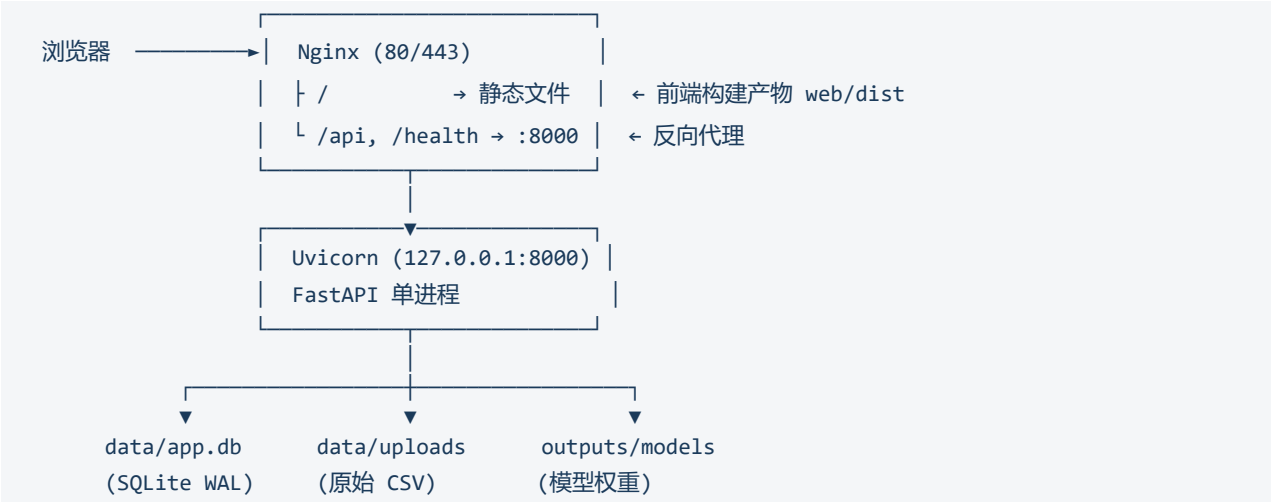
⚠ **首屏口径**：数值取自 **first-contentful-paint（FCP）**，**不是** loadEventEnd。load 事件早于 SPA 首屏绘制，且极易命中缓存而给出虚高的好数字（本项目曾测出 41~48 ms 的「假好」，换 FCP 后为 152~748 ms，中位数 504 ms）。

⚠ **极限规模走的是软失败路径**：DOD-EP9 允许「通过或正确软失败」，五项判据（非 5xx / 有错误码 / 有已完成阶段 / 无半成品 / 用户可理解）全满足。这意味着 60 s SLA 在该规模下**靠软失败兜底而非性能达标**，已登记为测试报告 P1-02。

⏱ TEST-4：性能测试的**对照基线**是 outputs/eval_report.json 中的实测值（total_ms: 7873）。新环境的实测值若超过基线 3 倍，需排查环境差异而非直接判定「性能不达标」。

10. 部署与运行

10.1 部署形态



决策	理由
Uvicorn 单进程	SQLite 写锁与模型单例加载都更适合单进程；NFR-PERF-09 只要求 3 个并发任务不崩溃
Nginx 托管静态资源	避免 Uvicorn 处理静态文件的开销；同时承担 HTTPS 终止
同源部署（Nginx 反代 /api）	生产环境**无跨域**，可移除 CORS 中间件，减少攻击面
未使用 Docker	⚠ 单机 Windows 环境部署，Docker 引入额外复杂度；若后续需要容器化，见 TQ-5

10.2 环境准备

```
# 1. 后端依赖
conda activate deploy
pip install -r requirements.txt -i https://mirrors.aliyun.com/pypi/simple/


# 2. 前端构建
cd web
```

```
npm install --registry=https://registry.npmmirror.com
npm run build          # 产出 web/dist


# 3. 数据库初始化（幂等）
sqlite3 data/app.db < code/db/schema.sql
sqlite3 data/app.db < code/db/seed.sql


# 4. 环境变量
cp .env.example .env
# 🔒 必须修改 JWT_SECRET 为随机长串
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

🔒 DEP-1：**生产环境的 `JWT_SECRET` 必须是随机值且不进入版本库。**开发期共用的默认值一旦带入生产，任何知道该项目的人都能伪造 Token。

10.3 Nginx 配置

```
server {
    listen 80;
    server_name your-domain.com;
    # 🔒 NFR-SEC-03：HTTP 强制跳转 HTTPS
    return 301 https://$host$request_uri;
}


server {
    listen 443 ssl http2;
    server_name your-domain.com;


    ssl_certificate     /etc/nginx/ssl/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;


    # 🔒 上传大小需大于业务上限 20MB
    client_max_body_size 25M;


    # 前端静态资源
    root /path/to/web/dist;
    index index.html;


    location / {
        try_files $uri $uri/ /index.html;    # SPA 路由回退
    }


    # 后端接口
    location /api/ {
```

```
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;


        # 🔒 预测任务最长 60 秒，代理超时必须大于它
        proxy_read_timeout 90s;
        proxy_send_timeout 90s;
    }


    # 健康检查（供监控）
    location /health {
        proxy_pass http://127.0.0.1:8000;
        access_log off;
    }


    # 静态资源缓存
    location ~* \.(js|css|png|svg|woff2?)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

🔒 DEP-2：**`proxy_read_timeout` 必须 > 60 秒**（设为 90s）。若用默认的 60s，预测任务恰好在临界点完成时会被 Nginx 切断，前端收到 504——恰好违反 NFR-REL-02「不得返回 502/504 原始错误」。这是最容易踩的部署坑。

🔒 DEP-3：`client_max_body_size` 必须大于业务上限（20MB），设为 25M。若设为默认的 1M，所有上传都会在 Nginx 层被拒，且**后端根本收不到请求**，排查时容易误判为前端问题。

10.4 Swagger UI 资源离线化（国内网络必需）

`/docs` 默认从 CDN 加载 Swagger UI 资源，在国内网络下会白屏。解决方式：

```
# app/main.py
from fastapi.openapi.docs import get_swagger_ui_html
from fastapi.staticfiles import StaticFiles
from fastapi import FastAPI


app.mount("/static", StaticFiles(directory="app/static"), name="static")


@app.get("/docs", include_in_schema=False)
```

```
async def custom_swagger_ui():
    return get_swagger_ui_html(
        openapi_url=app.openapi_url,
        title=f"{app.title} · 接口文档",
        swagger_js_url="/static/swagger-ui-bundle.js",    # 本地文件
        swagger_css_url="/static/swagger-ui.css",
        swagger_favicon_url="/static/favicon.png",
    )
```

需下载并放入 `app/static/`：`swagger-ui-bundle.js`、`swagger-ui.css`、`favicon.png`。

> DEP-4：FR-SYS-02 要求交付「API 接口文档」。除在线 /docs 外，还应导出静态文档归档：启动后访问 /openapi.json，用 openapi-generator 或直接保存 JSON 作为交付物。

10.5 启动与健康检查

开发环境

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

生产环境（Windows）：建议用 NSSM 或 `.bat` 脚本注册为服务；简单做法是用后台脚本：

```
:: scripts/start_server.bat
@echo off
cd /d %~dp0\..
call conda activate deploy
uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 1 >> logs\server.log 2>&1
```

健康检查（部署后第一件事）

```
curl http://127.0.0.1:8000/health
# 期望：{"code":0,"message":"ok","data":{"status":"ok","model_loaded":true,...}}
```

🔒 DEP-5：**`model_loaded` 必须为 `true`**。若为 `false`，服务虽在运行但所有预测都会失败（`40404`）。部署脚本应以此字段判定就绪，而非仅检查端口可达。

10.6 目录与磁盘

目录	挂载要求	磁盘预估
`data/app.db`	需要持久化	单任务 1,582 行数据约 200 KB；100 个任务约 20 MB
`data/uploads/`	需要持久化	按用户累计；单个数据集约 0.5–20 MB
`data/exports/`	可清理	TTL 7 天自动清理（正式导出产物）
`data/exports/tmp/`	可清理	🔒 TTL **1 小时**（临时错误明细，`JT-11`）。单文件 ≤ 20 MB（受 `MAX_FILE_SIZE_MB` 约束）
`outputs/models/`	需要持久化	单模型约 128 KB，极小

🔒 DEP-6：**磁盘告警阈值**。本机 C 盘可用空间实测约 29 GB（开发方案 R12）。建议在 `uploads/` 与 `exports/` 总量超过 5 GB 时告警。所有中间文件必须位于工作区内，禁止写系统临时目录。

⚠ DEP-7：**`data/exports/tmp/` 必须随服务启动立即创建**（等价 `mkdir -p`）。目录不存在时 `API-11` 写文件会抛 `FileNotFoundError`，而该错误只出现在「上传失败 → 点下载明细」这条低频路径上，极易漏测。

10.7 安全加固清单

#	项	实现
1	HTTPS	Nginx SSL 终止（`NFR-SEC-03`）
2	密码哈希	bcrypt rounds=12（`NFR-SEC-01`）
3	数据隔离	所有查询带 `user_id`（`NFR-SEC-02`）
4	越权返回 403	见 `get_owned_task`（`NFR-SEC-05`）

#	项	实现
5	SQL 注入	SQLAlchemy 参数化（NFR-SEC-06）
6	上传文件重命名	UUID 存储，不用原始文件名（NFR-SEC-07）
7	上传目录不可执行	Nginx 不配置该目录为静态根
8	日志脱敏	过滤 password / authorization（NFR-SEC-09）
9	生产删除演示账号	seed.sql 的 S2 段注释掉或删除
10	关闭 /docs（可选）	生产环境可设 docs_url=None，但本项目需交付接口文档，建议保留

---

11. 需求追溯与验收映射

本节是「可证伪」原则的落地：每一项需求都能追到具体的接口、数据库字段与前端组件。

11.1 功能需求追溯矩阵

PRD 需求	接口	数据库	前端
FR-USR-01 邮箱注册	API-01	users.email	Register.tsx
FR-USR-02 密码强度	API-01	——	Register.tsx + schemas.py 校验器
FR-USR-03 密码哈希	API-01	users.password_hash	——
FR-USR-04 登录签发 Token	API-02	users.last_login_at	Login.tsx + useAuthStore
FR-USR-05 登出失效	API-03	——	useAuthStore.logout
FR-USR-06 登录限流	API-02	users.fail_count / locked_until	Login.tsx 冷却提示
FR-USR-07 修改资料/密	API-04 / API-05	users.nickname	用户菜单

PRD 需求	接口	数据库	前端
码			
FR-USR-08 游客可体验	API-06 / API-07（OptionalUserId，免鉴权）	datasets.user_id = 0（哨兵，JT-10）	路由守卫允许
FR-UPL-01 拖拽上传	API-06	datasets.stored_name	DropZone.tsx
FR-UPL-02 示例数据	API-07	datasets.is_sample=1	上传页按钮
FR-UPL-03 大小/行数上限	API-06	——	precheckFile + 后端
FR-UPL-04 上传进度	API-06	——	onUploadProgress
FR-UPL-05 结构化错误	API-06	——	ValidationPanel.tsx
FR-UPL-06 错误明细下载	API-11	data/exports/	上传页下载按钮
FR-UPL-07 数据预览	API-10	——	PreviewTable.tsx
FR-UPL-08 模板下载	API-08	——	上传页链接
FR-UPL-09 表头校验	API-06	algo.REQUIRED_COLUMNS	错误面板
FR-UPL-10 日期行号	API-06	——	错误面板表格
FR-UPL-11 负值行号	API-06	——	错误面板表格
FR-UPL-12 断档区间	API-06	datasets.gap_count	断档提示卡
FR-UPL-13 断档二选一	API-06	datasets.gap_filled	确认弹窗
FR-UPL-14 序列数上限	API-06 / API-09	——	提交按钮禁用
FR-UPL-15 落盘 + 记录	API-06	datasets 全表	——
FR-FEA-01~11 特征工程	API-12（内部调用）	——	参数页开关
FR-MOD-07 预训练推理	API-12	tasks.model_version	参数页只读展示
FR-MOD-08 分级分流	API-12	predictions.granularity_level	结果页分级标签
FR-MOD-09 批量推理	API-12	——	——
FR-MOD-10 设备回退	API-13	tasks.device_fallback	执行页设备提示
FR-MOD-11 指标计算	API-14	metrics 全表	指标卡
FR-MOD-12 超标标记	API-14	metrics.is_over_threshold	不可折叠警告
FR-MOD-13 评估前提	API-14	metrics.eval_note	结果页固定文案
FR-VIS-01 折线对比	API-14	predictions.*	LineCompare
FR-VIS-02 指标卡	API-14	metrics.*	MetricCard
FR-VIS-03 明细表排序分页	API-15	predictions	ResultTable
FR-VIS-04 分级标签	API-14 / API-15	granularity_level	带色 Tag
FR-VIS-05 导出	API-16	——	导出按钮

PRD 需求	接口	数据库	前端
FR-VIS-06 超标警告	API-14	is_over_threshold	警示条
FR-VIS-07 覆盖率披露	API-14	tasks.l1/l2/l3_count	CoverageBar
FR-VIS-08 分布直方图	API-14	metrics.distribution_json	Histogram
FR-VIS-09 多序列筛选	API-14	——	序列选择器
FR-HIS-01 历史列表	API-17	视图 v_task_summary	History.tsx
FR-HIS-02 历史详情	API-19	——	HistoryDetail.tsx
FR-HIS-03 历史导出	API-16	——	详情页导出按钮
FR-HIS-04 跨任务对比	API-18	——	Compare.tsx
FR-HIS-05 软删除	API-20	tasks.is_deleted	二次确认弹窗
FR-SYS-01 统一响应	全部接口	——	api-client.ts 拦截器
FR-SYS-02 错误码体系	全部接口	——	ERROR_SUGGESTIONS
FR-SYS-03 操作日志	全部	operation_logs	——
FR-SYS-04 健康检查	API-21	——	——
FR-SYS-05 免责声明	全部页面	——	AppLayout 页脚
FR-SYS-06 HTTPS + 隔离	部署	datasets.user_id	——
FR-SYS-07 适用性说明	API-14	eval_note	结果页固定位置
FR-SYS-08 清理策略	——	data/exports/	——

11.2 非功能需求追溯

需求	实现位置	验证方式
NFR-PERF-01 端到端 ≤ 60s	predict_service 预算检查	性能测试
NFR-PERF-02 分阶段预算	algo.Timing 埋点	eval_report.json
NFR-PERF-03~05 上限	config.py 常量 + 校验	边界用例 B-06/07/08
NFR-PERF-06 首屏 ≤ 3s	ECharts 按需引入 + 路由懒加载	Lighthouse
NFR-PERF-09 并发 3 任务	WAL + 批量插入	用例 B-19
NFR-PERF-10 CPU 回退仍 ≤ 60s	resolve_device + OOM 重试	用例 B-17
NFR-SEC-01 密码哈希	security.hash_password	直查数据库
NFR-SEC-02 数据隔离	所有 DAO 带 user_id	用例 B-20
NFR-SEC-03 HTTPS	Nginx SSL	部署检查

需求	实现位置	验证方式
NFR-SEC-05 越权 403	get_owned_task	用例 B-20/21
NFR-SEC-06 SQL 注入	SQLAlchemy 参数化	用例 B-23
NFR-SEC-07 文件重命名	UUID 存储	检查文件系统
NFR-COMP-01 浏览器兼容	标准 CSS + AntD	四浏览器走查
NFR-COMP-04 本地时区	全链路 TEXT 时间	代码审查（无 utcnow）
NFR-COMP-05 CSV 带 BOM	to_csv_bytes	用 Excel 打开
NFR-COMP-07 版本锁定	requirements.txt 用 ==	重建环境验证
NFR-REL-02 软失败	completed_stage 字段	用例 B-16
NFR-REL-03 显存自愈	OOM 捕获 + CPU 重跑	用例 B-17
NFR-REL-04 WAL + 批量	PRAGMA + bulk_insert	用例 B-19
NFR-REL-05 中间文件在区内	BASE_DIR 拼接	检查路径
NFR-USE-02 默认值可用	表单 initialValues	走查
NFR-USE-03 错误可读	全部 suggestion 字段	用例 B-01~B-10
NFR-USE-05 简体中文	zhCN locale	走查

11.3 上游文档交叉引用

上游文档	引用内容	本文档章节
PRD §3.2.2	CSV 强制契约（列名、校验顺序）	§4.9.1、§5.5 API-06
PRD §3.3	三级分级规则（L1≥90 点）	§1.5、§5.6 API-14
PRD §3.4.3	指标口径（加权 MAPE + 达标占比）	§3.3 DB-T05、§4.7.3
PRD §4	全部 NFR	§11.2
PRD §5.1	18 个埋点事件	§4.11
PRD §8	验收标准 AC-*	§9.6
开发方案 §2.3	任务状态机	§3.3 DB-T03
开发方案 §4.2	分层职责边界	§2.2 BE-04
开发方案 §5.2	逻辑数据模型	§3（细化为 DDL）
开发方案 §6.2	接口约定	§5（细化为 schema）
开发方案附录 C	已确认决策（Q2/Q3/Q6）	§1.5、§4.7.3
开发方案附录 D	性能实测基准	§1.5.2

上游文档	引用内容	本文档章节
UI/UX §3	设计令牌	§6.4
UI/UX §6.1	11 个产品页	§2.3 `FE-06`
UI/UX §12.1	设计系统移植方案	§1.6
UI/UX §12.3	移除清单 `UX-GOV-01`	§1.7
UI/UX §12.5	已知问题 `UX-GOV-*`	§6.4.2

附录 A：接口字段数据字典

本节汇总 §5 中反复引用的复合结构，作为前后端共同字段的**单一出处**。配套：`code/frontend/types.ts` / `app/schemas/`（`code/backend/schemas.py` 为生成件）。

A.1 DatasetSummary 数据集概要

字段	类型	可空	数据库列	说明
`id`	integer	否	`datasets.id`	
`file_name`	string	否	`datasets.file_name`	用户原始文件名
`file_size`	integer	否	`datasets.file_size`	字节
`row_count`	integer	否	`datasets.row_count`	有效数据行数
`sku_count`	integer	否	`datasets.sku_count`	去重商品数
`warehouse_count`	integer	否	`datasets.warehouse_count`	去重仓库数
`valid_series_count`	integer	否	`datasets.valid_series_count`	序列数（`sku × wh`）
`date_start`	string	否	`datasets.date_start`	`YYYY-MM-DD`
`date_end`	string	否	`datasets.date_end`	`YYYY-MM-DD`
`gap_filled`	integer	否	`datasets.gap_filled`	0/1
`gap_count`	integer	否	`datasets.gap_count`	补齐天数合计

字段	类型	可空	数据库列	说明
`is_sample`	integer	否	`datasets.is_sample`	1 = 内置示例
`status`	integer	否	`datasets.status`	1 可用 / -1 失效
`created_at`	string	否	`datasets.created_at`	

A.2 MetricBundle 指标集

字段	类型	可空	说明	硬指标
`weighted_mape`	number	否	加权 MAPE（WAPE）	🔒 ≤ 20
`weighted_mae`	number	否	加权 MAE	——
`weighted_rmse`	number	否	加权 RMSE	——
`baseline_mape`	number	**是**	基线 MAPE	常为空
`baseline_mae`	number	否	基线 MAE（同预测点）	——
`baseline_rmse`	number	否	基线 RMSE	——
`mae_improve_pct`	number	否	MAE 相对基线降低幅度	🔒 ≥ 15
`rmse_improve_pct`	number	否	RMSE 相对基线降低幅度	🔒 ≥ 15
`series_evaluated`	integer	否	参与评估序列数（**分母**）	——
`series_passed`	integer	否	达标序列数	——
`series_pass_rate`	number	否	达标序列占比	≥ 60（辅助）
`l1_mape`	number	**是**	L1 分组 MAPE	——
`l2_mape`	number	**是**	L2 分组 MAPE，实测为 `null`	——

A.3 Coverage 覆盖率

字段	类型	说明
`L1` / `L2` / `L3`	integer	各级序列数
`total`	integer	🔒 必须等于 `L1 + L2 + L3`
`L1_pct` / `L2_pct` / `L3_pct`	number	各级占比（百分数）
`predictable`	integer	可预测序列数（L1 + L2）
`predictable_pct`	number	可预测占比

🔒 实测参考值：`{L1: 81, L2: 32, L3: 0, total: 113, L1_pct: 71.68, L2_pct: 28.32, L3_pct: 0.0}`

A.4 Distribution 逐序列 MAPE 分布

字段	类型	说明
`median`	number	中位数
`p75`	number	75 分位
`p90`	number	90 分位
`max`	number	最大值
`over_count`	integer	超阈值序列数
`threshold`	number	阈值，固定 20
`buckets`	integer[]	直方图分箱计数，🔒 分箱数 ≤ 20

A.5 TimingMs 分阶段耗时

字段	类型	实测值（ms）	预算
`preprocess_ms`	integer	675	≤ 10,000
`features_ms`	integer	763	≤ 15,000
`inference_ms`	integer	**21**	≤ 15,000
`metrics_ms`	integer	473	≤ 5,000
`total_ms`	integer	**7,873**	🔒 ≤ 60,000

A.6 SeriesCurve 序列曲线

字段	类型	说明	
key	string	${sku_id}-${warehouse_id}	
sku_id	string		
warehouse_id	string		
granularity_level	string	L1 / L2 / L3	
mape	number \	null	该序列 MAPE
passed	boolean	是否达标（≤ 20%）	
forecast_dates	string[]	长度 = horizon	
predicted	number[]	长度 = horizon	
actual	number[] \	null	无真值时为空
baseline	number[]	移动平均基线	
history_dates	string[]	折线图左侧实线段	
history_sales	number[]	与 history_dates 等长	

A.7 PredictionRow 明细行

字段	类型	可空	说明
sku_id	string	否	
warehouse_id	string	否	
forecast_date	string	否	
predicted_value	number	否	
actual_value	number	是	无真值时为空
baseline_value	number	是	
granularity_level	string	否	L1 / L2
error	number	是	abs(actual - predicted)

字段	类型	可空	说明
error_pct	number	是	误差百分比

A.8 HistoryItem 历史列表项

对应数据库视图 v_task_summary，字段见 §5.7 API-17 的响应示例。核心字段：task_id、task_name、status、horizon、dataset_name、series_total、duration_ms、device、weighted_mape、mae_improve_pct、series_pass_rate、error_code、created_at。

A.9 TaskStatusResponse 任务状态

字段见 §5.6 API-13。

附录 B：待确认技术问题清单（TQ-*）

✅ **原 10 项中，2 项阻塞级已关闭**（TQ-4 → JT-10 游客态、TQ-7 → JT-11 错误明细临时文件，见 §0.7.2）。下表保留其原始表述与决议结果，便于追溯。**剩余 8 项均不阻塞启动。**

编号	问题	建议	影响面	阻塞级别
TQ-1	是否在 users 表增加 token_version 字段？FR-USR-07 要求「修改密码后原 Token 失效并强制重新登录」，JWT 无状态，仅靠黑	**建议增加。**字段 token_version INTEGER DEFAULT 1，JWT 载荷携带该值，改密时自增	数据库 + 认	P1（不阻塞主流程，阻塞该项验

编号	问题	建议	影响面	阻塞级别
	名单无法枚举某用户的所有 Token		证逻辑	收）
TQ-2	`InferenceResult.warnings` 如何暴露给前端？算法会产出「N 条序列历史不足」等提示，但 `PredictionResultResponse` 目前无对应字段	**建议新增 `warnings: string[]` 字段**，前端在结果页以提示条展示	接口契约	P1
TQ-3	埋点采用同步写库还是异步队列？	**建议保持同步写 + 异常吞掉。**单机小规模场景下，异步会引入时序问题与丢数据风险	后端	P2
~~TQ-4~~ → ✅ **JT-10**	~~游客态如何处理？`FR-USR-08` 要求未登录可跑到结果页，但 `datasets.user_id` 为 NOT NULL~~	✅ **已决议：`user_id = 0` + `users` 表哨兵账号。**哨兵为外键完整性所必需，实测已通过。游客数据 TTL 24 小时优先清理	接口 + 数据库	✅ **已关闭**（见 §0.7.2）
TQ-5	是否需要容器化部署（Docker）？	**建议暂不引入。**单机 Windows 环境，Docker 会增加复杂度且无明确收益	部署	P2
TQ-6	`predictions` 表是否需要归档策略？极限规模下（1000 序列 × 30 天 × 多任务）会快速增长	**建议暂不做。**100 个任务约 20 MB，SQLite 完全可承受；设 5 GB 告警阈值即可	数据库	P2
~~TQ-7~~ → ✅ **JT-11**	~~错误明细（`API-11`）的临时文件存储位置与 TTL？~~	✅ **已决议：`data/exports/tmp/dataset_{dataset_id}_{upload_id}.csv`，TTL 1 小时**，与正式导出分目录	后端	✅ **已关闭**（见 §0.7.2）
TQ-8	前端是否需要引入 React Query / SWR？	**建议暂不引入。**三十余个接口、无实时推送需求，手工管理足够	前端	P2
TQ-9	结果页「多序列叠加」的默认条数与上限？	**建议默认 20 条、上限 50 条**	前端 + 接口	P1（影响 `FR-VIS-09` 性能）
TQ-10	模型历史版本保留策略？	**建议保留最近 3 个**	运维	P3

✅ **阻塞项已清零**：原先标为 P0/P1 的 `TQ-4` 与 `TQ-7` 已决议并关闭（编号升级为 `JT-10` / `JT-11`，见 §0.7.2），且两项均通过实测验证（`python scripts/verify_schema.py`）。**剩余 8 项均不阻塞启动**，可在开发过程中按建议值执行并逐步确认。

📌 本清单与 PRD 附录 A（`Q1~Q18`）、UI/UX 附录 C（`UQ-1~UQ-7`）相互独立：PRD 的问题是**需求决策**，UI/UX 的问题是**设计决策**，本清单的问题是**实现决策**。三者不重叠。

附录 C：本文档对上游的增量汇总

C.1 口径纠正（3 项，见 §0.7.1）

编号	纠正内容
`JT-01`	CSV 列名统一为**英文**（`sku_id` / `warehouse_id` / `date` / `sales`），覆盖开发方案 §6.2.2 的中文示例
`JT-02`	模型超参数分列**生产配置**（hidden=48 / layers=2 / window=21）与**基准配置**（hidden=32 / layers=1 / window=14）
`JT-03`	生产模型参数量为 **31,729**（非开发方案 R6 中提到的 5,454，后者对应基准配置）

C.2 接口增量（4 个，见 §5.3）

新增接口	对应需求	开发方案是否列出
`API-09` 数据集详情	参数页数据概要	✕ 遗漏
`API-11` 错误明细下载	`FR-UPL-06`	✕ 遗漏

新增接口	对应需求	开发方案是否列出
API-15 结果明细分页	FR-VIS-03	✗ 遗漏
API-18 跨任务对比	FR-HIS-04	✗ 遗漏

C.3 设计细化（把「逻辑模型」落到「DDL」）

开发方案 §5.2 明确留话：「⚠ 具体字段长度、索引与约束需在数据库设计文档中细化」。本文档第 3 章完成此项，产出 code/db/schema.sql：

项	开发方案 §5.2	本文档 §3
表数量	7 张（含建议新增）	**8 张**（events 独立成表）
索引	仅提到 1 个「建议索引」	**12 个**（含 3 个唯一索引、1 个部分唯一索引）
约束	未细化	**13 处 CHECK 约束** + 3 个触发器 + 1 个视图
时间类型	未明确	🔒 统一 TEXT，本地时区
软删除	提到 is_deleted	明确查询过滤规则与索引位置

C.4 技术约束补充（开发方案未覆盖）

编号	补充内容	章节
BE-13	SQLite PRAGMA foreign_keys=ON（默认关闭，不开启则外键形同虚设）	§3.9
DB-12	清理任务是唯一的全局跨用户操作，是最容易成为隔离漏洞的地方：只按时间与路径前缀判断，删游客数据时必须连 uploads/u0/ 一起清	§3.8
BE-39	/history/compare 必须在 /history/{task_id} 之前注册	§5.3

编号	补充内容	章节
BE-35	CSV 导出必须带 UTF-8 BOM	§4.10
🔒 BE-40	`hash_password` 必须先做字节校验（bcrypt 5.x 超 72 字节抛错）；`checkpw` 必须捕获 `ValueError`（哨兵哈希 `'!'` 会触发）	§4.4.1
🔒 BE-41	数据集归属判定必须按「游客数据先放行、再比 `user_id`」的顺序，两个分支不可交换	§4.3
BE-42	新数据归属一律写当前调用者身份（登录写真实 id，游客写 0），无认领迁移逻辑	§4.3
🔒 JT-12	密码上限是 **72 字节**而非 72 字符；「64 字符 + 72 字节」两道限制缺一不可	§0.7.3、§4.4.1
DEP-2	Nginx `proxy_read_timeout` 必须 > 60 秒	§10.3
DEP-3	Nginx `client_max_body_size` 必须 > 20MB	§10.3
DEP-7	`data/exports/tmp/` 必须随服务启动创建，否则 `API-11` 抛 `FileNotFoundError`（低频路径，易漏测）	§10.6
API-14-2	`l2_mape` 为 `NaN` 时必须序列化为 `null`，否则前端 `JSON.parse` 失败	§5.6
FE-09	主题必须在首屏渲染前应用，否则闪白	§6.2
FE-13	深色主题必须显式写回所有语义色令牌	§6.4.1

附录 D：配套资产与校验证据

D.1 配套文件清单

文件	规模	验证方式	验证结果
`code/db/schema.sql`	358 行 / 18,722 字符	`python scripts/verify_schema.py`	✅ 9 表、14 索引、3 触发器、1 视图；幂等；含游客哨兵与外键完整性实测 6 项
`code/db/seed.sql`	116 行	随 schema 执行	✅ `model_registry` 注册真实模型数据；游客哨兵幂等补插
`app/schemas/`（7 模块） `code/backend/schemas.py`（生成件 1,089 行）	**64 个 Pydantic 模型**（含 6 条 bcrypt 字节上限回归）	`verify_schemas.py` + `gen_backend_schemas.py --check`	✅ **61/61 通过，零告警**；生成件与真源逐字一致
`code/frontend/types.ts`	574 行 · 5 类型 + 30 接口 + 8 常量 + 2 函数	`node scripts/verify_types.mjs`	✅ 语法通过（TypeScript 5.4.5）
`code/frontend/api-client.ts`	266 行 · 4 函数 + 7 常量	同上	✅ 语法通过

D.2 校验脚本

脚本	用途	复现命令
`scripts/verify_schema.py`	数据库脚本可执行性、对象齐备性、幂等性、约束有效性，**以及游客哨兵与 `user_id = 0` 的可写性、非法 `user_id` 的被拒性**	`python scripts/verify_schema.py`
`scripts/verify_schemas.py`	后端模型导入、格式约束生效、常量同源性，**以及 bcrypt 72 字节边界（含精确 72 / 73 字节）**	`python scripts/verify_schemas.py`
`scripts/verify_types.mjs`	前端类型语法（transpileModule，无需安装依赖）	`node scripts/verify_types.mjs`

D.3 本文档引用的实测数据来源

数据	来源文件	复现命令
端到端耗时 7,873 ms、加权 MAPE 14.26%、覆盖率 113=L1 81+L2 32+L3 0	`outputs/eval_report.json`	`python scripts/eval_report.py --horizon 14`
模型超参（window=21 / hidden=48 / layers=2）、参数量 31,729、验证 MAPE 14.07%	`outputs/models/lstm_v1.report.json`	`python scripts/train_and_register.py`
示例数据集 14,305 行 / 40 商品 / 5 仓	`outputs/models/demo_sales.csv`	直接读取
依赖版本（torch 2.9.1+cu128 / pandas 2.3.3 / numpy 2.5.3 / fastapi 0.141.1 / pydantic 2.9.2）	本机 deploy 环境	`pip list`
逐序列曲线（113 条 × 14 天，含真值与基线）	`outputs/eval_series.json`	`python scripts/eval_report.py --export-series`

🔒 **数据同源约束**：本文档中出现的所有性能与效果数值，均来自上述文件，**无手工编造值**。若模型重新训练，须重跑命令并同步更新 §1.5 与相关章节（GIT-7）。

D.4 文档与代码的同步责任

变更内容	必须同步更新
接口字段	`schemas.py` → `types.ts` → `api-client.ts` → 本文档 §5 → 跑校验
表结构	`schema.sql` → 本文档 §3 → 跑 `verify_schema.py`
算法参数	`algo/config.py` → 重训 → 重跑评估 → 本文档 §1.5
设计令牌	`tokens.css` → 本文档 §6.4 → 跑 `a11y_audit.py`

文档结束

项	内容
文档版本	V1.0
完成日期	2026-10-09
配套代码资产	`code/` 目录 5 个文件，全部通过实际执行验证
校验脚本	`scripts/verify_{schema,schemas,types}` 三个，构成提交前自动门
待确认问题	10 项（`TQ-1`~`TQ-10`），其中 **2 项阻塞级已关闭**（`JT-10` / `JT-11`），剩 8 项非阻塞
下一步	✅ 阻塞项已清零（`TQ-4`→`JT-10`、`TQ-7`→`JT-11`，见 §0.7.2）→ 直接按 §7.2 的 S0~S5 阶段顺序进入编码